

TÜBİTAK BİLGEM

MA3 API

İçindekiler

1. AÇIK ANAHTAR ALTYAPISI (AAA)	8
1.1 Giriş.....	8
1.2 Sertifikalar	8
1.2.1 Nitelikli Sertifika.....	9
1.2.2 Sertifika Bütünlüğünün Korunması	10
1.3 Sertifika İptal Listesi (SİL).....	10
1.4 Sertifika Makamı	11
1.5 Çevrimiçi Sertifika Durum Protokolü (ÇİSDUP)	11
1.6 Akıllı kartlar	12
2. ELEKTRONİK İMZA	14
2.1 İmza Tipleri	14
2.1.1 CAdES-BES	15
2.1.2 CAdES-EPES	15
2.1.3 CAdES-T (Zaman Damgası Eklenmiş Elektronik İmza).....	16
2.1.4 CAdES-C (Tüm Doğrulama Verilerinin Referanslarının Eklendiği İmza).....	17
2.1.5 CAdES-X-LONG (Genişletilmiş Uzun Elektronik İmza)	18
2.1.6 CAdES-X-Type 1 (Genişletilmiş Elektronik İmza Tip 1 Zamanlı).....	18
2.1.7 CAdES-X-Type 2 (Genişletilmiş Elektronik İmza Tip 2 Zamanlı).....	19
2.1.8 CAdES-X-Long-Type 1 or Type 2 (Genişletilmiş Uzun Elektronik İmza Tip 1 ve Tip 2 Zamanlı)	19
2.1.9 CAdES-A (Arşiv Elektronik İmza)	20
2.2 İmza Profilleri	21
2.2.1 P1: Anlık - İmza Profili	21
2.2.2 P2: Kısa Süreli - İmza Profili	21
2.2.3 P3: Uzun Süreli - İmza Profili.....	21
2.2.4 P4: Uzun Süreli - İmza Profili.....	22
3. SERTİFİKA DOĞRULAMA.....	23
3.1 Giriş.....	23
3.2 Gerekler	23
3.3 Sertifika Doğrulama	23

3.3.1 Sertifikalar	23
3.3.2 Çevrimiçi Sertifika Durum Protokolü (ÇİSDUP)	24
3.3.3 SİL Dosyaları	25
3.3.4 X.509 Sertifika ve SİL Doğrulama	25
3.3.4.1 Sertifika Zinciri Oluşturma	26
3.3.4.2 Sertifika Zinciri Doğrulama	27
3.4 Sertifika Doğrulama Kütüphanesi Bileşenleri	29
3.4.1 Kontrolcüler	30
3.4.1.1 Yapısal Kontroller	30
3.4.1.2 Zincir İlişkisi Kontrolleri	31
3.4.1.3 İptal Kontrolleri	31
3.4.1.4 İsim Kontrolleri	31
3.4.1.5 Politika Kontrolleri	32
3.4.2 Bulucular	32
3.4.2.1 Güvenilir Sertifika Bulucular	32
3.4.2.2 Sertifika Bulucular	32
3.4.2.3 SİL Bulucular	32
3.4.2.4 OCSP Cevabı Bulucular	33
3.4.3 Eşleştiriciler	33
3.4.3.1 Sertifika Eşleştiriciler	33
3.4.3.2 SİL Eşleştiriciler	33
3.4.3.3 OCSP Cevabı Eşleştiriciler	33
3.4.3.4 Delta SİL Eşleştiriciler	33
3.4.3.5 Çapraz Sertifika Eşleştiriciler	34
3.4.4 Kaydediciler	34
3.4.5 Sertifika Doğrulama Politikası	34
3.4.5.1 Sertifika Doğrulama Politika Dosyasının Güvenliği	35
3.4.6 Sertifika - SİL Durum Bilgisi	36
3.4.7 XML Sertifika Deposu	36
3.5 Sertifika Doğrulama Kütüphanesi Kullanımı	38
3.6 Sertifika Doğrulama Sonucunun Yorumlanması	38
3.7 Politika Dosyası	39

3.7.1 Politikanın Çalışma Zamanında Düzenlenmesi	39
3.8 Politika Dosyası Elemanları	40
4. CMS İMZA	56
4.1 Giriş.....	56
4.2 Gereklere	56
4.3 İmza Tipleri	56
4.4 İmza Atma İşlemleri	57
4.5 İmzasız Bir Verinin İmzalanması.....	57
4.6 İmzalı Bir Veriye İmza Eklenmesi.....	59
4.6.1 Paralel İmza	59
4.6.2 Seri İmza	61
4.7 Ayırık İmza.....	62
4.8 Farklı İmza Tiplerinin Oluşturulması.....	64
4.8.1 BES	64
4.8.2 EST	66
4.8.3 ESXLong	67
4.8.4 ESA	67
4.9 Zorunlu Olmayan Özelliklerin Eklenmesi	68
4.10 Sertifika Doğrulama	71
4.11 İmza Doğrulama İşlemleri.....	71
4.12 İmza Doğrulama Sonucu	73
4.13 Ön Doğrulama	74
4.14 Ayırık İmzanın Doğrulanması	75
4.14.1 Ayırık İmzanın Bütünleşik İmzaya Çevrilmesi	76
4.15 Sertifika Doğrulama	76
4.15.1 İmzacıların Alınması	76
4.16 İmza Tipleri Arasında Dönüşüm.....	77
4.17 İmza Zamanının Belirlenmesi.....	79
4.18 Zaman Damgası	79
4.18.1 İmzadaki Zaman Damgasından İmza Zamanının Alınması.....	80
4.18.2 Zaman Damgası Sunucusunun Test Edilmesi	81
4.18.3 Zaman Damgası Alma	82

4.19 Parametreler	83
5. XML İMZA	86
5.1 Giriş.....	86
5.2 XML İmza Çeşitleri	86
5.2.1 Yapısına Göre	86
5.2.2 İmza Tipleri.....	87
5.3 İmza Atma.....	87
5.3.1 Detached	87
5.3.2 Enveloping	88
5.3.3 Enveloped	88
5.4 İmza Doğrulama	89
5.5 Akıllı Kart İşlemleri	89
5.6 XML İmza Konfigürasyonu	90
5.6.1 Yerelleştirme	90
5.6.2 Proxy Ayarları	90
5.6.3 Kaynak Çözücüler.....	90
5.6.4 Zaman Damgası	91
5.6.5 Varsayılan Algoritmalar.....	91
5.6.6 Doğrulama Parametreleri.....	91
5.6.7 Doğrulayıcılar	93
5.7 Sertifika Doğrulama	93
5.8 XAdES Çoklu İmza Atma	93
5.8.1 Paralel İmza	94
5.8.2 Seri İmza	95
5.8.3 P1: Anlık - İmza Profili	97
5.8.4 P2: Kısa Süreli - İmza Profili	97
5.8.5 P3: Uzun Süreli - İmza Profili.....	98
5.8.6 P4: Uzun Süreli - İmza Profili.....	98
5.8.7 Arşiv İmza	99
6. PAdES İMZA.....	100
6.1 Giriş.....	100
6.2 ES_BES İmza Atma	101

6.3 İmza Doğrulama	101
6.4 ES_T İmza Atma	101
6.5 LTV (ES_A) İmza Atma	102
7. ASİC E-İMZA	103
7.1 Giriş.....	103
7.2 Gerekler	103
7.3 Kavramlar	103
7.4 Anahtar API Arayüzleri ve Tasarım.....	104
7.5 Kütüphane Kullanımı	104
7.5.1 Basit Paket Oluşturma	104
7.5.2 Çoklu Paket Oluşturma.....	104
7.5.3 Paket Doğrulama	105
7.5.4 İmza Geliştirme	105
8. AKILLI KART	106
8.1 Giriş.....	106
8.2 Gereksinimler.....	106
8.3 Akıllı Karta Erişim	106
8.4 Akis Kartlara Erişim	107
8.5 Akıllı Karttan Sertifikanın Okunması	108
8.6 Akıllı Karttaki Nesne Adlarının Okunması.....	110
8.7 Akıllı Kartta İmzalama - Şifreleme İşlemlerinin Yapılması	110
8.8 Akıllı Kart Kütüphanesi Konfigürasyonu.....	111
8.9 SmartCardManager Sınıfı	113
8.10 SmartCard Modülü ile BES Tipi İmza Atılması	114
9. MOBİL İMZA	116
9.1 Mobil İmza İstemci Tarafı	116
9.2 Mobil İmza Sunucu Tarafı	117
10. ANDROID'DE İMZA ATMA.....	119
Ek A. Hızlı Başlangıç.....	122
Ek B. Lisans Ayarları	123
1. Bakım Sözleşme Bitiş Tarihi.....	123
Ek C. Parola Tabanlı Şifreleme	124

Ek D.	Log Tutma.....	125
Ek E.	SÖZLÜK.....	128

1. AÇIK ANAHTAR ALTYAPISI (AAA)

1.1 Giriş

Açık anahtar altyapısı sistemleri bize kimlik doğrulama, inkar edememezlik, mesaj bütünlüğü ve gizlilik gibi hizmetleri simetrik kriptografinin kullanıldığı bir yöntemle sunarlar. Böylece anahtar dağıtımı ve elektronik imza özelliklerini asimetrik kriptografinin, gizlilik hizmetini simetrik kriptografinin sağladığı güvenli bir altyapıya kavuşabiliriz.

Açık anahtar altyapısına aşağıdaki iki sebepten dolayı ihtiyaç duyulur:

- Asimetrik kriptografi sistemlerini gerçeklemek
- Açık ve özel anahtar yönetmek

Bu bölümde açık anahtar altyapısına ait temel bilgiler ele alınacaktır. Açık anahtar altyapısının en önemli ürünü olan sertifikalardan bahsedilecek ve Basit Sertifikalar, Açık Anahtar Sertifikaları gibi konu başlıklarında sertifikanın detaylı tanımı verilecektir. Sertifika İptal Listeleri, Sertifikasyon Prensipleri ve Sertifikasyon Yolu gibi kavramlar da bu bölümde sırasıyla işlenecektir.

1.2 Sertifikalar

Asimetrik kriptografide bir kişi için üretilen anahtar çifti, özel ve açık anahtardan oluşur. Bu anahtarlardan açık olanı anahtarın sahibiyle haberleşmek isteyen herkes tarafından görülebilir ve kullanılabilir. Bu açık anahtarın isteyen kişilerce kullanımını kolaylaştırmak için değişik şekillerde yayınlanması ve isteyenlerin erişimine açılması mümkündür. Bu yayınlama şekline sertifika adı verilmektedir.

Açık anahtarı yayınlamak için kullanılacak ideal bir sertifika aşağıdaki özellikleri taşımalıdır:

- Elektronik ortamda (örneğin: internette) yayınlanabilmesi ve otomatik olarak işlenebilmesi için tamamen sayısal olmalıdır.
- Özel anahtarın sahibinin adını, çalıştığı şirketin/kurumun adını ve irtibat kurmak için gerekli bilgileri içermelidir.
- Sertifikanın ne zaman yayınlandığını anlamak kolay olmalıdır.
- Özel anahtar, sahibi tarafından değil güvenilir bir 3. kurum tarafından yaratılmalıdır.
- Güvenilen kurum birçok sertifika yaratacağı için (aynı kullanıcı için bile birden fazla) her bir sertifikanın diğerinden kolayca ayırt edilebilmesi gereklidir.
- Bir sertifikanın gerçek veya sahte olduğu kolayca tespit edilebilmelidir.
- Değiştirilmeye karşı korunmuş olmalıdır.
- İçindeki bilgilerin güncel olup olmadığı istendiği anda tespit edilebilmelidir.
- Hangi uygulamalar için kullanılabileceği sertifikanın içinde yazmalıdır.

İdeal sertifika tanımına uygun bir elektronik sertifika oluşturabilmek için uluslararası ITU kurumu X.509 standardını tanımlamıştır. Bu özellikleri taşıyan örnek bir sertifika aşağıdaki gibi olabilir:

Seri No	2368
Sertifika Sahibi	Ahmet Uzun-43657465098
Şirket/Kurum	Uzun Finans A.Ş.
Yayınlayan	KamuSM
Yayın Tarihi	05.02.2018
Son Kullanım	05.02.2023
Açık Anahtar	2489349e894859f45489450dab45454ca0908d8809
KamuSM Elektronik İmzası	
ae89349c989893e8989548d0823048b08023f9e903	

Şekil 1: Örnek Sertifika

1.2.1 Nitelikli Sertifika

Nitelikli Sertifika (Qualified Certificate), X.509 Sertifikası baz alınarak hazırlanan ve sadece gerçek kişilere verilen bir sertifika çeşididir. Bu sertifika tipi RFC 3739'da tanımlanmıştır. Nitelikli sertifikalar Türkiye'de ve birçok Avrupa ülkesinde elle atılan ıslak imzaya eşdeğer elektronik imzalar atmak için kullanılır. Bu sertifikaları standart X.509 sertifikasından farklı kılan **en önemli** özellik, üretiminde ve sahibine verilmesinde çok sıkı kimlik doğrulama kurallarının uygulanması ve sertifika merkezlerinin işletiminin denetlenmesidir.

Nitelikli sertifika alanları ile ilgili önemli bilgiler aşağıda verilmiştir.

Yayınlayıcı adı (Issuer Name), aşağıdakilerin bir alt kümesidir:

domainComponent, countryName, stateOrProvinceName, organizationName, localityName, serialNumber.

Kullanıcı adı (Subject Name), aşağıdakilerin bir alt kümesidir:

countryName, commonName, surname, givenName, pseudonym, serialNumber, organizationName, organizationalUnitName, stateOrProvinceName, localityName, postalAddress.

1.2.2 Sertifika Bütünlüğünün Korunması

X.509 Sertifikasının bütünlüğünün korunması için sertifika içinde yer alan tüm bilgiler sertifikayı veren makam tarafından elektronik olarak imzalanır ve oluşan elektronik imza bu sertifikanın arkasına eklenir. Böylece sertifika alanlarından herhangi birisi üzerinde sonradan değişiklik yapıp yapılmadığı kontrol edilebilir. Bir elektronik sertifikanın üstündeki elektronik imza doğrulanarak sertifikanın geçerliliği kontrol edilir.

1.3 Sertifika İptal Listesi (SİL)

Sertifika sahibi sertifikasını kullanmak isteyen kişilere dağıttıktan sonra geri toplayamaz. Ayrıca sertifika sahibinin kendisi ile ilgili değişiklikleri duyurması çok zordur. Bu nedenle sertifika iptal listeleri (SİL) kullanılır.

Sertifika iptal listeleri aşağıdaki özellikleri taşır:

- Sayısaldır.
- Artık güvenilemeyecek olan ve kullanım süresi dolmamış sertifikaların seri numaralarını içerir.
- Yayın tarihini ve son kullanım tarihini içerir.
- Yayınlayan kuruluşun adını ve sayısal imzasını içerir.
- Sık aralıklarla elektronik ortamda (örneğin internette) yayınlanır.

Örnek bir sertifika iptal listesi aşağıdaki gibi olabilir:

Yayınlayan	KamuSM
Yayın Tarihi	22.02.2018
Son Kullanım	23.02.2018
İptal Olan Sertifikaların Listesi 55, 678, 2164, 3403, 4034, 5677	
KamuSM Sayısal İmzası	6656e345200cde989228d082 3aec8b08023f9

Şekil 2: Örnek SİL

AAA sisteminde, sertifika tabanlı işlem yapan (kimlik doğrulama, elektronik imza doğrulama vb.) her türlü yazılım ve donanım, kullanacağı sertifikaların geçerliliğini kontrol ederken güncel bir sertifika iptal listesine bakmak zorundadır.

Eğer işlemde kullanılacak bir sertifikanın seri numarası SİL içinde bulunursa o sertifika geçersiz kabul edilir. Açık anahtar sertifikasının içeriğinin güncel olup olmadığı sorusuna cevap vermek gerekmektedir. Çünkü;

- Sertifika sahibinin erişim bilgileri değişmiş olabilir.
- Sertifika sahibi özel anahtarını kaybettiği için yeni bir açık anahtar kullanmaya başlamış olabilir.

1.4 Sertifika Makamı

Sertifika makamı (SM), açık anahtar altyapısında yer alan sertifikaları ve sertifika iptal listelerini üretmekle görevli olan merkezi birime verilen addır. Sertifika makamı, donanım ve yazılım parçalarından ve sistemi işleten kişiler ve kurallardan oluşan bir yapıdır. Bir sertifika makamını diğer sertifika makamlarından ayırt eden özellikleri; adı ve anahtar çiftidir (açık ve özel anahtardan oluşan).

Sertifika makamının görevleri şunlardır:

- Sertifika yayınlamak
- Sertifika durum bilgilerini güncel tutmak ve sertifika iptal listeleri (SİL) hazırlamak
- Güncel sertifikaları ve SİL'leri isteyen kişilere sunmak
- Süresi dolan ya da iptal edilen sertifikaların arşivini tutmak

Açık anahtar altyapısının ürettiği sertifikaları ve anahtarları kullanan kişiler bu yapının önemli bir parçasını oluştururlar. Sertifika kullanıcıları, açık anahtar altyapısını kullanırken sertifika makamına sertifika talebinde bulunurlar.

Sertifika kullanıcıları aldıkları sertifika ile bir özel ve bir açık anahtar sahibi olurlar ve elektronik imza, simetrik anahtar değiş tokuşu, kimlik doğrulama gibi işlemleri yaparlar.

Sertifika kullanıcıları;

- Sertifikalarını yayınlayan SM'yi güvenilen taraf olarak kabul ederler.
- Sertifikaları kullanarak karşı tarafın sayısal imzasını ve kimliğini doğrulayabilirler.
- İletişim kurmak istedikleri kişinin sertifikasına ve SM'nin yayınladığı SİL'lere ulaşabilirler.

1.5 Çevrimiçi Sertifika Durum Protokolü (ÇİSDUP)

SİL'lerin bir periyot boyunca geçerli olması, örneğin 24 saatte bir yenilenmesi, bu periyot boyunca iptal edilen sertifikalardan haberdar olmayı geciktirmektedir. Kullanıcılar iptal olan bir sertifikayı bir sonraki periyotta yayınlanan SİL içinde görebilmektedirler. Finansal işlemler gibi anlık doğrulamaya ihtiyaç duyulan uygulamalarda SİL yöntemi yeterince güvenlik sağlamamaktadır. Bu nedenle çözüm olarak Çevrimiçi Sertifika Durum Protokolü (ÇİSDUP) kullanılmaktadır (OCSP-Online Certificate Status Protocol).

Bu protokol şu şekilde çalışır:

Kullanıcı isteği:

10 numaralı sertifikanın durumu nedir?

ÇİSDUP sunucusu yanıtı (Aşağıdakilerden biri olabilir):

Bu sertifikanın durumu;

- İyi (İptal edilmemiş)
- Kötü (İptal edilmiş)
 - İptal nedeni
 - İptal zamanı
- Bilinmiyor

Her SM'ye ait bir veya birden fazla ÇİSDUP sunucusu olabilir. ÇİSDUP sunucusu bağlı olduğu SM'nin yayınladığı sertifikaların iptal edilip edilmediği bilgisine ulaşır ve kendisine gelen kullanıcı isteklerini cevaplar. ÇİSDUP cevap mesajları elektronik imza ile imzalanarak güvenlik sağlanır.

1.6 Akıllı kartlar

X.509 Sertifikalarını ve bunlara bağlı olan anahtarları taşımak için kullanılan en yaygın ve güvenli cihazlar akıllı kartlardır. Akıllı kartlar, programlanabilir alanları olan, dayanıklı, taşınabilir bilgisayarlardır. Akıllı kartlar veri güvenliği, kimlik gizliliği ve mobil kullanıcı ihtiyaçlarına sahip sistemlerde faydalıdır. Bu kartların kriptolojik işlemci taşıyan modelleri aşağıdaki özellikleri taşır:

- Kart üzerinde şifreleme ve şifre çözme
- Kart üzerinde imzalama ve imza onaylama
- Kart üzerinde özel ve açık anahtarların tutulması
- Kart içine bilgi yazabilme
- Kartın şifre ile korunması



Şekil 3: Sim kart boyutunda bir akıllı kart

Akıllı kartların özel ve genel alanları vardır. Özel alanda anahtar üretimi, imzalama, şifre çözme gibi işlemler yapılır. Bu alana erişim yasaktır. Genel alanda ise erişime açık genel bilgiler bulunur.

Bir AAA sistemi tarafından kullanılan akıllı kartta olması gerekenler şu şekilde sıralanabilir:

- İmzalama özel anahtarı
- Şifreleme özel anahtarı
- O an geçerli olan imzalama sertifikası
- O an geçerli olan şifreleme sertifikası
- Daha önce geçerli olan şifreleme özel anahtarları ve karşılığı olan sertifikalar

2. ELEKTRONİK İMZA

5070 sayılı Elektronik İmza Yasası'nda güvenli elektronik imza, "Münhasıran imza sahibine bağlı olan, sadece imza sahibinin tasarrufunda bulunan güvenli elektronik imza oluşturma aracı ile oluşturulan, nitelikli elektronik sertifikaya dayanarak imza sahibinin kimliğinin tespitini sağlayan, imzalanmış elektronik veride sonradan herhangi bir değişiklik yapıp yapılmadığının tespitini sağlayan imzadır." şeklinde tanımlanmaktadır.

Elektronik imza kimlik doğrulama ve onaylama, veri bütünlüğü ve inkar edilemezlik olmak üzere üç temel özelliği sağlamaktadır.

- **Kimlik doğrulama ve onaylama**, imzalanan dokümanın mesaj sahibine ait olduğunu ve geçerliliğini sağlar.
- **Veri bütünlüğü**, imzalanan verinin değiştirilmesini, silinmesini veya veriye ekleme-çıkarma yapılmasını önler.
- **İnkâr edilemezlik**, kişi veya kurumların elektronik ortamda gerçekleştirdikleri işlemleri inkar etmelerini önler.

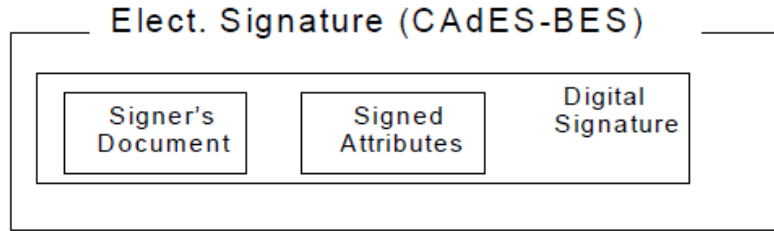
Elektronik imza oluşturmak için sertifika ve özel anahtara sahip olmak gerekmektedir. Bu özel anahtar, yukarıda bahsedilen güvenli elektronik imza oluşturma aracında genel olarak akıllı kart içinde saklanmaktadır.

2.1 İmza Tipleri

Bu bölümde, basitten karmaşığa doğru yapılarına göre e-imza tipleri açıklanmaktadır. İmza tiplerinin detayı için ETSI TS 101733 dokümanına bakılabilir. Açıklamalarda ETSI TS 101733 dokümanında yer alan şekillerden faydalanılmıştır.

2.1.1 CAdES-BES

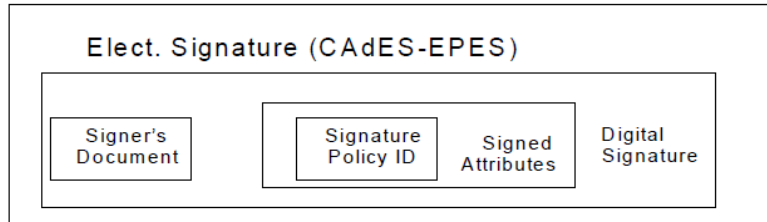
BES imza tipi en basit şekli ile oluşturulan elektronik imzadır. Şekil 4'te de görüldüğü gibi imza dosyası içeriğinde imzalanan belge ve imzaya eklenen diğer imza özellikleri (signed attributes) ile birlikte imzanın kendisi tutulur. BES imzada imza zamanı ile ilgili herhangi bir bilgi yoktur. Bu yüzden uzun süre saklanması gereken e-imzalı belgeler için güvenli kabul edilen bir yöntem değildir. İmzada kullanılan sertifikanın kalan geçerlilik süresinden daha uzun süre saklanması gereken belgeler BES imza ile imzalanmamalıdır. Aksi durumda imzada kullanılan sertifikanın süresi dolduktan sonra geçmişte oluşturulan imzalar güvenilir bir şekilde doğrulanamayacaktır.



Şekil 4: CAdES-BES İmza Yapısı

2.1.2 CAdES-EPES

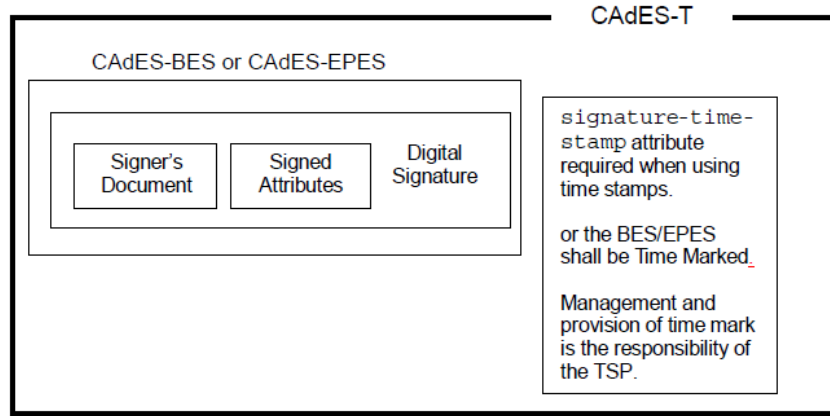
BES imza tipine çok benzer yapıdadır. BES'ten tek farkı Şekil 5'te de görüldüğü gibi imza dosyası içeriğine imza politikasını belirten bir imza özelliği (signed attributes) eklenmesidir. EPES imza tipinin kullanılabilmesi için imzanın daha önceden tanımlanmış bir politikaya uygun bir biçimde oluşturulması gerekmektedir. İmzanın belirtilen politikaya uygunluğu imza dosyası içeriğine konulan imza politika numarası (Signature Policy ID) ile belirlenir.



Şekil 5: CAdES-EPES İmza Yapısı

2.1.3 CAdES-T (Zaman Damgası Eklenmiş Elektronik İmza)

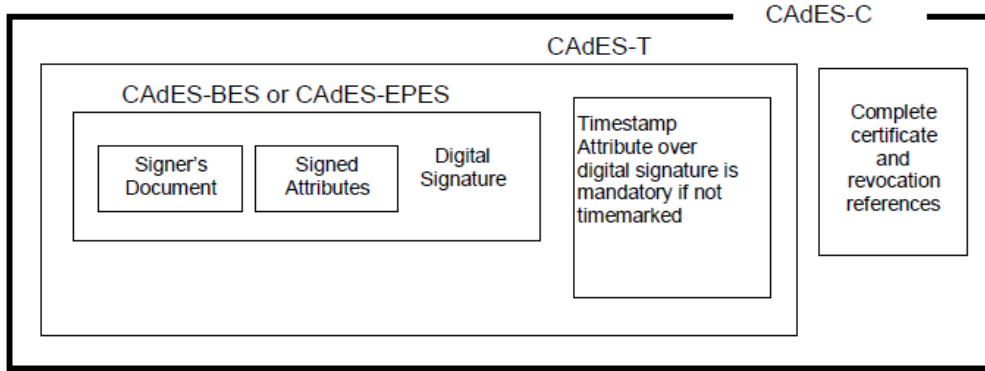
BES veya EPES üzerine kurulan bir imza tipidir. Şekil 6'da da görüldüğü gibi BES veya EPES imzaya zaman damgası alınarak oluşturulur. Zaman damgası imzanın oluşturulduğu tarihi belirlemek için kullanılır. Zaman damgasının, Türkiye'de yetkilendirilmiş bir ESHS'den alınması zorunludur. İmza doğrulanırken zaman damgası üzerinde bulunan zaman bilgisi referans alınarak doğrulama işlemi gerçekleştirilir. İmzanın zaman damgası, üzerindeki zaman bilgisinden önceki bir tarihte oluşturulmuş olduğu kesin bir şekilde tespit edilebilir. Uzun dönem saklanması gereken e-imzalı belgelerin mutlaka CAdES-T imza tipinde oluşturulması gerekmektedir. İmzalama işlemi yapıldıktan hemen sonra imzaya zaman damgası alınması önerilmektedir.



Şekil 6: CAdES-T İmza Yapısı

2.1.4 CAdES-C (Tüm Doğrulama Verilerinin Referanslarının Eklendiği İmza)

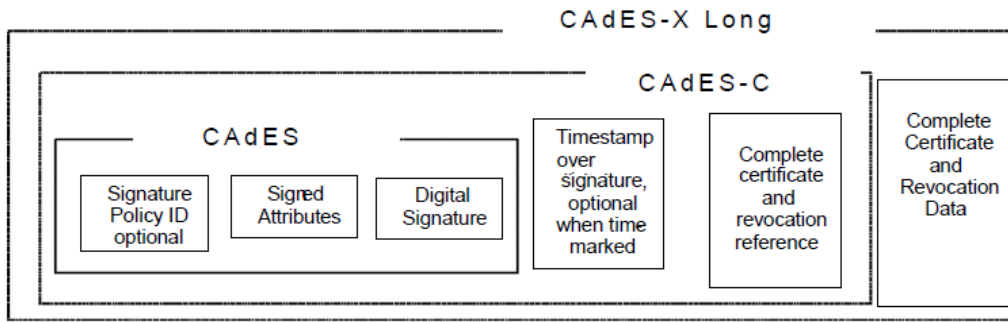
Şekil 7'de görüldüğü gibi ES-T imzası üzerine kurulan bir imza tipidir. İmza doğrulamada kullanılan ESHS'ye ait kök ve alt kök sertifikaları ile sertifikaların iptal kontrollerinin yapıldığı SİL (Sertifika İptal Listesi) veya OCSP cevaplarının referans değerleri, imza dosyasına eklenir. Sertifika ve iptal bilgileri imza dosyasına eklenmez, sadece bu bilgilere erişim için tanımlanmış eşsiz referans değerleri imza dosyasına eklenir. İmza doğrulama yapılırken eklenen referans değerleri ile ilgili sertifikalar, SİL ve OCSP cevaplarının dışarıdaki bir sistemden temin edilmesi ve imza doğrulamanın bu veriler kullanılarak yapılması gerekir. İmza dosyasına eklenen referans değerleri ile ilgili sertifikalar, SİL ve OCSP cevapları imza doğrulama sistemi içinde ortak bir veri tabanı veya dizin yapısı içinde tutulabilir. ES-C imza tipinin kullanılması önerilmemekle beraber kullanımının uygun olduğu durumlar da olabilir.



Şekil 7: CAdES-C İmza Yapısı

2.1.5 CADES-X-LONG (Genişletilmiş Uzun Elektronik İmza)

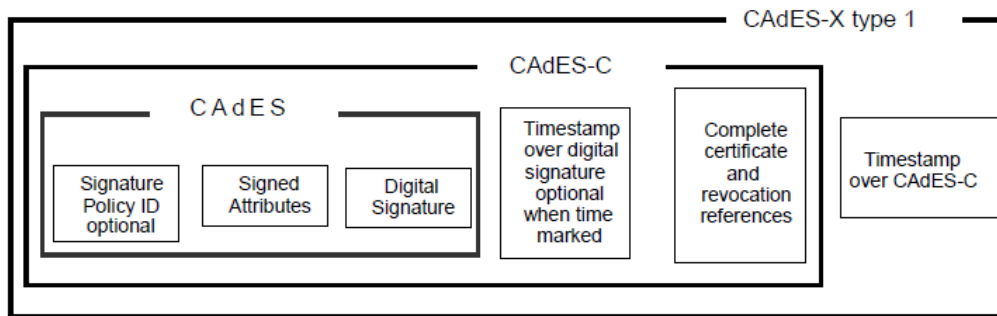
Şekil 8'de görüldüğü gibi ES-C imzası üzerine kurulan bir imza tipidir. İmza doğrulamada kullanılan ESHS'ye ait kök ve alt kök sertifikaları ile sertifikaların iptal kontrollerinin yapıldığı SIL veya OSCP cevapları imza dosyasına eklenir. İmza doğrulama yapılırken imza dosyasına eklenmiş bu veriler kullanılır. İmza doğrulama için dışarıdaki herhangi bir sisteme bağlanıp doğrulama verilerinin edinilmesine gerek kalmaz; doğrulama için gereken tüm veriler imza dosyasının içeriğinden edinilir. İmza doğrulaması yapacak tarafın doğrulama verilerini bulmasını gerektirmediğinden kullanılması en çok tavsiye edilen imza formatıdır. Özellikle oluşturulan imzalı belgelerin kurum dışına gönderileceği durumlarda kullanılması önerilmektedir. İmzalama işlemi yapıldığı anda, imza dosyası ES-X-LONG tipinde oluşturulup saklanabilir.



Şekil 8: CADES-X-LONG İmza Yapısı

2.1.6 CADES-X-Type 1 (Genişletilmiş Elektronik İmza Tip 1 Zamanlı)

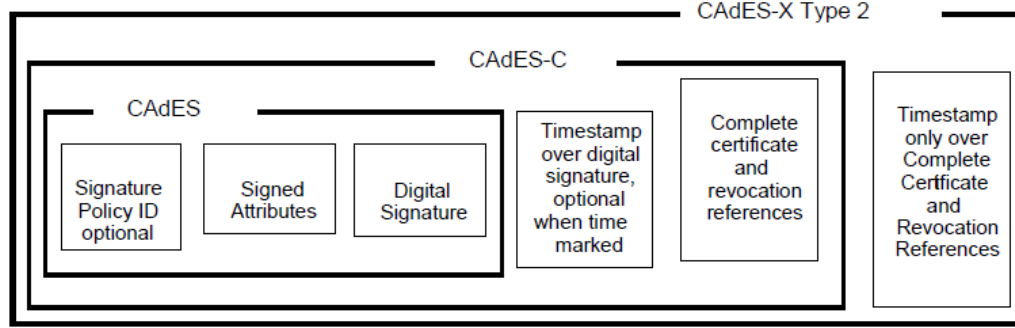
Şekil 9'da görüldüğü gibi ES-C imzası üzerine kurulan bir imza tipidir. ES-C imzasının tamamına zaman damgası alınması ve alınan zaman damgasının imza dosyasına eklenmesi ile oluşturulur. Zaman damgası, imza doğrulamada kullanılan sertifika ve iptal verileri referans değerlerinin ve diğer verilerin koruma altına alınması, imza dosyasına hangi tarihten önce eklendiğinin belirlenmesi amacıyla kullanılır. İmza dosyasına fazladan bir zaman damgası alınmasını gerektirdiğinden çok tercih edilen ve kullanılan bir imza tipi değildir. Ancak kullanımının uygun olduğu durumlar da olabilir.



Şekil 9: CADES-X-Type 1 İmza Yapısı

2.1.7 CAdES-X-Type 2 (Genişletilmiş Elektronik İmza Tip 2 Zamanlı)

CAdES-X-Type 1 ile çok benzer yapıdadır. Aralarındaki tek fark zaman damgasının, ES-C'nin tamamına değil sadece içeriğindeki referans değerlerine alınmış olmasıdır.

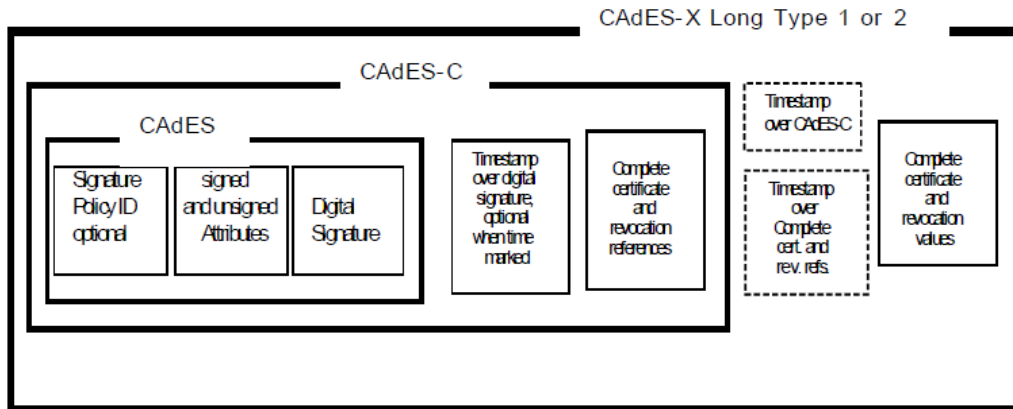


Şekil 10: CAdES-X-Type 2 İmza Yapısı

2.1.8 CAdES-X-Long-Type 1 or Type 2 (Genişletilmiş Uzun Elektronik İmza Tip 1 ve Tip 2 Zamanlı)

CAdES-X-Type 1 ve Type 2 ile çok benzer yapıdadır. Aralarındaki tek fark, CAdES-X-Long-Tip 1 veya Tip 2'de imza doğrulamada kullanılan sertifikaların, SİL ve OCSP cevaplarının da imza dosyasına eklenmesidir.

Doğrulama verilerinin tamamını içerdiğinden CAdES-X-Long ile de benzerlik göstermektedir. Ancak CAdES-X-Long'dan farklı olarak fazladan bir zaman damgası alınmasını gerektirmekte olup kullanımı çok fazla önerilmemektedir. Kullanımının uygun olduğu durumlar oluşabilir ancak genel olarak CAdES-X-Long tipinin kullanılması daha uygundur.



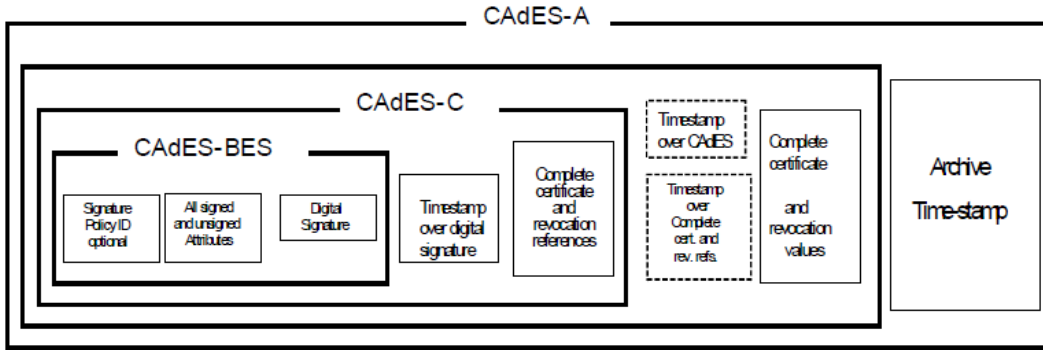
Şekil 11: CAdES-X-Long-Type 1 or Type 2 İmza Yapısı

2.1.9 CAdES-A (Arşiv Elektronik İmza)

E-imzalı belgelerin, ESHS'ye ait kök/alt kök ve zaman damgası sertifikalarının geçerlilik süresinden daha uzun bir süre saklanması gerektiği durumlarda kullanılması gereken bir imza formatıdır. ES-A imza formatı tüm doğrulama verileri eklenmiş imza formatları üzerine oluşturulur. ES-A tipinde imza dosyası aşağıdaki imza dosyalarından birisinin üzerine arşiv zaman damgası alınması yoluyla yapılır.

- CAdES-X-Long
- CAdES-X-Long-Type 1
- CAdES-X-Long-Type 2

Arşivleme, ESHS'ye ait sertifikaların geçerlilik sürelerinin sonuna yaklaşılması, sertifikaların iptal olması veya kullanılan algoritmaların kırıldığıнын duyurulması durumlarında yapılır. Arşivlemenin yukarıdaki durumlar oluşmadan önce yapılmasında bir sakınca yoktur. Arşivleme, yukarıda belirtilen imza dosyalarına alınan arşiv zaman damgası sertifikasının geçerliğinin sona ermesine yakın, gerektiğinde tekrarlanmalıdır. Uygulamada bununla ilgili altyapı sağlanmış olmalıdır.



Şekil 12: CAdES-A İmza Yapısı

2.2 İmza Profilleri

Bu bölümde, Bilgi Teknolojileri ve İletişim Kurumu (BTK) tarafından yayınlanmış, Türkiye’de geçerli imza tipleri ve imza içerisinde yer alması gereken özellikler ifade edilmektedir.

E-imza yaratma ve doğrulama süreçleri oluşturulurken veya bir uygulamaya e-imza kabiliyeti eklenirken kullanılacak e-imzanın kullanım ömrü belirlenmelidir. İmzanın kullanım ömrüne göre farklı imza profillerinde imza atılabilir.

Profil	İmza Ömrü	Zaman Damgası	İptal Bilgisi ¹	Kesinleşme Süresi	İmza Formatı	İmza Dosya Boyutu
P1	Anlık	Yok	SİL/ ÇiSDuP	Uygulanmaz	BES	Düşük
P2	Kısa	Var	SİL	Uygulanır	ES-T	Orta
P3	Uzun	Var	SİL	Uygulanır	ES-XL	Çok yüksek
P4	Sürelili	Var	ÇiSDuP	Uygulanmaz	ES-XL	Yüksek

2.2.1 P1: Anlık - İmza Profili

Kullanım ömrü bir sonraki iptal bilgisinin yayınlanmasından daha kısa süren uygulamalarda kullanılır. Örneğin 4 saatte bir SİL yayınlanan senaryoda anlık imzaların ömrü birkaç dakikadan başlayıp 4 saate kadar uzayabilir. Güvenlik ihtiyacı düşük seviyede olan uygulamalarda kullanılır. İmza doğrulayıcının eline geçtiği an, imza zamanı sayılır. Gelecekte imzayı tekrar doğrulama ihtiyacı olmayacak senaryolarda tercih edilmelidir. Bu imza profili tanımlanmış bir imza politikasına referans vermemektedir.

2.2.2 P2: Kısa Sürelili - İmza Profili

İmza ömrünün, imza sertifikasının kalan ömründen kısa olduğu uygulamalarda tercih edilmelidir. Kısa ömürlü imzalar sertifika ömrü kadar ömre sahip olabilirler; örneğin 3 yıla kadar. Kısa süreli kullanım ömrü olan imzalar, ÇiSDuP erişimi bulunmayan ortamlarda tercih edilmelidir. ÇiSDuP erişimi olan ortamlarda P4 profili kullanılmalıdır.

2.2.3 P3: Uzun Sürelili - İmza Profili

İlk iki kategoriye girmeyen, imza sertifikasının ve imza sertifikasını imzalayan ESHS sertifikasının geçerlilik süresi dolduktan sonra da doğrulanması istenen durumlarda kullanılmalıdır. ÇiSDuP erişimi olmayan ortamlarda kullanılabilir. Aksi durumda P3 profilinin, P4 profiline tercih edilebilecek herhangi bir avantajı yoktur, SİL boyutundan ve imzadan sonra SİL yayınlanmasını bekleme gerekliliğinden dolayı mecbur kalmadıkça tercih edilmemelidir.

2.2.4 P4: Uzun Süreli - İmza Profili

İlk iki kategoriye girmeyen, imza sertifikasının ve imza sertifikasını imzalayan ESHS sertifikasının geçerlilik süresi dolduktan sonra da doğrulanabilen imza tipidir. En güvenilir, uzun ömürlü ve sorunsuz imza profilidir. Validasyon verisi imza içinde yer alır.

3. SERTİFİKA DOĞRULAMA

3.1 Giriş

Bu dokümanda, MA3 Sertifika Doğrulama API'sinin temel çalışma şekli ve kullanımı anlatılmaktadır. Konunun daha iyi anlaşılabilmesi amacıyla sertifika ve sertifika doğrulama konuları kısaca özetlenmektedir.

3.2 Gerekler

MA3 API Sertifika Doğrulama Kütüphanesinin kullanımı için sertifika deposu dosyasına, sertifika doğrulamanın nasıl yapılacağını tanımlayan politika dosyasına ve MA3 API sertifika doğrulama kütüphanelerine ihtiyacınız vardır.

İndirdiğiniz paketle birlikte kullanmanız gereken sertifika deposu dosyası ve örnek bir politika dosyası paketin içinde gelmektedir. Örnek politika dosyası OCSP'ye öncelik vererek sertifika doğrulamayı gerçekleştirmektedir. Eğer özel olarak yapmak istediğiniz bir kontrol yoksa verilen örnek politika dosyasını kullanabilirsiniz.

MA3 API Sertifika Doğrulama kütüphanesinin kullanımı için [SERTİFİKA DOĞRULAMA KÜTÜPHANESİ KULLANIMI](#) bölümünden faydalanabilirsiniz. Sertifika deposunun nasıl kullanılacağı için [Yerel Sertifika Deposu](#) bölümüne ve politika dosyası için [Sertifika Doğrulama Politikası](#) bölümüne bakabilirsiniz.

3.3 Sertifika Doğrulama

3.3.1 Sertifikalar

Sertifikalar, kısaca bir PKI anahtar çifti ile herhangi bir bilgiyi (kimlik bilgisi, yetki, rol vb.) kriptografik olarak ispatlanabilir ve inkar edilemez bir şekilde birbirine bağlayan veri yapılarıdır. Bu anahtar çifti ile veri imzalama, şifreleme, kimlik doğrulama gibi temel PKI işlemleri gerçekleştirilebilmektedir. Bir sertifikanın güvenilirliği, sertifikanın güvenilir bir makam (yayıncı kuruluş) tarafından taklit edilemez bir şekilde imzalanmış olmasından kaynaklanmaktadır. Bu imza sertifikanın üzerinde yer almaktadır ve açık anahtar altyapısının doğası gereği istenildiğinde rahatlıkla doğrulanabilmektedir. Yayıncı kuruluşlar doğrudan güvenin kaynağı olabileceği gibi kendileri de bir başka güvenilir kaynaktan sertifika yayınlama sertifikası almış olabilirler. Birinci durumdaki kuruluşlara **Kök Sertifika Makamı (KSM)**, ikinci durumdaki ara makamlara ise **Sertifika Makamı (SM)** denilmektedir. Kök sertifika makamları, güveni kendisinden var olan ve kendi sertifikasını kendisi imzalayan son derece güvenilir olduğu düşünülen kuruluşlardır. Sertifika imzalama fonksiyonu olmayan, diğer amaçlarla kullanılan sertifikalara **kullanıcı sertifikası** denilir.

Bir kullanıcı sertifikası için, o sertifikadan başlayıp sertifika imzalama (sertifika yayınlama) ilişkisi ile oluşan ve kök sertifika makamına kadar giden sertifika listesine **sertifika zinciri** denir. Bir kullanıcı sertifikasına güvenilebilmesi için her şeyden önce bu sertifikanın, güvenilen bir SM ya da KSM sertifikasında sonlanan bir sertifika zincirine sahip olması gerekir. Bir sertifikanın güvenilir olup olmadığının ve dolayısıyla kullanılmakta olduğu iş (imzalama, şifreleme, kimlik doğrulama vb.) için uygun olup olmadığının bütün yönleriyle kontrol edildiği işlemler bütünü, **sertifika doğrulama** işlemi olarak isimlendirilmektedir.

Bütün dünyada farklı SM'lerin farklı amaçlarla yayınladığı sertifikaların ortak bir yapısı olması amacıyla birtakım standartlar tanımlanmıştır. X.509 bu anlamda dünyaca kabul gören bir açık anahtar altyapısı (PKI) standardıdır. Bu standartta, açık anahtar altyapısının en temel nesneleri olan sertifikalar ve bu sertifikaların iptal durumlarının raporlandığı SİL'ler yapısal detaylarıyla tanımlanmıştır. X.509 standardına uyumlu sertifika ve SİL'lerin yapısı ve doğrulama adımları RFC 5280'de yer almaktadır.

Sertifika yayınlayan kuruluşlar, sertifika oluşturma yanı sıra yayınlamış oldukları sertifikaları iptal edebilirler. Bu durumda bu sertifikalar geçersiz başka bir ifade ile güvenilirmez olurlar. Örneğin; sertifika sahibinin, sertifikasına ait özel anahtarının istenmeyen kişiler tarafından ele geçirildiğini yayıncı SM'ye bildirmesi sonucunda, SM tarafından bu sertifikanın iptal işlemi gerçekleştirilir.

Sertifikanın iptal durumunu öğrenmek için iki yöntem kullanılmaktadır: Sertifikaların iptal durumlarının çevrimiçi yollarla gerçek zamanlı olarak sorgulandığı Çevrimiçi Sertifika Durum Protokolü ve İptal edilmiş sertifikaların iptal bilgilerinin bir veri yapısı oluşturularak dosya tabanlı olarak yayımlandığı SİL'ler. Bu iki yöntem bütün dünyada aynı şekilde uygulanır ve uyumluluğun sağlanması amacıyla belirli standartlar çerçevesinde gerçekleştirilir.

3.3.2 Çevrimiçi Sertifika Durum Protokolü (ÇİSDUP)

RFC 2560'da tanımlı protokole uygun olarak bir veya daha fazla sertifikanın iptal durumunun bir sunucu vasıtasıyla çevrimiçi olarak sorgulanması yöntemidir. İngilizce kısaltmasıyla **OCSP** olarak daha yaygın bilinen bu yöntemde, bir OCSP sorgusuna karşın sunucunun cevap olarak gönderdiği bir OCSP cevabı söz konusudur. Gönderilen bu cevapta, sertifikanın geçerlilik durumu yer almaktadır. Bu sorgu geçerlilik süresi dolmuş sertifikalar için yapılamaz. Protokolün detayları ilgili RFC'de yer almaktadır.

3.3.3 SİL Dosyaları

Yine RFC'de, X.509 standardında sertifika iptal bilgilerinin listelendiği SİL dosyalarının yapısal özellikleri detaylı olarak tanımlanmıştır. SİL dosyaları temel olarak iptal edilmiş sertifikaların listesinden ve bu listenin SM ya da SM tarafından yetkilendirilmiş başka bir makam tarafından atılmış elektronik imzasından oluşur.

Bu listede geçerlilik süresi dolmamış sertifikalar bulunmaktadır. Bir sertifikanın iptal durumu kontrol edilirken bir SİL'e başvurulduğunda, öncelikle bu SİL'in de X.509'a uygunluğu ve geçerliliği kontrol edilmelidir. SİL için yapılması gereken bu kontroller bütününe, **SİL Doğrulama** denilmektedir. SİL dosyalarının yapısal detayları RFC 5280'de yer almaktadır.

3.3.4 X.509 Sertifika ve SİL Doğrulama

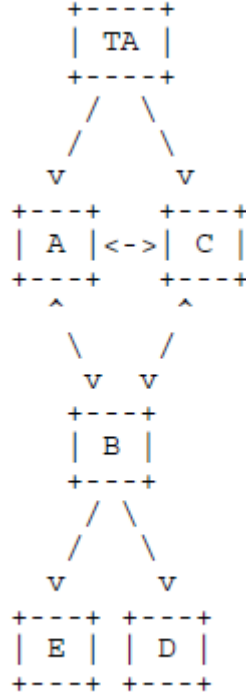
Sertifika ve SİL doğrulama işlemi, sertifikanın ya da SİL'in güvenilir bir yetkili makam sertifikasına kadar uzanan sertifika zincirinin oluşturulması (**path building**) ve bu zincir üzerindeki bütün sertifikaların ve zincir ilişkisinin doğrulanmasından oluşan (**path validation**) iki alt kısımda incelenebilir.

Zincir doğrulama işlemi de zincir üzerindeki sertifikaların doğrulanması (**yapısal doğrulama**) ve zincir ilişkisinin doğrulanması (**zincir doğrulama**) şeklinde iki temel bölümden oluşur. Yapısal doğrulamada sertifika veya SİL'in, X509'da belirtilen yapısal özellikleri karşılayıp karşılamadığına bakılır. Başka bir ifadeyle Sertifika veya SİL'in üzerinde yer alan bilgilerin standarda uygunluğu ve standarda göre geçerliliği kontrol edilir. Örneğin; sertifikada seri numarası alanının bulunup bulunmadığı eğer bulunuyorsa pozitif bir sayı olup olmadığı, sertifikanın üzerinde yazan geçerlilik başlangıç ve bitiş tarihlerinin doğrulama zamanını kapsayıp kapsamadığı kontrol edilir.

MA3 API Sertifika Doğrulama Kütüphanesi, yapısal doğrulama kapsamında RFC 5280'de tanımlı bütün yapısal özellikleri kontrol eder. Zincir oluşturma algoritması olarak RFC 4158'de tanımlı zincir oluşturma algoritmasını ve zincir doğrulama algoritması olarak da yine RFC 5280'de tanımlanmış zincir doğrulama algoritmasını gerçekler. [Bölüm 3.3.4.1](#) ve [Bölüm 3.3.4.2](#)'de zincir doğrulama ve zincir oluşturma nasıl gerçekleştirildiği kısaca özetlenecektir.

3.3.4.1 Sertifika Zinciri Oluşturma

Zincir oluşturma algoritması RFC 4158’de detaylı bir şekilde anlatılmaktadır. Algoritma, bir başlangıç sertifikasından başlayarak güvenilir bir kök sertifikaya ulaşınca kadar adım adım ilerleyerek bir sertifika zinciri oluşturur. Örneğin Şekil 13’teki gibi kurgulanmış bir sertifika ağacı olduğunu düşünelim.



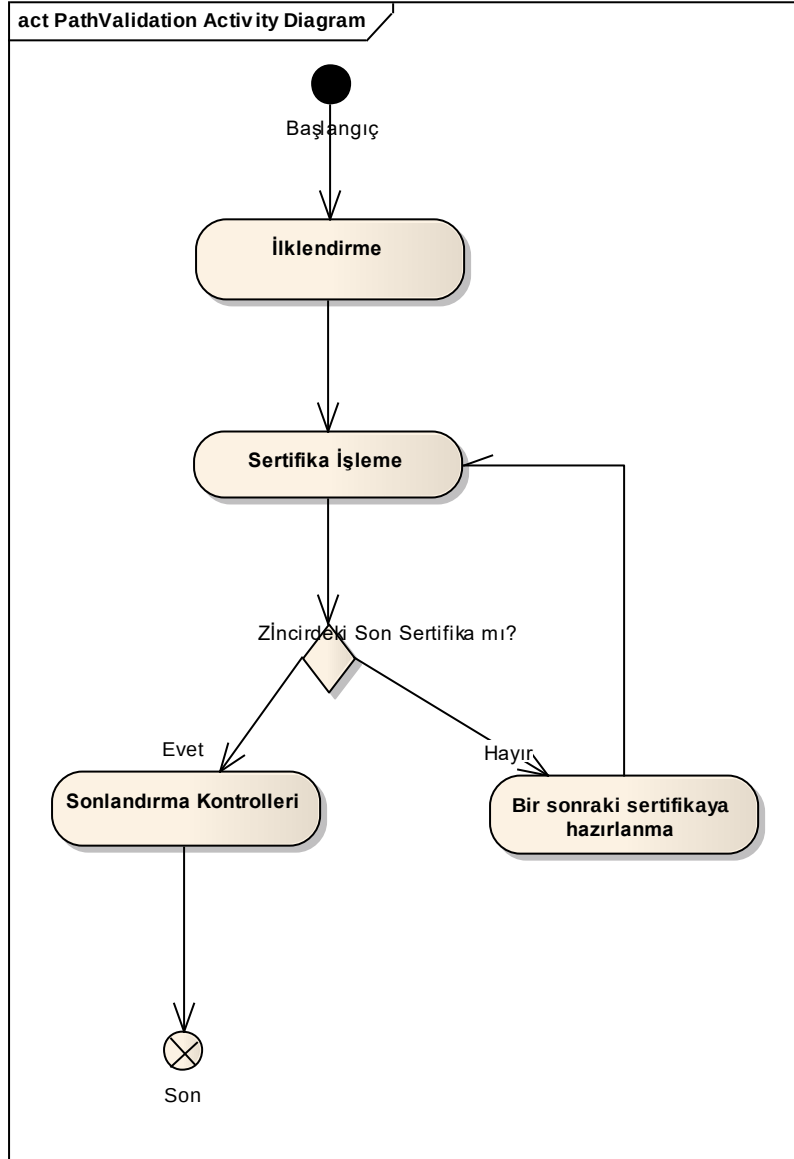
Şekil 13: Örnek Sertifika Ağacı

Sertifika ağacında **TA** ile gösterilen sertifika güvenilir bir SM ya da KSM sertifikasını temsil etmektedir. Bu durumda **E** sertifikası için **E**’den başlayarak **B**, **A** veya **C**’den geçerek **TA**’da biten bir sertifika zinciri oluşturulur.

Sertifika zinciri: **E → B → A → TA**

3.3.4.2 Sertifika Zinciri Doğrulama

Bir sertifika zinciri oluşturulduktan sonra bu zincirin doğrulanması gerekmektedir. Şekil 14'te zincir doğrulama algoritmasının akış diyagramı görülmektedir.



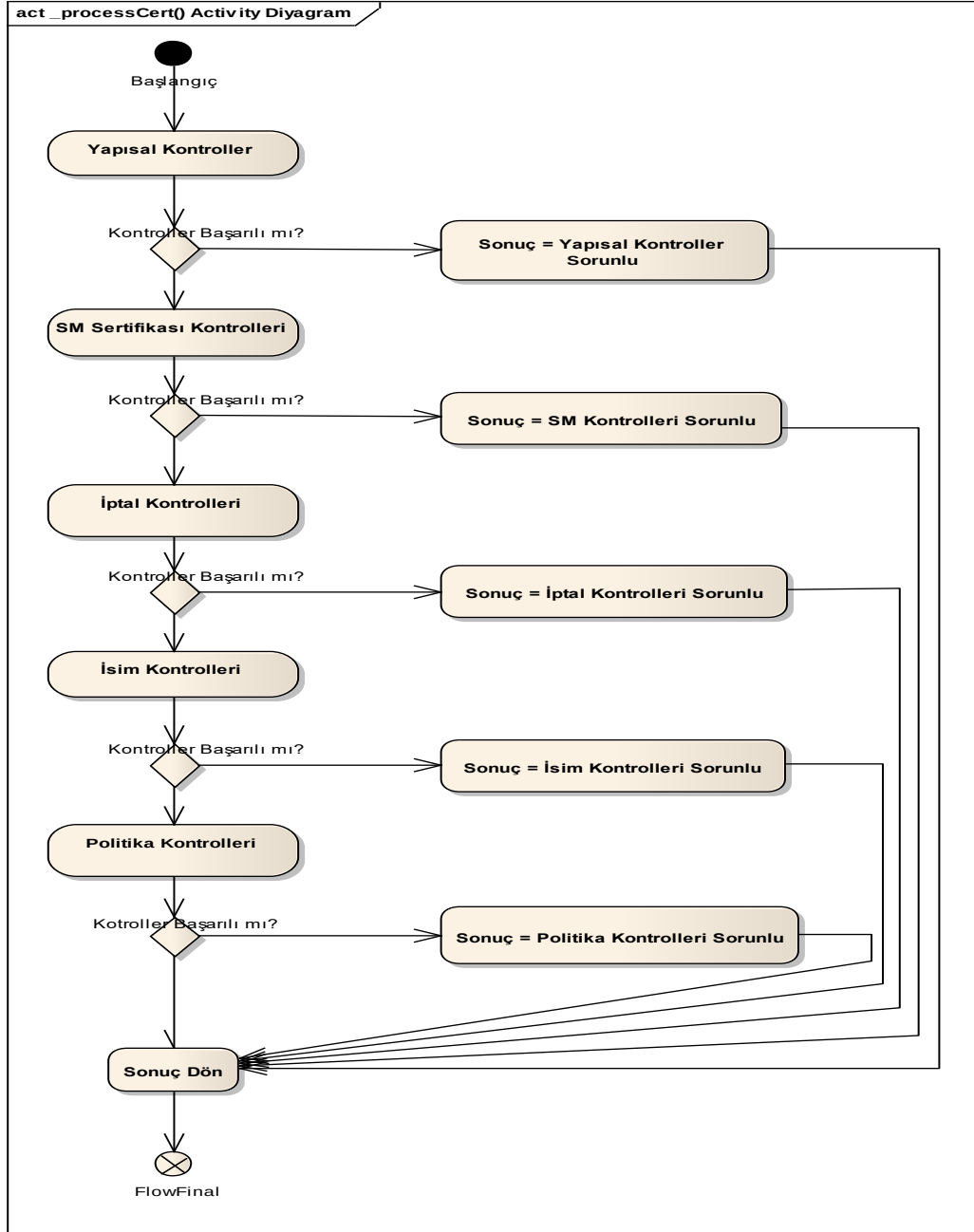
Şekil 14: Zincir Doğrulama Aktivite Diyagramı

Şekil 14'te yer alan *Sertifika İşleme* ve *Sonlandırma Kontrolleri* işlemlerinin detayları RFC 5280'de verilmekte olup kısaca bir takım yapısal ve kriptografik kontrollerden oluştuğu söylenebilir.

SİL doğrulama işlemi, sertifika doğrulama işlemi ile hemen hemen aynıdır. Buradaki tek fark, sertifika doğrulamada oluşturulan zincirin ilk elemanı bir kullanıcı sertifikası iken SİL doğrulamada oluşturulan zincirin ilk elemanının bir SİL olmasıdır. Ve tabii ki SİL'in yapısal doğrulamasında, sertifika yerine SİL'in yapısal özellikleri doğrulanmaktadır. Bunun dışında temel akış sertifika doğrulama ile aynıdır.

3.4 Sertifika Doğrulama Kütüphanesi Bileşenleri

MA3 API Sertifika Doğrulama kütüphanesi, [Bölüm 3.3.4.1](#)'de yer alan **zincir oluşturma** ve [Bölüm 3.3.4.2](#)'de açıklanan **zincir doğrulama** algoritmalarını kullanır. Özetle bir sertifika ya da SİL doğrulama isteğinde bulunulduğunda, önce güvenilir bir kök sertifikada sonlanan bir sertifika zinciri oluşturulur ve bu zincir doğrulanmaya çalışılır. Geçerli bir zincir bulununcaya kadar bu işlem tekrarlanır.



Şekil 15: MA3 API Sertifika Doğrulama - Sertifika İşleme Aktivite Diyagramı

MA3 API Sertifika Doğrulama Kütüphanesi, zincir doğrulama sırasında gerçekleştirilen yapısal doğrulama ve zincir ilişkisinin doğrulanması ile ilgili kontrolleri, Şekil 14'teki diyagramda yer alan *Sertifika İşleme* adımıyla gerçekleştirir. Sertifika İşleme adımının detayları Şekil 15'te görüntülenmektedir.

Bu aktivite gruplara ayrılmış bir takım kontrol işlemlerinden oluşur ve bu kontrol işlemleri [sertifika doğrulama politika dosyasında](#) çalışma zamanında yapılandırılabilir. Bu politika dosyasının yapısı [Bölüm 3.4.5'te](#) yer almaktadır. Bu kontrol işlemleri atomik kontrollerden oluşur ve her bir atomik kontrol için bir [kontrolcü](#) (*Checker*) sınıf tanımlanmıştır. Kontrolcü sınıfların, kontrol işlemlerini yaparken ihtiyaç duyduğu sertifika, SİL, OCSP cevabı gibi kaynakları çeşitli yollarla ve çeşitli yerlerden bulmak için [bulucu](#) (*Finder*) sınıflar tanımlanmıştır. Bulucuların bulunduğu kaynakların doğru kaynaklar olup olmadığının test edilmesi gerekmektedir ve bu amaçla [eşleştirici](#) (*Matcher*) sınıflar oluşturulmuştur. MA3 API Sertifika Doğrulama Kütüphanesi hangi kontrollerin yapılacağını yani hangi kontrolcü sınıflarının kullanılacağını, ihtiyaç duyulan kaynaklar için nerelere bakılacağını yani hangi bulucu sınıflarının kullanılacağını ve bulunan kaynakların nasıl eşleştirileceğini yani hangi eşleştirici sınıflarının kullanılacağını belirlemek amacıyla çalışma zamanında işlemek üzere **doğrulama politika dosyasını** kullanır. Doğrulama işleminin öncesinde, bu politika dosyası okunarak gerekli kontrolcü, bulucu ve eşleştirici alt sınıfları oluşturulur.

3.4.1 Kontrolcüler

Sertifika ve SİL'lerin yapısal özelliklerini ve sertifika zincir ilişkisinin standartlara uygunluğunu kontrol eden sınıflardır. Bu sınıfların detayları, bu dokümanın kapsamı dışındadır. Bu kontrolcülerin hangilerinin çalışacağı doğrulama politika dosyası aracılığıyla belirlenir. Sertifika doğrulama işleminde gerçekleştirilen ve RFC 5280'de tanımlanan kontrolcüler temel olarak şunlardır:

3.4.1.1 Yapısal Kontroller

Yapısal kontroller sertifikanın ve SİL'in üzerindeki bilgilerin içerik ve biçimsel olarak geçerliliğinin kontrollerini kapsar. Temel olarak yapısal kontroller aşağıdaki gibi listelenebilir.

- **Seri Numarası Kontrolü:** Sertifika üzerindeki sertifika seri numarasının pozitif bir tam sayı olup olmadığının kontrolünü yapar.
- **Sertifika Geçerlilik Tarihi Kontrolü:** Sertifika üzerindeki sertifika geçerlilik tarihlerinin, doğrulama zamanını kapsayıp kapsamadığının kontrolünü yapar.
- **Versiyon Kontrolü:** Sertifika üzerindeki versiyon bilgisinin geçerli olup olmadığının kontrolünü yapar.
- **Eklenti Kontrolü:** Sertifika üzerindeki eklentilerin geçerli olup olmadığının kontrolünü yapar.

- **İmza Algoritması Kontrolü:** Sertifika üzerinde yazan imza algoritma bilgisi ile sertifikanın imzalandığı imza algoritma bilgisinin aynı olup olmadığının kontrolünü yapar.

3.4.1.2 Zincir İlişkisi Kontrolleri

Zincir ilişkisi kontrolleri, sertifika veya SİL ile bu sertifika veya SİL'i yayınlayan yetkili makam sertifikası arasındaki ilişkiyi inceleyen kontrolleri içerir.

- **İsim Kontrolü:** Sertifika veya SİL üzerindeki yayıncı(issuer) alanı ile yayıncı sertifikası üzerindeki özne(subject) alanının aynı olup olmadığının kontrolünü yapar.
- **İmza Kontrolü:** Sertifika veya SİL üzerindeki imzanın, yayınlayan sertifikasındaki açık anahtar ile kriptografik olarak doğrulanıp doğrulanmadığının kontrolünü yapar.
- **Temel Kısıtlar Kontrolü:** Yayıncı sertifikası üzerinde Temel Kısıtlar (Basic Constraints) eklentisinin bulunup bulunmadığının ve bu eklentideki yayıncı işaretçisinin RFC 5280'de belirtildiği şekilde yer alıp almadığının kontrolünü yapar.
- **Anahtar Tanımlayıcısı Kontrolü:** Sertifika veya SİL üzerindeki yetkili anahtar tanımlayıcısı (Authority Key Identifier) eklentisindeki değer ile yayıncı sertifikasındaki anahtar tanımlayıcısı değerinin uyuşup uyuşmadığının kontrolünü yapar.
- **Yol Uzunluğu Kontrolü:** Sertifika zinciri uzunluğunun, yayıncı sertifikasındaki yol uzunluğu eklentisinde yer alan değerden kısa olup olmadığının kontrolünü yapar.
- **Anahtar Kullanımı Kontrolü:** Yayıncı sertifikasındaki anahtar kullanımı (Key Usage) eklentisinde, sertifika imzalama özelliğinin bulunup bulunmadığının kontrolünü yapar.

3.4.1.3 İptal Kontrolleri

Sertifikanın iptal durum kontrollerini içerir.

- **SİL'den İptal Durum Kontrolü:** Sertifikaya ilişkin SİL'den, sertifikanın iptal durum kontrolü yapılır.
- **OCSP'den İptal Durum Kontrolü:** Sertifikaya ilişkin OCSP cevabından sertifikanın iptal durum kontrolü yapılır.

3.4.1.4 İsim Kontrolleri

Sertifikadaki özne bilgisi, yayıncı sertifikasında yer alan isim kısıtlamaları eklentisinde tanımlı birtakım kurallara göre değerler alabilir. Bu detaylar RFC 5280'de belirtilmiştir ve isim kontrolleri kapsamında bu kurallara uyulup uyulmadığı kontrol edilmektedir.

3.4.1.5 Politika Kontrolleri

Sertifikadaki politika bilgileri eklentisinde yer alan politika bilgileri, yayıncı sertifikasındaki isim kısıtlamaları eklentisinde tanımlı birtakım kurallara göre değerler alabilir. Bu detaylar RFC 5280'de belirtilmiştir ve politika kontrolleri kapsamında bu kurallara uyulup uyulmadığı kontrol edilmektedir.

MA3 API Sertifika Doğrulama kütüphanesinde tanımlı ve politika dosyasında kullanılabilecek tüm kontrolcülerin listesi EK-A' daki [Tablo1](#)'de yer almaktadır.

3.4.2 Bulucular

Kontrolcü sınıflar çalışırken bir takım dış verilere ihtiyaç duyabilirler. Bu bir SİL bilgisi, bir OCSP sorgusu ya da bir SM sertifikası olabilir. Yine zincir oluşturma sırasında SM sertifikalarının çeşitli yerlerden (http adresi, yerel sertifika deposu vb.) bulunması gerekebilir. Genel olarak sertifika doğrulama sırasında dışarıdan bulunması gereken sertifika, SİL, OCSP cevabı gibi verileri bulma işlemini bulucu sınıflar gerçekleştirir. Bir sertifika, SİL ya da OCSP cevabı bulunurken sertifika doğrulama politika dosyasında tanımlı bulucular sırayla çalıştırılır ve aranılan veri bulunduğunda bulma işlemi sonlandırılır. Bu nedenle bulucuların uygun bir sırayla politika dosyasına yerleştirilmiş olması performans açısından oldukça önemlidir. Örneğin yerel depodan sertifika bulucunun, uzak kaynaklardan (LDAP, HTTP vb.) sertifika buluculardan önce yerleştirilmesi, sertifika arama işlemlerinin sertifika yerel depoda varsa uzak kaynaklara başvurulmadan sonuçlanmasını sağlar ve bu da ciddi performans kazanımları getirir.

MA3 API Sertifika Doğrulama Kütüphanesinde tanımlı bulucular şu şekilde gruplandırılabilir:

3.4.2.1 Güvenilir Sertifika Bulucular

Doğrulama kütüphanesinin güvenilir olarak tanıdığı SM sertifikalarını bulan sınıflardır.

3.4.2.2 Sertifika Bulucular

SM sertifikası bulmaktan sorumlu sınıflardır. Bu bulucular sertifikanın üzerinde yer alan Yetkili Erişim Noktası (AIA) eklentisine bakarak SM sertifikası bulan sınıflar olabileceği gibi bir http adresinden, bir LDAP adresinden, dosya sistemindeki bir adresten ya da yerel sertifika deposundan sertifika bulan sınıflar da olabilir.

3.4.2.3 SİL Bulucular

SİL bulmaktan sorumlu sınıflardır. Bu sınıflar, sertifika üzerinde yazan Sil Dağıtım Noktaları (CDP) eklentisine bakarak SİL bulan sınıflar olabileceği gibi bir http adresinden, bir LDAP adresinden, dosya sistemindeki bir adresten ya da yerel sertifika deposundan SİL bulan sınıflar da olabilir.

3.4.2.4 OCSP Cevabı Bulucular

OCSP cevabı bulmaktan sorumlu sınıflardır. Bu sınıflar, sertifika üzerinde yazan Yetkili Erişim Noktası (AIA)'na bakarak OCSP cevabı bulan sınıflar olabileceği gibi bir http adresinden, bir LDAP adresinden, dosya sistemindeki bir adresten ya da yerel sertifika deposundan OCSP cevabı bulan sınıflar da olabilir.

MA3 API Sertifika Doğrulama Kütüphanesinde tanımlı ve politika dosyasında kullanılabilecek tüm bulucuların listesi EK-A' daki [Tablo 3](#)'te yer almaktadır.

3.4.3 Eşleştiriciler

Bulunan sertifika, SİL ya da OCSP cevap bilgilerinin gerçekten aranılan veriler olup olmadığını anlamak için gerekli eşleştirmelerden sorumlu sınıflardır. Bu eşleştirme kuralları RFC 5280'de yer almaktadır.

MA3 API Sertifika Doğrulama Kütüphanesi'nde tanımlı eşleştiriciler şu şekilde gruplandırılabilir:

3.4.3.1 Sertifika Eşleştiriciler

Sertifika ile yayıncı sertifikasını eşleştiren sınıflardır. Örneğin RFC 5280'de "Bir X.509 sertifikası üzerinde yazan *issuer* alanı ile yayıncı sertifikası üzerinde yer alan *subject* alanı aynı olmalıdır." şeklinde bir ifade yer alır. MA3 API Sertifika Doğrulama Kütüphanesi'nde, bu eşleştirme işini gerçekleştiren *IssuerSubjectMatcher* isimli bir eşleştirici sınıf bulunmaktadır. Eşleştiricilerin eşleştiremediği sertifikalar, yayıncı sertifikası olarak sertifika zincirine eklenmezler.

3.4.3.2 SİL Eşleştiriciler

Sertifika ile SİL'i ya da SİL ile SİL'i yayınlayan sertifikayı eşleştiren sınıflardır. Örneğin RFC 5280'de, "Dolaylı SİL (indirect crl) kullanılmıyorsa, bir SİL üzerinde yazan *issuer* alanı ile yayıncı sertifikası üzerinde yer alan *subject* alanı aynı olmalıdır." şeklinde bir ifade yer alır. MA3 API Sertifika Doğrulama Kütüphanesi'nde bu eşleştirme işini gerçekleştiren *CRLIssuerMatcher* isimli bir eşleştirici sınıf bulunmaktadır.

3.4.3.3 OCSP Cevabı Eşleştiriciler

Sertifika ile OCSP cevabını eşleştiren sınıflardır. Çalışma şekilleri, sertifika ve SİL eşleştiriciler gibidir.

3.4.3.4 Delta SİL Eşleştiriciler

SİL ile Delta-SİL'i eşleştiren sınıflardır. Çalışma şekilleri, sertifika ve SİL eşleştiriciler gibidir.

3.4.3.5 Çapraz Sertifika Eşleştiriciler

SM Sertifikası ile çapraz SM sertifikasını eşleştiren sınıflardır. Çalışma şekilleri, sertifika ve SİL eşleştiriciler gibidir.

MA3 API Sertifika Doğrulama Kütüphanesi'nde tanımlı tüm eşleştiricilerin listesi EK-A [Tablo 2](#)'de yer almaktadır.

3.4.4 Kaydediciler

MA3 API Sertifika Doğrulama Kütüphanesi, doğrulama sırasında bulduğu sertifika ve SİL'leri yerel depoya kaydeder. Bu işlemi kaydediciler (Saver) gerçekleştirir ve yine politika dosyası aracılığıyla hangi kaydedicilerin çalışacağı tanımlanabilir. Örneğin istenildiği kadar kaydedici tanımlanarak doğrulama sırasında bulunan ve doğrulanan bütün sertifikalar istenilen yerlere kaydedilebilirler.

MA3 API Sertifika Doğrulama Kütüphanesi'nde tanımlı tüm kaydedicilerin listesi EK-A [Tablo 4](#)'te yer almaktadır.

3.4.5 Sertifika Doğrulama Politikası

Sertifika doğrulama işlemi birçok kontrol işleminin arka arkaya yapılmasından oluşan bir süreçtir. Bu kontrol işlemlerinin ve kontroller sırasında kullanılan birtakım parametrelerin, çalışma zamanında belirlenebilmesi için XML formatında bir doğrulama politika dosyası kullanılmaktadır. Örnek bir doğrulama politika dosyasının bir bölümü Şekil 16'da bulunmaktadır. Politika dosyasında yer alan her bir **class** (sınıf) elemanı doğrulamada kullanılacak kontrol, bulma ya da eşleştirme sınıflarından birini temsil eder.

Doğrulama politika dosyası, doğrulama işlemlerinin başında okunarak doğrulama politika nesnesi oluşturulur ve bütün doğrulama adımları bu politikada belirtilen yapılandırma bilgileri doğrultusunda gerçekleştirilir.

```

<?xml version="1.0" encoding="UTF-8"?>
<policy>
  <find>
    <trustedcertificate>
      <class name="tr.gov.tubitak.uekae.esya.api.certificate.validation.find.certificate.trusted.TrustedCertificateFinderFromXml">
        <param name="securitylevel" value="organizational,legal"/>
        <param name="updatepath" value="http://depo.kamusm.gov.tr/depo/SertifikaDeposu.xml"/>
      </class>
      <!-- Use TrustedCertificateFinderFromFileSystem only for TESTING purposes. -->
      <!--class name="tr.gov.tubitak.uekae.esya.api.certificate.validation.find.certificate.trusted.TrustedCertificateFinderFromFileSystem">
        <param name="dizin" value="C:\\??\\depo\\cert"/>
      </class-->
    </trustedcertificate>
    <certificate>
      <class name="tr.gov.tubitak.uekae.esya.api.certificate.validation.find.certificate.CertificateFinderFromHTTP"/>
      <class name="tr.gov.tubitak.uekae.esya.api.certificate.validation.find.certificate.CertificateFinderFromXml"/>
    </certificate>
    <deltacrl/>
  </find>
  <match>
    <certificate>
      <class name="tr.gov.tubitak.uekae.esya.api.certificate.validation.match.certificate.IssuerSubjectMatcher"/>
      <class name="tr.gov.tubitak.uekae.esya.api.certificate.validation.match.certificate.KeyIDMatcher"/>
    </certificate>
    <crl>
      <class name="tr.gov.tubitak.uekae.esya.api.certificate.validation.match.crl.CRLIssuerMatcher"/>
    </crl>
    <deltacrl>
      <class name = "tr.gov.tubitak.uekae.esya.api.certificate.validation.match.deltacrl.BaseCRLNumberMatcher"/>
      <class name = "tr.gov.tubitak.uekae.esya.api.certificate.validation.match.deltacrl.CRLNumberMatcher"/>
      <class name = "tr.gov.tubitak.uekae.esya.api.certificate.validation.match.deltacrl.DeltaCRLIssuerMatcher"/>
      <class name = "tr.gov.tubitak.uekae.esya.api.certificate.validation.match.deltacrl.ScopeMatcher"/>
    </deltacrl>
    <ocsp>
      <class name = "tr.gov.tubitak.uekae.esya.api.certificate.validation.match.ocsp.CertIDOCSPResponseMatcher"/>
    </ocsp>
  </match>
</policy>

```

Şekil 16: Örnek Sertifika Doğrulama Politikası dosyası

3.4.5.1 Sertifika Doğrulama Politika Dosyasının Güvenliği

Politika dosyası, sertifika doğrulama sırasında kullanılacak verilerin nasıl bulunacağını (*find*), doğru verilerin bulunup bulunmadığının kontrolü için nasıl eşleştirme yapılacağını (*match*) ve doğrulama sırasında hangi kontrollerin yapılacağını (*validate*) belirten sınıfları bulunduran dosyadır.

Politika dosyasının değiştirilmemesi imza doğrulama için hayati önem taşır. Politika dosyasında hangi kontrollerin yapılacağı ve hangi sertifikalara güvenileceği bilgisi yer almaktadır. Kötü niyetli kişiler politika dosyasını değiştirerek, kötü niyetle yaratılmış imzaların doğrulanmasını sağlayabilirler.

Politika dosyası için aşağıdaki güvenlik önlemleri uygulanabilir.

- Politika dosyası sunucudan çekilir ve bellekte tutulur. Böylelikle politika dosyasına dışardan müdahale olasılığı azalır.
- Politika dosyasının özeti sunucuda tutulur. İstemcideki politika dosyasının özeti alınır ve sunucudan çekilen özet ile karşılaştırılır. *DigestUtil* sınıfı ile özet işlemini gerçekleştirebilirsiniz.
- Politika dosyası istemcide parola tabanlı şifrelenerek tutulur ve şifresi bellekte çözülür.
- Bunun nasıl yapıldığını gösteren örneği aşağıdaki adreste bulabilirsiniz.
<http://www.java2s.com/Tutorial/Java/0490Security/PBEFileEncrypt.htm>
- Politika dosyası imzalı bir şekilde tutulur. Politika dosyasına karar verildikten sonra politika dosyası bir kere imzalanır ve bu imzalı dosya istemcilerde tutulur. İmzalanan politika dosyasının imzası doğrulanırsa ve imza doğru kişi tarafından atılmışsa; politika dosyasına güvenilebilir. Doğru kişi imzalamış mı kontrolünün yapılması için, kodunuzun içine imzacı sertifikasının gömülmesi gerekmektedir. Kodunuzu, devamlı değiştiremeyeceğinizden imzalama işleminde kullandığınız akıllı kartı veya pfx dosyasını kesinlikle kaybetmemeniz gerekmektedir. Politika dosyasını PKCS7 standardında imzalayabilirsiniz. PKCS7 tipindeki imza işlemlerine smartcard modülünden erişebilirsiniz.

Yukarıda sayılan güvenlik önlemleri, birlikte veya tek olarak uygulanabilir. Politika dosyasının güvensiz bir şekilde istemcilerde durmasını **kesinlikle önermiyoruz**.

3.4.6 Sertifika - SİL Durum Bilgisi

MA3 API Sertifika Doğrulama Kütüphanesi sertifika ve SİL doğrulama işlemi sonucunda Sertifika Durum Bilgisi (*CertificateStatusInfo*) ve SİL Durum Bilgisi (*CRLStatusInfo*) sınıflarından bir nesne oluşturur. Bu sınıf içerisinde yapılan kontrollerle ilgili bilgiler ve oluşturulmuş, doğrulanmaya çalışılan farklı sertifika zincirleri doğrulanma sonuçlarıyla birlikte detaylı olarak saklanır. Bu sınıfa ait nesnenin kullanımı, [Bölüm 4](#)'teki örnek kodlarda yer almaktadır.

3.4.7 XML Sertifika Deposu

Güvenilir olduğu kabul edilen ESHS kök sertifikaları sertifika deposunda saklanır.

MA3 API Sertifika Doğrulama Kütüphanesi, sertifika deposunda güvenilir kök sertifikası olarak tanımlanmış seritikaları güvenilir kabul eder ve doğrulama sırasında oluşturduğu sertifika zincirlerinin de bu sertifikalardan biriyle sonlanmasını bekler.

XML depodan güvenilir kök sertifikaları bulmak için sertifika doğrulama konfigürasyon dosyasına aşağıdaki tanım eklenmelidir. Lokal sertifika deposundaki güvenilir kök sertifikalar `updatepath` adresinde bulunan depo ile belirli aralıklarla güncellenmektedir. Varsayılan lokal dosya yolunda sertifika deposu yoksa `updatepath` adresinde bulunan sertifika deposu varsayılan lokasyona kopyalanır.

```
<class
name="tr.gov.tubitak.uekae.esya.api.certificate.validation.find.certificate.CertificateFinderFromXml">
<param name="updatepath" value="http://depo.kamusal.gov.tr/depo/SertifikaDeposu.xml"/>
</class>
```

Sertifika deposunun varsayılan yeri, kullanıcının ana klasörünün (user home directory) altındaki **.sertifikadeposu** klasörünün içidir. Varsayılan dosya ismi ise **SertifikaDeposu.xml**'dir. Eğer farklı bir dosya yolu ve dosya ismi kullanılması isteniyorsa, sertifika doğrulama politika dosyasında belirtilmelidir. Bunun için `storepath` parametresi kullanılmalıdır.

XML sertifika deposu, sadece MA3 API Sertifika Doğrulama Kütüphanesi ile kullanılmalıdır. Aksi takdirde man-in-the-middle saldırısına kapı açılmış olur. MA3 API Kütüphanesi, bu depodaki güvenilir sertifikalar için imza kontrolü yapmaktadır.

Test sertifikalarıyla çalışırken sertifikanın doğrulanması sırasında `PATH_VALIDATION_FAILURE` hatası alınabilir. Bunun sebebi test sertifikalarına ait köklerin, sertifika deposunda güvenilir sertifika olarak tanımlanmaması olabilir. Böyle bir durumda kullanılan sertifikaya ait kök sertifikalar, dosya yoluyla gösterilebilir fakat bu kullanım sadece test amaçlı yapılmalıdır.

```
<class
name="tr.gov.tubitak.uekae.esya.api.certificate.validation.find.certificate.trusted.TrustedCertificateFinderFromFileSystem">
<param name="dizin" value="C:\\???\\ma3api\\TrustedRootCerts"/>
</class>
```

Güvenilir sertifika olarak sadece kendi kendini imzalamış sertifikaların kullanılması gerekmektedir. Yanlışlıkla alt-kök ve son kullanıcı sertifikaların güvenilir kök olarak eklenmesi engellenmiştir. Bu engeli devre dışı bırakmak için `onlyselfsigned` parametresi `false` değeri ile birlikte kullanılmalıdır.

```
<class
name="tr.gov.tubitak.uekae.esya.api.certificate.validation.find.certificate.trusted.TrustedCertificateFinderFromFileSystem">
<param name="dizin" value="C:\\???\\ma3api\\TrustedRootCerts"/>
<param name="onlyselfsigned" value="false"/>
</class>
```

Dosya sistemine erişimin olmadığı durumlarda uzaktaki bir depo ile çalışabilmek mümkün olmaktadır. `storepath` parametresine bir http adresi verilerek kullanım sağlanabilir. Yalnız bu durumda uzak noktaya bağımlılık eklendiği için kırılganlık artacaktır.

```
<class
name="tr.gov.tubitak.uekae.esya.api.certificate.validation.find.certificate.trusted.TrustedCertificateFinderFromXml">
<param name="storepath" value="http://depo.kamusm.gov.tr/depo/SertifikaDeposu.xml"/>
</class>
```

3.5 Sertifika Doğrulama Kütüphanesi Kullanımı

Sertifika doğrulama kütüphanesinin kullanımı için gerekenlere, [Gerekler](#) bölümünden bakabilirsiniz. Sertifika doğrulama kütüphanesi, aldığı sertifikayı verilen politika dosyasına uygun olarak doğrulamaktadır.

Sertifika doğrulama işlemi için kütüphaneye, sertifikanın hangi tarihte doğrulanacağı bilgisi verilmelidir. Sertifika iptal kontrolü verilen tarihe göre yapılacaktır. Verilen tarihten önce iptal edilmiş sertifikaların durumu geçersiz olacaktır. Şu anki zamanda sertifika doğrulama için aşağıdaki örnek kod kullanılabilir.

Java

```
ValidationSystem vs = CertificateValidation.createValidationSystem(policy);
vs.setBaseValidationTime(Calendar.getInstance());
CertificateStatusInfo csi = CertificateValidation.validateCertificate(vs, cert);
```

C#

```
ValidationSystem vs = CertificateValidation.createValidationSystem(policy);
vs.setBaseValidationTime(DateTime.UtcNow);
CertificateStatusInfo csi = CertificateValidation.validateCertificate(vs, cert);
```

3.6 Sertifika Doğrulama Sonucunun Yorumlanması

Sertifika doğrulama sonucunda *CertificateStatusInfo* nesnesi dönmektedir. Bu nesnenin *toString()* metodu çağrılarak sertifika doğrulamanın sonucu metin olarak alınabilir. *getDetailedMessage()* fonksiyonu ile de sertifika doğrulama sonucunun kullanıcı dostu mesajı görülebilir. Yine aynı nesnenin *getCertificateStatus()* fonksiyonu kullanılarak *CertificateStatus* tipinde sertifika doğrulamanın durumu alınabilir. Bu nesne aşağıdaki değerleri alabilir.

VALID: Sertifika geçerlidir.

REVOCATION_CHECK_FAILURE: Sertifika iptal edilmiştir.

CERTIFICATE_SELF_CHECK_FAILURE: Sertifikada yapısal bozukluk vardır.

NO_TRUSTED_CERT_FOUND: Güvenilir sertifika bulunmamaktadır.

PATH_VALIDATION_FAILURE: Güvenilir bir sertifika zinciri oluşturulamamıştır. Sertifikaya ait kök sertifika, güvenilir sertifikalar arasında bulunmamaktadır.

NOT_CHECKED: Sertifika iptal kontrolü yapılamadığından dolayı sertifika doğrulanamamaktadır.

3.7 Politika Dosyası

İndirdiğiniz kütüphane ile birlikte gelen politika dosyası, bir sertifika doğrulama işleminin başarılı sayılabilmesi için gerekli yeterliliği sağlamaktadır. Değiştirilme ihtiyacının duyulabileceği iki ayar olabilir.

Örnek olarak verilen politika dosyası sadece sertifika deposu ile çalışmaktadır. Sertifika deposunda ise sadece kanuni geçerliliği olan kök sertifikalar bulunmaktadır. Dolayısıyla test sistemi ile çalışmak için kullanılan test sertifikalarına ait köklerin de kütüphaneye güvenilir kök olarak gösterilmesi gerekmektedir. Test sertifikalarının kökü, güvenilir kök sertifika olarak dizin sistemi üzerinden gösterilebilir. Nasıl gösterilmesi gerektiği ile ilgili bilgiye [MA3 API Bulucu Listesi](#) (Tablo 3) başlığı altındaki **TrustedCertificateFinderFromFileSystem** politika dosyası elemanı incelenerek ulaşılabilir.

Politika dosyasında yapılacak bir diğer düzenleme, sertifika iptal kontrollerinde ÇİSDUP veya SİL kullanımının ayarlanmasıdır. Bir sertifikanın doğrulanması sırasında sertifika zincirinin tamamının doğrulanması gerekmektedir. Zincirdeki bütün sertifikalar için ÇİSDUP desteği verilmeyebilir. Örneğin KamuSM sisteminde, alt kökler için ÇİSDUP hizmeti verilmemektedir. Dolayısıyla sadece ÇİSDUP kullanarak bir sertifikayı doğrulamak mümkün değildir. Bu durumda, ÇİSDUP öncelikli olmak üzere SİL ve ÇİSDUP'un bulunduğu konfigürasyon veya sadece SİL ile çalışan konfigürasyon yaratılabilir.

Politika dosyasında, ÇİSDUP veya SİL'den hangisine öncelik verilmek isteniyorsa o sınıf üste yazılmalıdır.

3.7.1 Politikanın Çalışma Zamanında Düzenlenmesi

Politika dosyası, dosyadan okuduktan sonra çalışma zamanında düzenlenebilir. Aşağıdaki kod parçası, klasörden güvenilir sertifikaları gösteren *TrustedCertificateFinderFromFileSystem* sınıfını çalışma zamanında eklemektedir.

Java

```
ValidationPolicy POLICY = PolicyReader.readValidationPolicy(new
FileInputStream(POLICY_FILE));

//For UEKAE Test Environment, we add our test roots.
HashMap<String, Object> parameters = new HashMap<String, Object>();
parameters.put("dizin", "T:\\MA3\\api-
cmssignature\\testdata\\support\\UGRootCerts\\");
POLICY.bulmaPolitikasiAl().addTrustedCertificateFinder("tr.gov.tubitak.uekae.esya.api
.certificate.validation.find.certificate.trusted.TrustedCertificateFinderFromFileSyst
em", parameters);
```

C#

```
ValidationPolicy POLICY = PolicyReader.readValidationPolicy(POLICY_FILE);

//For UEKAE Test Environment, we add our test roots.
Dictionary<String, Object> parameters = new Dictionary<String, Object>();
parameters.Add("dizin", "T:\\MA3\\api-
cmssignature\\testdata\\support\\UGRootCerts\\");
POLICY.bulmaPolitikasiAl().addTrustedCertificateFinder("tr.gov.tubitak.uekae.esya.api
.certificate.validation.find.certificate.trusted.TrustedCertificateFinderFromFileSyst
em", parameters);
```

3.8 Politika Dosyası Elemanları

Politika dosyasında tanımlanan sınıflar aşağıdaki tablolarda listelenmiştir.

Bu tablolardaki gösterimde;

- Parametrenin alabileceği değerlerin listelendiği yerde * ile belirtilen değer parametrenin varsayılan değeridir.
- [O] değeri ile belirtilen parametre, opsiyonel parametre anlamına gelir.
- Başlık satırlarındaki başlığın yanında bulunan XML tagları, o başlık altında yer alan sınıfların politika dosyasında hangi XML tag değerinin altında listelenmesi gerektiğini (yani politika dosyasındaki yerini) belirtir.

Tablo 1 MA3 API Kontrolcü (Checker) Listesi

GÜVENİLİR SERTİFİKA YAPISAL KONTROLCÜLER<policy><validate><certificate><trustedcertificate>	
CertificateDateChecker	Sertifika üzerinde yer alan geçerlilik zaman aralığının doğrulama zamanını kapsamaması kuralını doğrular.
SelfSignatureChecker	Sertifika üzerindeki imzayı, sertifikanın açık anahtarı ile kriptografik olarak doğrular.
SERTİFİKA YAPISAL KONTROLCÜLER<policy><validate><certificate><self>	
CertificateDateChecker	Sertifika üzerinde yer alan geçerlilik zaman aralığının doğrulama zamanını kapsamaması kuralını doğrular.
CertificateExtensionChecker	Sertifika üzerinde yer alan eklenti bilgilerinin RFC 5280 uyumluluğunu kontrol eder.
PositiveSerialNumberChecker	Sertifika seri numarasının pozitif bir tamsayı olması kuralını doğrular.
SignatureAlgConsistencyChecker	Sertifika üzerinde yer alan imza algoritmasının, sertifikanın imzalanma algoritması ile aynı olup olmadığının kontrolünü yapar.
VersionChecker	Sertifika versiyon bilgisinin RFC 5280 uyumluluğunu kontrol eder.

QualifiedCertificateChecker		Sertifikanın niteliklilik kontrolü yapılır.	
	Parametreler	statementoids	Verilen OID'lere göre sertifikanın niteliklilik kontrolleri yapılır. Bazı nitelikli sertifika kontrollerinde iki tane OID kontrol edilmek istenebilir. Bu gibi durumlarda AND ile OID'ler birleştirilebilir. Değişik ülkeler değişik OID'ler kullanabilmektedir. Bu tür durumlar için de OID'ler OR ile ayrılabilir. Örn: (0.4.0.1862.1.1 AND 2.16.792.1.61.0.1.5070.1.1) OR (4.3.2.1 AND 1.2.3.4)
SERTİFİKA ZİNCİR KONTROL CÜLERİ <policy><validate><certificate><issuer>			
BasicConstraintCAChecker		Yayıncı Sertifikası üzerindeki <i>Temel Kısıtlamalar (BasicConstraints)</i> eklentisinin <i>RFC 5280</i> uyumluluğunu kontrol eder.	
CertificateKeyUsageChecker		Yayıncı Sertifikası üzerindeki <i>Anahtar Kullanımı (KeyUsage)</i> eklentisinin <i>RFC 5280</i> uyumluluğunu kontrol eder.	
CertificateNameChecker		Sertifika üzerindeki Yayıncı Özne Adı (<i>Issuer</i>) alanı ile yayıncı sertifikası üzerindeki <i>Özne Adı (Subject)</i> alanlarının eşleşmesi kuralını doğrular.	
CertificateSignatureChecker		Sertifika üzerindeki imzayı yayıncı sertifikasının açık anahtarı ile kriptografik olarak doğrular.	
KeyIdentifierChecker		Sertifika üzerindeki <i>Yetkili Anahtar Tanımlayıcısı (AuthorityKeyIdentifier)</i> eklentisi ile yayıncı sertifikası üzerinde yer alan <i>Özne Anahtar Tanımlayıcısı (SubjectKeyIdentifier)</i> eklentilerinin uyumluluğunu kontrol eder.	

NameConstraintsChecker	Sertifikanın <i>Özne Adı</i> ile yayıncı sertifikasında (varsa) yer alan <i>İsim Kısıtlamaları</i> (NameConstraints) arasındaki ilişkinin RFC 5280 uyumluluğunu kontrol eder.
PathLenConstraintChecker	Yayıncı sertifikası üzerindeki <i>Yol Uzunluğu Kısıtlamaları</i> (PathLengthConstraints) eklentisinin RFC 5280 uyumluluğunu kontrol eder.
PolicyConstraintsChecker	Yayıncı Sertifikası üzerindeki <i>Politika Kısıtlamaları</i> (PolicyConstraints) eklentisinin RFC 5280 uyumluluğunu kontrol eder.

SERTİFİKA İPTAL KONTROLCÜLERİ <policy><validate><certificate><revocation>			
RevocationFromCRLChecker	SİL üzerinden sertifikanın iptal durumunu kontrol eder.		
	Parametreler	cevrimdisicalis	[true, false*] Bu parametre doğrulamanın çevrimdışı olarak yürütüleceğini belirtir. True olarak tanımlanmışsa iptal kontrolcü hiçbir SİL bulamasa bile iptal kontrolünü başarılı olarak sonlandırır. Çevrimdışı ortamlarda SİL'e ulaşılamadığında bile sertifika doğrulamanın gerçekleşebilmesi için tanımlanmıştır. Dikkatli kullanılmalıdır.
		checkAllCRLs	[true, false*] Doğrulama sırasında bir tane geçerli SİL kontrolü, iptal kontrolünün tamamlanması için yeterlidir. Ancak bazı durumlarda kullanıcılar politika dosyasında tanımlı bütün SİL bulucuların getirdikleri SİL'lere bakılmasını isteyebilirler. Bu durumda bu değer true yapılmalıdır.
		devam	[true, false*] Doğrulama sırasında bir tane iptal kontrolcünün başarılı sonuçlanması, iptal kontrolünün tamamlanması için yeterlidir. Ancak kullanıcılar daha güvenli olması amacıyla bir iptal kontrolü başarılı sonuçlansa bile diğer iptal kontrolcülerin kontrollerine devam etmesini isteyebilirler. Bu durumda bu parametre true yapılmalıdır.

SİL YAPISAL KONTROLCÜLER <policy><validate><crl><crlself>	
CRLDateChecker	SİL üzerinde yer alan geçerlilik zaman aralığının doğrulama zamanını kapsamaları kuralını doğrular.
CRLExtensionChecker	SİL üzerinde yer alan eklenti bilgilerinin RFC 5280 uyumluluğunu kontrol eder.
SİL ZİNCİR KONTROLCÜLERİ <policy><validate><crl><crlissuer>	
CRLKeyUsageChecker	Yayıncı sertifikası üzerindeki <i>Anahtar Kullanımı (KeyUsage)</i> eklentisinin RFC 5280 uyumluluğunu kontrol eder.
CRLSignatureChecker	SİL üzerindeki imzayı, yayıncı sertifikasının açık anahtarı ile kriptografik olarak doğrular.
DELTA SİL KONTROLCÜLERİ <policy><validate><deltacrl>	
FreshestCRLChecker	Delta SİL üzerindeki <i>En Güncel SİL (FreshestCRL)</i> eklentisinin RFC 5280 uyumluluğunu kontrol eder.
DeltaCRLIndicatorChecker	Delta SİL üzerindeki <i>Delta SİL Belirteci (DeltaCRLIndicator)</i> eklentisinin RFC 5280 uyumluluğunu kontrol eder.

OCSP CEVABI KONTROL CÜLERİ<policy><validate><ocsp>	
SigningCertificateChecker	OCSP cevabını imzalayan yayıncı sertifikasını doğrular.
OCSPSignatureChecker	OCSP cevabı üzerindeki imzayı, yayıncı sertifikasının açık anahtarı ile kriptografik olarak doğrular.
ResponseStatusChecker	OCSP cevabı üzerindeki <i>Cevap Durumu</i> alanının geçerliliğini doğrular.
OCSPResponseDateChecker	OCSP üzerinde yer alan geçerlilik zaman aralığının doğrulama zamanını kapsamaması kuralını doğrular.

Table 2 MA3 API Eşleştirici (Matcher) Listesi

SERTİFİKA EŞLEŞTİRİCİLER<policy><match><certificate>	
IssuerSubjectMatcher	Sertifika üzerinde yer alan <i>issuer</i> alanı ile yayıncı sertifikası üzerinde yer alan <i>subject</i> alanını eşleştirir.
CertStoreCRLSaver	Sertifika doğrulama sırasında uzakta (LDAP, http vb.) bulunan SİL'leri yerel sertifika deposuna kaydeder.
SİL EŞLEŞTİRİCİLER<policy><match><crl>	
CRLDistributionPointMatcher	Sertifika üzerindeki <i>SİL Dağıtım Noktaları</i> (<i>CRLDistributionPoints</i>) eklentisi ile SİL üzerindeki (varsa) <i>Yayıncı Dağıtım Noktası</i> (<i>IssuingDistributionPoint</i>) eklentisine bakarak SİL ile sertifikayı eşleştirir.
CRLDistributionPointOnlyContainsMatcher	SİL üzerindeki <i>Yayıncı Dağıtım Noktası</i> (<i>Issuing Distribution Point</i>) eklentisindeki <i>onlyContains</i> özellikleri ile sertifika üzerindeki <i>Temel Kısıtlamalar</i> (<i>BasicConstraint</i>) eklentisine bakarak SİL ile sertifikayı eşleştirir.
CRLIssuerMatcher	SİL üzerindeki <i>Yayıncı Adı</i> (<i>issuer</i>) alanı ile sertifika üzerindeki <i>Yayıncı Adı</i> (<i>issuer</i>) alanına bakarak SİL ile sertifikayı eşleştirir.
CRLKeyIDMatcher	SİL üzerindeki <i>Yetkili Anahtar Tanımlayıcısı</i> (<i>AuthorityKeyIdentifier</i>) eklentisi ile sertifika üzerindeki <i>Yetkili Anahtar Tanımlayıcısı</i> (<i>AuthorityKeyIdentifier</i>) eklentisine bakarak SİL ile sertifikayı eşleştirir.

Delta SİL EŞLEŞTİRİCİLER<policy><match><delcacrl>	
BaseCRLNumberMatcher	Delta SİL üzerindeki <i>Temel SİL Numarası (BaseCRLNumber)</i> eklentisi ile temel SİL üzerindeki <i>SİL Numarası (CRLNumber)</i> eklentisine bakarak temel SİL ile delta SİL'i eşleştirir.
CRLNumberMatcher	Delta SİL üzerindeki <i>SİL Numarası (CRLNumber)</i> eklentisinde bulunan SİL numarasının, temel SİL üzerindeki <i>SİL Numarası (CRLNumber)</i> eklentisinde bulunan SİL numarasından büyük olduğunu doğrulayarak temel SİL ile delta SİL'i eşleştirir.
DeltaCRLIssuerMatcher	Delta SİL üzerindeki <i>Yayıncı Adı (issuer)</i> alanı ile temel SİL üzerindeki <i>Yayıncı Adı (issuer)</i> alanına bakarak temel SİL ile delta SİL'i eşleştirir.
ScopeMatcher	Delta SİL üzerindeki <i>Yayıncı Dağıtım Noktası (IssuingDistributionPoint)</i> eklentisi ile temel SİL üzerindeki <i>Yayıncı Dağıtım Noktası (IssuingDistributionPoint)</i> eklentisine bakarak temel SİL ile delta SİL'i eşleştirir.
OCSP CEVABI EŞLEŞTİRİCİLER<policy><match><ocsp>	
CertIDOCSPResponseMatcher	OCSP cevabı üzerindeki <i>Sertifika Tanımlayıcısı (CertID)</i> alanına bakarak sertifika ile OCSP cevabını eşleştirir.

ÇAPRAZ SERTİFİKA EŞLEŞTİRİCİLER<policy><match><crosscertificate>	
PublicKeyMatcher	Yayıncı sertifikasındaki açık anahtar ile çapraz sertifika üzerindeki açık anahtarı eşleştirir.
SKIMatcher	Yayıncı sertifikasındaki <i>Özne Anahtar Tanımlayıcısı (SubjectKeyIdentifier)</i> ile çapraz sertifika üzerindeki <i>Özne Anahtar Tanımlayıcısını (SubjectKeyIdentifier)</i> eşleştirir.
SubjectMatcher	Yayıncı sertifikasındaki <i>Özne Adı (subject)</i> alanı ile çapraz sertifika üzerindeki <i>Özne Adı (subject)</i> alanını eşleştirir.

Table 3 MA3 API Bulucu (Finder) listesi

GÜVENİLİR SERTİFİKA BULUCULAR<policy><find><trustedcertificate>			
TrustedCertificateFinderFromECertStore	Yerel sertifika deposundaki yayıncı sertifikalarını getirir ve bu sertifikaları güvenilir kabul eder.		
	Parametreler	securitylevel	[PERSONAL*, ORGANIZATIONAL, LEGAL] Depodan getirilecek sertifikaların güven seviyesini belirler. Kişisel sertifikalar için PERSONAL kurumsal sertifikalar için ORGANIZATIONAL kanuni sertifikalar için LEGAL olarak belirtilmelidir.
		storepath [0]	Yerel sertifika deposunun dosya sistemindeki yerini belirler.
TrustedCertificateFinderFromFileSystem	Dosya sisteminde verilen dizin adresindeki yayıncı sertifikalarını getirir. Bu sertifikaları güvenilir kabul eder.		
	Parametreler	dizin	Dizin adresi

TrustedCertificateFinderFromXml		Verilen URL adresindeki yayıncı sertifikalarını getirir. Bu sertifikaları güvenilir kabul eder.	
	Parametreler	storepath [0]	Sertifika deposunun okunacağı URL adresi.
		updatepath [0]	Yerel sertifika deposunun güncelleneceği URL adresi. Eğer storepath'de veya varsayılan dosya yolunda dosya yoksa bu adresten kopyalama yapılır. Belirli aralıklarla lokal dosya içerisindeki güvenilir kök sertifikalar updatepath adresinden güncellenir.
SERTİFİKA BULUCULAR<policy><find><certificate>			
CertificateFinderFromECertStore		Yerel sertifika deposundaki sertifikaları getirir.	
	Parametreler	storepath [0]	Yerel sertifika deposunun dosya sistemindeki yerini belirler.
CertificateFinderFromFile		Dosya sisteminde verilen dosya adresindeki sertifikaları getirir.	
	Parametreler	dosyayolu	Dosya adresi
CertificateFinderFromXml		Verilen URL adresindeki sertifikaları getirir.	

	Parametreler	storepath [0]	URL adresi
CertificateFinderFromHTTP		Sertifika üzerinde yer alan <i>Yetkili Erişim Bilgileri (AuthorityInfoAccess)</i> eklentisindeki HTTP adresinden yayıncı sertifikasını getirir.	
CertificateFinderFromLDAP		Sertifika üzerinde yer alan <i>Yetkili Erişim Bilgileri (AuthorityInfoAccess)</i> eklentisindeki LDAP adresinden yayıncı sertifikasını getirir.	

SİL BULUCULAR<policy><validate><certificate><revocation><find>				
CRLFinderFromECertStore		Yerel sertifika deposundaki SİL'leri getirir.		
		Parametreler	storepath [0]	Yerel sertifika deposunun dosya sistemindeki yerini belirler.
			getactivecrl	[true, false*] Sertifika doğrulama sonucunun kritik olduğu durumlarda, sertifika doğrulamanın yapıldığı tarihten sonra yayınlanan SİL(ler) kullanılır. Yeni yayınlanacak SİL(ler) için beklenmeyip, yayında olan SİL(ler) kullanılmak istendiğinde ise getactivecrl parametresi için true verilmelidir .
CRLFinderFromFile		Dosya sisteminde verilen dosya adresindeki SİL'leri getirir.		

	Parametreler	dosyayolu	Dosya adresi
CRLFinderFromHTTP		Sertifika üzerinde yer alan <i>SİL Dağıtım Noktaları</i> (<i>CRLDistributionPoints</i>) eklentisindeki HTTP adresinden SİL'i getirir.	
CRLFinderFromLDAP		Sertifika üzerinde yer alan <i>SİL Dağıtım Noktaları</i> (<i>CRLDistributionPoints</i>) eklentisindeki LDAP adresinden yayıncı sertifikasını getirir.	

OCSP CEVABI BULUCULAR<policy><validate><certificate><revocation><find>			
OCSPResponseFinderFromECertStore		Yerel sertifika deposundaki kayıtlı OCSP cevaplarını getirir.	
	Parametreler	storepath [0]	Yerel sertifika deposunun, dosya sistemindeki yerini belirler.
OCSPResponseFinderFromAIA		Sertifika üzerinde yer alan Yetkili Erişim Bilgileri (AuthorityInfoAccess) eklentisindeki OCSP adresine, OCSP sorgusu yaparak OCSP cevabı getirir.	
DELTA SİL BULUCULAR<policy><validate><certificate><revocation><find>			
DeltaCRLFinderFromECertStore		Yerel sertifika deposundaki Delta SİL'leri getirir.	
	Parametreler	storepath [0]	Yerel sertifika deposunun dosya sistemindeki yerini belirler.
DeltaCRLFinderFromFile		Dosya sisteminde verilen dosya adresindeki Delta SİL'leri getirir.	

	Parametreler	dosyayolu	Dosya adresi
--	--------------	-----------	--------------

ÇAPRAZ SERTİFİKA BULUCULAR<policy><validate><certificate><revocation><find><crosscertificate>			
CrossCertificateFinderFromECertStore		Yerel sertifika deposundaki çapraz sertifikaları getirir.	
	Parametreler	storepath [0]	Yerel sertifika deposunun dosya sistemindeki yerini belirler.
CrossCertificateFinderFromFile		Dosya sisteminde verilen dosya adresindeki çapraz sertifikaları getirir.	
	Parametreler	dosyayolu	Dosya adresi

Table 4 MA3 API Kaydedici (Saver) Listesi

KAYDEDİCİLER<policy><save>			
CertStoreCertificateSaver		Sertifika doğrulama sırasında bulunan ve doğrulanan sertifikaları yerel sertifika deposuna kaydeder.	
	Parametreler	storepath [0]	Yerel sertifika deposunun dosya sistemindeki yerini belirler.
CertStoreCRLSaver		Sertifika doğrulama sırasında uzakta (LDAP, http vb.) bulunan SİL'leri yerel sertifika deposuna kaydeder.	
	Parametreler	storepath [0]	Yerel sertifika deposunun dosya sistemindeki yerini belirler
CertStoreOCSPResponseSaver		Sertifika doğrulama sırasında uzakta (LDAP, http vb.) bulunan OCSP'leri yerel sertifika deposuna kaydeder.	
	Parametreler	storepath [0]	Yerel sertifika deposunun dosya sistemindeki yerini belirler.

4. CMS İMZA

4.1 Giriş

Bu doküman, MA3 API CMS Signature kütüphanesinin nasıl kullanılacağı hakkında bilgi vermektedir. CMS Signature kütüphanesi ile akıllı kartları yönetebilir, sertifika doğrulayabilir, elektronik imza atabilir ve elektronik imza doğrulayabilirsiniz. Kütüphaneyi kullanabilmeniz için API lisansına ihtiyacınız vardır.

İmza atma işlemi için kullanıcının sertifikaya ve özel anahtarını güvenli bir şekilde muhafaza edebileceği bir ögeye ihtiyacı vardır. Özel anahtarların muhafazası için genel olarak akıllı kart kullanıldığından, dokümanda da bu öge için **akıllı kart** kavramı kullanılacaktır. İmza atma hakkında ayrıntılı bilgi için [İmza Atma İşlemleri](#) bölümüne bakınız.

İmza doğrulama işlemleri, sertifikanın doğrulanmasından ve imzanın yapısal olarak doğrulanmasından oluşmaktadır. Sertifika doğrulama için sertifika deposuna ve sertifika doğrulamanın nasıl yapılacağını belirten politika dosyasına ihtiyaç vardır. Daha ayrıntılı bilgi için [İmza Doğrulama İşlemleri](#) bölümüne bakınız.

4.2 Gereker

MA3 API CMS Signature kütüphanesinin kullanılabilmesi için lisans dosyasına, sertifika doğrulama politika dosyasına ve sertifika deposu dosyasına ihtiyacınız vardır.

İmza doğrulama işlemi için ise yukarıdaki dosyalarla birlikte MA3 API kütüphanesi yeterli olacaktır. Kanuni geçerliliği olan nitelikli imzaların atılabilmesi için ise güvenli bir donanım kullanılması zorunluğu vardır. Genel kullanım olarak akıllı kart kullanılmaktadır.

Akıllı karta erişilebilmesi için akıllı kart okuyucusu sürücüsünün ve akıllı kart sürücüsünün kurulması gerekmektedir. Akıllı kart üreticisinin sağladığı kart izleme programı ile bilgisayarın karta erişimi kontrol edilebilir.

Hızlı bir başlangıç için [Hızlı Başlangıç](#) bölümüne bakabilirsiniz.

4.3 İmza Tipleri

API, ETSI TS 101 733 dokümanında anlatılan aşağıdaki imza tiplerini desteklemektedir:

1. CAdES-BES (Basic Elektronik Signature-Basit Elektronik İmza)
2. CAdES-EPES (Explicit Policy Based Electronic Signature- Belirlenmiş Politika Temelli Elektronik İmza)
3. CAdES-T (Electronic Signature with Time- Zaman Damgası Eklenmiş Elektronik İmza)
4. CAdES-C (Electronic Signature with Complete Validation Data References-Tüm Doğrulama Verilerinin Referanslarının Eklendiği İmza)

5. CAdES-X-Long (EXtended Long Electronic Signature- Genişletilmiş Uzun Elektronik İmza)
6. CAdES-X-Type 1 (EXtended Electronic Signature with Time Type 1- Genişletilmiş Elektronik İmza Tip 1 Zamanlı)
7. CAdES-X-Type 2 (EXtended Electronic Signature with Time Type 1- Genişletilmiş Elektronik İmza Tip 2 Zamanlı)
8. CAdES-X-Long-Type 1 or Type 2 (EXtended Long Electronic Signature with Time Type 1- Genişletilmiş Uzun Elektronik İmza Tip 1 veya Tip 2 Zamanlı)
9. CAdES-A (Archival Electronic Signature- Arşiv Elektronik İmza)

Yukarıdaki imza tiplerinin detayı için ETSI TS 101 733 dokümanına bakılabilir. Ancak burada da imza tipleri için kısa açıklamalar verilmiştir. Açıklamalarda ETSI TS 101 733 dokümanında yer alan şekillerden faydalanılmıştır.

4.4 İmza Atma İşlemleri

İmzalama işlemi genel olarak iki aşamada gerçekleşmektedir. Öncelikle imzacı sertifikasının doğrulanma işlemi yapılmakta ve sonrasında imza atma işlemi gerçekleştirilmektedir.

4.5 İmzasız Bir Verinin İmzalanması

Veriyi imzalama işleminden *BaseSignedData* sınıfı sorumludur. Bu sınıfa öncelikle *addContent(...)* fonksiyonu ile imzalanacak veri eklenmelidir. *addContent(...)* fonksiyonu yalnızca bir kere çağırılmalıdır. İmzalanacak veri *addContent(...)* ile eklendikten sonra değiştirilemez. *addSigner(...)* fonksiyonu ile veriye imza bilgileri eklenir.

İmza eklenirken imzanın türü, imzacının sertifikası, imza işlemi gerçekleştirecek kripto nesnesi, varsa ekstra imza özellikleri ve imza üretiminde kullanılması gereken parametreler *addSigner(...)* fonksiyonuna parametre olarak geçilmelidir. İmza atan örnek kod bloğu:

Java

```
BaseSignedData bs = new BaseSignedData();
ISignable content = new SignableByteArray("test".getBytes());
bs.addContent(content);

HashMap<String, Object> params = new HashMap<String, Object>();

//if the user does not want certificate validation at generating signature, he can
add
//P_VALIDATE_CERTIFICATE_BEFORE_SIGNING parameter with its value set to false
//params.put(EParameters.P_VALIDATE_CERTIFICATE_BEFORE_SIGNING, false);

//necessary for certificate validation. By default, certificate validation is done
params.put(EParameters.P_CERT_VALIDATION_POLICY, getPolicy());
```

```

//By default, QC statement is checked and signature won't be created if it is not a
//qualified certificate.
boolean checkQCStatement = isQualified();

//Get qualified or non-qualified certificate.
ECertificate cert =
SmartCardManager.getInstance().getSignatureCertificate(checkQCStatement);
BaseSigner signer = SmartCardManager.getInstance().getSigner(getPin(), cert);

//add signer
//Since the specified attributes are mandatory for bes, null is given as parameter
//for optional attributes
bs.addSigner(ESignatureType.TYPE_BES, cert, signer, null, params);

SmartCardManager.getInstance().logout();

byte[] signedDocument = bs.getEncoded();

//write the contentinfo to file
AsnIO.dosyayaz(signedDocument, getTestDataFolder() + "testdata/BES-1.p7s");

```

C#

```

BaseSignedData bs = new BaseSignedData();
ISignable content = new SignableByteArray(ASCIIEncoding.ASCII.GetBytes("test"));
bs.addContent(content);

Dictionary<String, Object> params_ = new Dictionary<String, Object>();
//if the user does not want certificate validation at generating signature, he can
add
//P_VALIDATE_CERTIFICATE_BEFORE_SIGNING parameter with its value set to false
params_[EParameters.P_VALIDATE_CERTIFICATE_BEFORE_SIGNING] = false;

//necessary for certificate validation. By default, certificate validation is done
params_[EParameters.P_CERT_VALIDATION_POLICY] = getPolicy();

//By default, QC statement is checked and signature won't be created if it is not a
//qualified certificate.

bool checkQCStatement = isQualified();

//Get qualified or non-qualified certificate.
ECertificate cert =
SmartCardManager.getInstance().getSignatureCertificate(checkQCStatement);

BaseSigner signer = SmartCardManager.getInstance().getSigner(getPin(), cert);

```

```

//add signer
//Since the specified attributes are mandatory for bes, null is given as parameter
//for optional attributes
try
{
    bs.addSigner(ESignatureType.TYPE_BES, cert, signer, null, params_);
}
catch (CertificateValidationException cve)
{
    Console.WriteLine(cve.getCertStatusInfo().getDetailedMessage());
}

SmartCardManager.getInstance().logout();

byte[] signedDocument = bs.getEncoded();

//write the contentinfo to file
DirectoryInfo di = Directory.CreateDirectory(testDataDic+"\\testVerileri");
AsnIO.dosyayaz(signedDocument, di.FullName + "\\BES-1.p7s");

```

4.6 İmzalı Bir Veriye İmza Eklenmesi

Bir veri birkaç kişi tarafından imzalanabilir. İmzalar iki şekilde atılabilir.

- Paralel İmza Ekleme
- Seri İmza Ekleme

4.6.1 Paralel İmza

Bu tür imzalarda bütün imzacıların imzaladıkları veri aynı veridir. Bütün imzalar aynı seviyededir ve bir imzacının imzası dokümandan çıkartılırsa fark edilemez.

Java

```

byte[] signature = AsnIO.dosyadanOKU(SIGNATURE_FILE);
BaseSignedData bs = new BaseSignedData(signature);

//create parameters necessary for signature creation
HashMap<String, Object> params = new HashMap<String, Object>();
ValidationPolicy policy = PolicyReader.readValidationPolicy(new
FileInputStream(POLICY_FILE));
params.put(EParameters.P_CERT_VALIDATION_POLICY, policy);
/*necessary for certificate validation. By default, certificate validation is done.
But if the user does not want certificate validation, he can add
P_VALIDATE_CERTIFICATE_BEFORE_SIGNING parameter with its value set to false*/
//By default, QC statement is checked and signature won't be created if it is not a
//qualified certificate.

```

```

boolean checkQCStatement = isQualified();

//Get qualified or non-qualified certificate.
ECertificate cert =
SmartCardManager.getInstance().getSignatureCertificate(checkQCStatement,
!checkQCStatement);
BaseSigner signer = SmartCardManager.getInstance().getSigner(getPin(), cert);

//add signer. Since the specified attributes are mandatory for bes, null is given as
//parameter for optional attributes
bs.addSigner(ESignatureType.TYPE_BES, cert, signer, null, params);

//write the contentinfo to file
AsnIO.dosyayaz(bs.getEncoded(),NEW_SIGNATURE_ADDED_FILE);

SmartCardManager.getInstance().logout();

```

C#

```

byte[] signature = AsnIO.dosyadanOKU(SIGNATURE_FILE);
BaseSignedData bs = new BaseSignedData(signature);

//create parameters necessary for signature creation
Dictionary<String, Object> params_ = new Dictionary<String, Object>();
ValidationPolicy policy = PolicyReader.readValidationPolicy(new
FileStream(POLICY_FILE, FileMode.Open, FileAccess.Read));
params_[EParameters.P_CERT_VALIDATION_POLICY] = policy;

/*necessary for certificate validation. By default, certificate validation is done.
But if the user does not want certificate validation, he can add
P_VALIDATE_CERTIFICATE_BEFORE_SIGNING parameter with its value set to false*/

bool checkQCStatement = isQualified();

//Get qualified or non-qualified certificate.
ECertificate cert =
SmartCardManager.getInstance().getSignatureCertificate(checkQCStatement);

BaseSigner signer = SmartCardManager.getInstance().getSigner(getPin(), cert);

//add signer. Since the specified attributes are mandatory for bes, null is given as
//parameter for optional attributes
bs.addSigner(ESignatureType.TYPE_BES, cert, signer, null, params_);

//write the contentinfo to file
AsnIO.dosyayaz(bs.getEncoded(), NEW_SIGNATURE_ADDED_FILE);

SmartCardManager.getInstance().logout();

```

4.6.2 Seri İmza

Seri imza eklerken, imzanın eklendiği seviyeye kadar olan bütün imzacıların imzası ve imzalanmak istenen veri imzalanır. Dolayısıyla bir imzacının imzası çıkartılırsa o imzacıdan sonra imza atan imzacıların da imzalarının çıkartılması gerekmektedir.

Aşağıdaki kod örneğinde ilk imzacıya seri imza eklenmektedir.

Java

```
byte[] signature = AsnIO.dosyadanOKU(SIGNATURE_FILE);
BaseSignedData bs = new BaseSignedData(signature);

//create parameters necessary for signature creation
HashMap<String, Object> params = new HashMap<String, Object>();
ValidationPolicy policy = PolicyReader.readValidationPolicy(new
FileInputStream(POLICY_FILE));
params.put(EParameters.P_CERT_VALIDATION_POLICY, policy);
/*necessary for certificate validation. By default, certificate validation is done.
But if the user does not want certificate validation, he can add
P_VALIDATE_CERTIFICATE_BEFORE_SIGNING parameter with its value set to false*/

//By default, QC statement is checked and signature won't be created if it is not a
//qualified certificate.
boolean checkQCStatement = isQualified();

//Get qualified or non-qualified certificate.
ECertificate cert =
SmartCardManager.getInstance().getSignatureCertificate(checkQCStatement);
BaseSigner signer = SmartCardManager.getInstance().getSigner(getPin(), cert);

Signer firstSigner = bs.getSignerList().get(0);

firstSigner.addCounterSigner(ESignatureType.TYPE_BES, cert, signer, null, params);

//write the contentinfo to file
AsnIO.dosyayaz(bs.getEncoded(),NEW_SIGNATURE_ADDED_FILE);
SmartCardManager.getInstance().logout();
```

C#

```

byte[] signature = AsnIO.dosyadanOKU(SIGNATURE_FILE);
BaseSignedData bs = new BaseSignedData(signature);

//create parameters necessary for signature creation
Dictionary<String, Object> params_ = new Dictionary<String, Object>();
ValidationPolicy policy = PolicyReader.readValidationPolicy(new
FileStream(POLICY_FILE, FileMode.Open, FileAccess.Read));
params_[EParameters.P_CERT_VALIDATION_POLICY] = policy;
/*necessary for certificate validation. By default, certificate validation is done.
But if the user does not want certificate validation, he can add
P_VALIDATE_CERTIFICATE_BEFORE_SIGNING parameter with its value set to false*/

bool checkQCStatement = isQualified();

//Get qualified or non-qualified certificate.
ECertificate cert =
SmartCardManager.getInstance().getSignatureCertificate(checkQCStatement);

BaseSigner signer = SmartCardManager.getInstance().getSigner(getPin(), cert);

Signer firstSigner = bs.getSignerList()[0];

firstSigner.addCounterSigner(ESignatureType.TYPE_BES, cert, signer, null, params_);

//write the contentinfo to file
AsnIO.dosyayaz(bs.getEncoded(), NEW_SIGNATURE_ADDED_FILE);
SmartCardManager.getInstance().logout();

```

4.7 Ayırık İmza

Ayırık imzada imzalanacak veri *BaseSignedData.addContent(...)* fonksiyonunun ikinci parametresi false verilerek atanır.

Dahili imza ile büyük boyutlu dosyalar imzalanamaz. İmzanın yapısı gereği imzalanacak verinin hepsi belleğe alınmaktadır. Bundan dolayı büyük boyutlu dosyaların imzalanması için ayırık imza kullanılması gerekmektedir.

Java

```

BaseSignedData bs = new BaseSignedData();

File file = new File(MOVIE_FILE);
ISignable signable = new SignableFile(file, 2048);
bs.addContent(signable, false);

//create parameters necessary for signature creation
HashMap<String, Object> params = new HashMap<String, Object>();

```

```

ValidationPolicy policy = PolicyReader.readValidationPolicy(new
FileInputStream(POLICY_FILE));
params.put(EParameters.P_CERT_VALIDATION_POLICY, policy);

//By default, QC statement is checked and signature won't be created if it is not a
//qualified certificate.
boolean checkQCStatement = isQualified();

//Get qualified or non-qualified certificate.
ECertificate cert =
SmartCardManager.getInstance().getSignatureCertificate(checkQCStatement);
BaseSigner signer = SmartCardManager.getInstance().getSigner(getPin(), cert);

bs.addSigner(ESignatureType.TYPE_BES, cert, signer, null, params);

AsnIO.dosyayaz(bs.getEncoded(), SIGNATURE_FILE);

SmartCardManager.getInstance().logout();

```

C#

```

BaseSignedData bs = new BaseSignedData();

FileInfo file = new FileInfo(MOVIE_FILE);
ISignable signable = new SignableFile(file, 2048);
bs.addContent(signable, false);

//create parameters necessary for signature creation
Dictionary<String, Object> params_ = new Dictionary<String, Object>();

ValidationPolicy policy = PolicyReader.readValidationPolicy(new
FileStream(POLICY_FILE, FileMode.Open, FileAccess.Read));
params_[EParameters.P_CERT_VALIDATION_POLICY] = policy;

bool checkQCStatement = isQualified();

//Get qualified or non-qualified certificate.
ECertificate cert =
SmartCardManager.getInstance().getSignatureCertificate(checkQCStatement);

BaseSigner signer = SmartCardManager.getInstance().getSigner(getPin(), cert);

bs.addSigner(ESignatureType.TYPE_BES, cert, signer, null, params_);

AsnIO.dosyayaz(bs.getEncoded(), SIGNATURE_FILE);

SmartCardManager.getInstance().logout();

```

4.8 Farklı İmza Tiplerinin Oluşturulması

API tarafından desteklenen imza tiplerinden çok kullanılanlar hakkında kısa açıklamaları burada bulabilirsiniz. Hangi imzanın size uygun olduğuna karar vermek için **E-imza Profilleri** dokümanını inceleyebilirsiniz. İmza tipleri hakkında daha geniş bilgi için ise **ETSI TS 101 733** dokümanına bakınız.

4.8.1 BES

BES imza, en basit imza türüdür. BES imza sadece o kişinin imzayı attığını garanti eder. İmza zamanı belli olmadığından ancak sertifika geçerli iken imza doğrulanabilir. Sertifika iptal edildiğinde veya sertifika süresi dolduğunda imza doğrulanamaz. BES imza içersine zaman bilgisi eklenebilir fakat eklenen zamanın herhangi bir hukuki yükümlülüğü, kesinliği yoktur. Eklenen bu zaman bilgisi, sadece beyan edilen zaman şeklinde kullanılabilir.

Java

```
BaseSignedData bs = new BaseSignedData();
bs.addContent(new SignableByteArray("test".getBytes()));

//Since SigningTime attribute is optional, add it to optional attributes list
List<IAAttribute> optionalAttributes = new ArrayList<IAAttribute>();
optionalAttributes.add(new SigningTimeAttr(Calendar.getInstance()));

HashMap<String, Object> params = new HashMap<String, Object>();
params.put(EParameters.P_CERT_VALIDATION_POLICY, VALIDATION_POLICY);

bs.addSigner(ESignatureType.TYPE_BES, cert, signer, optionalAttributes, params);
```

C#

```
BaseSignedData bs = new BaseSignedData();
bs.addContent(new SignableByteArray(Encoding.ASCII.GetBytes("test")));

//Since SigningTime attribute is optional, add it to optional attributes list
List<IAAttribute> optionalAttributes = new List<IAAttribute>();
optionalAttributes.Add(new SigningTimeAttr(DateTime.UtcNow));

Dictionary<String, Object> params_ = new Dictionary<String, Object>();
params_[EParameters.P_CERT_VALIDATION_POLICY] = VALIDATION_POLICY;

bs.addSigner(ESignatureType.TYPE_BES, cert, signer, optionalAttributes, params_);
```


İmzanın beyan edilen zamanını almak için aşağıdaki örnek kod kullanılabilir.

Java

```
byte[] input = AsnIO.dosyadanOKU(BESwithSIGNING_TIME);
BaseSignedData bs = new BaseSignedData(input);

List<EAttribute> attrs =
bs.getSignerList().get(0).getSignedAttribute(SigningTimeAttr.OID);

Calendar time = SigningTimeAttr.toTime(attrs.get(0));
System.out.println(time.getTime().toString());
```

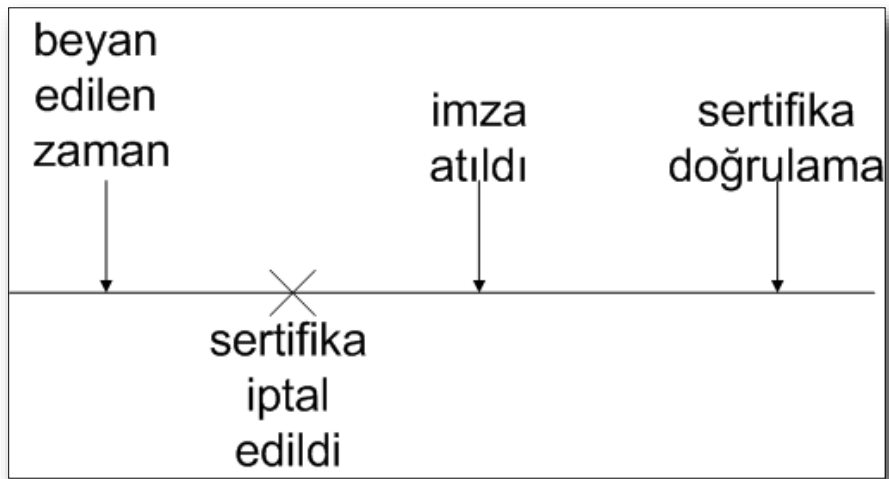
C#

```
byte[] input = AsnIO.dosyadanOKU(BESwithSIGNING_TIME);
BaseSignedData bs = new BaseSignedData(input);

List<EAttribute> attrs = bs.getSignerList()[0].
    getSignedAttribute(AttributeOIDs.id_signingTime);

DateTime? time = SigningTimeAttr.toTime(attrs[0]);
Console.WriteLine(time.Value.ToString());
```

Beyan edilen zamanın kullanımında oluşabilecek kötü senaryo Şekil 17'deki gibidir. İmza atıldığı sırada imzacı sertifikası iptal edilmiştir; yalnız kullanıcı imza zamanı olarak daha önceki bir zamanı beyan etmiştir. Beyan edilen zamana güvenildiği durumda geçersiz olan bu imza doğrulanacaktır.



Şekil 17: Yanlış Beyan ile Kötü Kullanım Senaryosu

4.8.2 EST

EST imza, BES imzadan türemiştir. İçerisinde imzalama zamanını gösterir zaman damgası bulundurmaktadır. İmza atılırken zaman damgası ayarlarının verilmesi gerekmektedir. Aşağıdaki örnek kodda bir EST imzanın nasıl atılacağını bulabilirsiniz.

Java

```
BaseSignedData bs = new BaseSignedData();
bs.addContent(new SignableByteArray("test".getBytes()));

HashMap<String, Object> params = new HashMap<String, Object>();

//for getting signaturetimestamp
TSSettings tsSettings = new TSSettings("http://tzd.kamusm.gov.tr", 1,
"12345678".toCharArray(), DigestAlg.SHA256);
params.put(ESignatureParameters.P_TSS_INFO, tsSettings);
params.put(ESignatureParameters.P_CERT_VALIDATION_POLICY, getPolicy());

//add signer
bs.addSigner(ESignatureType.TYPE_EST, cert, signer, null, params);
```

C#

```
BaseSignedData bs = new BaseSignedData();
bs.addContent(new SignableByteArray(Encoding.ASCII.GetBytes("test")));

Dictionary<String, Object> params_ = new Dictionary<String, Object>();

//for getting signaturetimestamp
TSSettings tsSettings = new TSSettings("http://tzd.kamusm.gov.tr", 1, "12345678",
DigestAlg.SHA256);
params_[ESignatureParameters.P_TSS_INFO] = tsSettings;
params_[ESignatureParameters.P_CERT_VALIDATION_POLICY] = getPolicy();

//add signer
bs.addSigner(ESignatureType.TYPE_EST, cert, signer, null, params_);
```

BES tipi imzadan farklı olarak EST tipi imzada imza zamanı belli olduğundan, sertifika doğrulamada kesin sonuçlara ulaşılabilir. Aşağıdaki örnek kod ile imza zamanını alabilirsiniz.

Java

```
BaseSignedData bs = new BaseSignedData(input);
EST estSign = (EST)bs.getSignerList().get(0);
Calendar time = estSign.getTime();
System.out.println(time.getTime().toString());
```

C#

```
BaseSignedData bs = new BaseSignedData(input);
EST estSign = (EST)bs.getSignerList()[0];
Console.WriteLine(time.Value.ToString());
DateTime? time = estSign.getTime();
```

4.8.3 ESXLong

ESXLong imza aynı zamanda bir EST imzadır. ESXLong imza, EST imzadan farklı olarak imza içerisinde sertifika doğrulamada kullanılacak doğrulama verisini içerir. Bundan dolayı ESXLong imza atılırken veya bir imza ESXLong'a çevrilirken sertifika doğrulama işlemi yapılmak zorundadır. Sertifika ömrü dolduktan sonra doğrulamada kullanılacak veriye ulaşmada sorunlar çıkabilir. ESXLong imza, doğrulama verisini içinde barındırdığından bu tür sorunların çıkmasını engeller. EST imza atılırken kullanılan örnek kodun aynısı, sadece eklenecek imza türü ESXLong olarak değiştirilerek kullanılabilir.

4.8.4 ESA

ESA tipi imza, kriptografik algoritmaların zamanla güvenilirliğini kaybetmesine karşı geliştirilmiş bir imza türüdür. Şu anda güvenerek kullandığımız algoritmalar, 5-10 yıl sonra güvenilemez duruma gelebilir. Bu algoritmalar güvenilirmez duruma geçmeden önce, imzaların ESA tipine çevrilmesi gerekmektedir. Yalnız imza atıldıktan hemen sonra imzanın ESA'ya çevrilmesi fazladan bir güvenlik sağlamaz. Algoritmaların bir kısmı güvenilirmez duruma geçerken daha güvenilir yeni algoritmalar kullanılmaya başlanacaktır. ESA imza tipine çevrim sırasında bu yeni algoritmalar kullanılmalıdır.

ESA'yı kullanmanın bir diğer amacı imza üzerinde değişiklik yapılmasını engellemek olabilir. Eğer seri imza kullanılıyorsa ESA'ya çevrilen imzanın altındaki imzaların veri yapısında bir değişiklik yapılamaz. Bu değişiklikler; imza türünün değiştirilmesi, imzanın silinmesi ve yeni bir imza eklenmesi olabilir. Arşiv zaman damgası v2 özelliği bulunduran ESA tipindeki imzaya, yeni bir seri imza veya doğrulama verisi eklenemez. Arşiv zaman damgası v3 özelliği bulunduran ESA tipindeki imzada ise bu eklemeler yapılabilir.

Bir imza atılırken ESA tipinde atılamaz. Öncelikle başka bir tipte atılmalı, daha sonra ESA tipine çevrilmelidir. Nasıl yapılacağına [İmza Tipleri Arasında Dönüşüm](#) bölümünden bakabilirsiniz.

ESA imza tipine dönüşüm sırasında, arşiv tipi zaman damgası kullanılmaktadır. Bundan dolayı parametreler yardımıyla zaman damgası ayarları verilmelidir.

Java

```
byte[] signatureFile = AsnIO.dosyadanOKU(getTestDataFolder() + "ESXLong-1.p7s");
BaseSignedData bs = new BaseSignedData(signatureFile);

Map<String, Object> parameters = new HashMap<String, Object>();

//Archive time stamp is added to signature, so time stamp settings are needed.
parameters.put(EParameters.P_TSS_INFO, getTSSettings());
parameters.put(EParameters.P_CERT_VALIDATION_POLICY, getPolicy());

bs.getSignerList().get(0).convert(ESignatureType.TYPE_ESA, parameters);

AsnIO.dosyayaz(bs.getEncoded(), getTestDataFolder() + "ESA-2.p7s");
```

C#

```
byte[] content = AsnIO.dosyadanOKU(getTestDataFolder() + "ESXLong-1.p7s");
BaseSignedData bs = new BaseSignedData(content);

Dictionary<String, Object> parameters = new Dictionary<String, Object>();

//Archive time stamp is added to signature, so time stamp settings are needed.
parameters[EParameters.P_TSS_INFO] = getTSSettings();
parameters[EParameters.P_CERT_VALIDATION_POLICY] = getPolicy();

bs.getSignerList()[0].convert(ESignatureType.TYPE_ESA, parameters);

AsnIO.dosyayaz(bs.getEncoded(), getTestDataFolder() + "ESA-2.p7s");
```

4.9 Zorunlu Olmayan Özelliklerin Eklenmesi

İmzaya adres bilgisi, imza zamanı bilgisi gibi opsiyonel bilgileri özellik olarak ekleyebilirsiniz. API'de tanımlı olan ve kullanabileceğiniz özellikler:

SigningTimeAttr

Beyan edilen imza zamanını içerir. Ancak beyan edilen bu tarih güvenilir bir tarih olmadığından imzanın bu tarihte atıldığını garanti etmez, bilgi amaçlı kullanılabilir.

SignerLocationAttr

İmzacının adresi hakkında bilgiler içerir. İmzacının ülkesi, şehri ve posta adresi belirtilebilir. Bilgi amaçlı olduğundan bu bilgilerden bazılarının değeri null olabilir.

CommitmentTypeIndicationAttr	İmza amacını belirtmek için kullanılabilir. İmzalanan verinin tarafınızdan oluşturulmuş olduğunu, sadece imzanın içeriğini onayladığınızı vs. belirtebilirsiniz. <i>CommitmentType</i> sınıfında tanımlanmış aşağıdaki değerler verilebilir.
RECEIPT	İmza sahibinin imzalı belgeyi aldığını (bir yerden geliyor ise) belirtmek için kullanılır.
SENDER	İmzalı veriyi gönderenin (imzalı veri bir yere gönderiliyor ise) veriyi gönderen kişi olduğunu belirtmek için kullanılır. Yani imza sahibinin gönderilen verinin içeriğini onayladığı anlamına gelmez sadece bunu ben gönderdim demektir.
APPROVAL	İmza sahibinin, belgenin içeriğini onayladığını belirtmek için kullanılır.
APPROVAL, DELIVERY	Bir mesaj gönderildiğinde, bu mesajın karşı tarafa iletildiğini belirtmede kullanılır. Bu tür bir imza genelde güvenilir servis sağlayıcılar (TSP-Trusted Service Provider) tarafından kullanılır.
CREATION	İmza sahibinin, belgeyi oluşturan kişi olduğunu belirtmek için kullanılır. İmza sahibinin, belge içeriğini onayladığı veya gönderdiği anlamına gelmez.
ORIGIN	İmza sahibinin belgeyi oluşturduğunu, içeriğini onayladığını ve gönderenin de kendisi olduğunu belirtmek için kullanılır.
ContentIdentifierAttr	İmzalanan içeriği tanımlamak için kullanılır. Özellikle ayrıık imzada imzalanan dokümanı imza ile eşleştirmek için kullanılabilir. byte[] olacak şekilde herhangi bir değer olabilir.
ContentHintsAttr	İmzalanan içerik hakkında alıcıya fikir vermek amacıyla kullanılır.
SignerAttributesAttr	İmzalayan kişi hakkında bilgiler içerir. İmzalayanın iddia ettiği özellikleri veya imzalayanın yetki sertifikasını barındırabilir.

İmza atma sırasında imzaya eklenmek istenen özellikler, bir listeye konularak kütüphaneye verilir. Eklenecek alanın değeri, alanın yaratılması sırasında kurucu fonksiyona verilir. Aşağıda yer alan örnekteki gibi imzaya eklenirler ve imzadan okunurlar.

Java

```
List<IAAttribute> optionalAttributes = new ArrayList<IAAttribute>();
optionalAttributes.add(new SigningTimeAttr(Calendar.getInstance()));
optionalAttributes.add(new SignerLocationAttr("TURKEY", "KOCAELİ", new
String[]{"TUBITAK UEKAE", "GEBZE"}));
optionalAttributes.add(new CommitmentTypeIndicationAttr(CommitmentType.CREATION));
optionalAttributes.add(new ContentIdentifierAttr("PL123456789".getBytes("ASCII")));

bs.addSigner(ESignatureType.TYPE_BES, cert, signer, optionalAttributes, params);

SmartCardManager.getInstance().logout();

//reading Attributes
BaseSignedData bs2 = new BaseSignedData(bs.getEncoded());
List<EAAttribute> attrs;
Signer aSigner = bs2.getSignerList().get(0);

attrs = aSigner.getAttribute(SigningTimeAttr.OID);
Calendar st = SigningTimeAttr.toTime(attrs.get(0));
System.out.println("Signing time: " + st.getTime());

attrs = aSigner.getAttribute(SignerLocationAttr.OID);
ESignerLocation sl = SignerLocationAttr.toSignerLocation(attrs.get(0));
StringBuilder sb = new StringBuilder();
for(String address : sl.getPostalAddress())
sb.append(" " + address);
System.out.println("\nCountry: " + sl.getCountry() +
"\nCity: " + sl.getLocalityName() + "\nAddress: " + sb);
attrs = aSigner.getAttribute(ContentIdentifierAttr.OID);
byte[] ci = ContentIdentifierAttr.toIdentifier(attrs.get(0));
System.out.println("\n" + Arrays.toString(ci));

attrs = aSigner.getAttribute(CommitmentTypeIndicationAttr.OID);
CommitmentType ct = CommitmentTypeIndicationAttr.toCommitmentType(attrs.get(0));
System.out.println("\n" + ct);
```

C#

```
List<IAAttribute> optionalAttributes = new List<IAAttribute>();
optionalAttributes.Add(new SigningTimeAttr(DateTime.UtcNow));
optionalAttributes.Add(new SignerLocationAttr("TURKEY", "KOCAELI",
new String[] { "TUBITAK UEKAE", "GEBZE" }));
optionalAttributes.Add(new CommitmentTypeIndicationAttr(CommitmentType.CREATION));
optionalAttributes.Add(new ContentIdentifierAttr(
ASCIIEncoding.ASCII.GetBytes("PL123456789")));

bs.addSigner(ESignatureType.TYPE_BES, cert, signer, optionalAttributes, params_);

SmartCardManager.getInstance().logout();
```

```
//reading Attributes
BaseSignedData bs2 = new BaseSignedData(bs.getEncoded());
List<EAttribute> attrs;
Signer aSigner = bs2.getSignerList()[0];

attrs = aSigner.getAttribute(SigningTimeAttr.OID);
DateTime? st = SigningTimeAttr.toTime(attrs[0]);
Console.WriteLine("Signing time: " + st.Value.ToLocalTime().ToString());

attrs = aSigner.getAttribute(SignerLocationAttr.OID);
ESignerLocation sl = SignerLocationAttr.toSignerLocation(attrs[0]);
StringBuilder sb = new StringBuilder();
foreach(String address in sl.getPostalAddress())
    sb.Append(" " + address);

Console.WriteLine("\nCountry: " + sl.getCountry() +
"\nCity: " + sl.getLocalityName() + "\nAddress: " + sb);

attrs = aSigner.getAttribute(ContentIdentifierAttr.OID);
byte[] ci = ContentIdentifierAttr.toIdentifier(attrs[0]);
Console.WriteLine("\n" + BitConverter.ToString(ci));

attrs = aSigner.getAttribute(CommitmentTypeIndicationAttr.OID);
CommitmentType ct = CommitmentTypeIndicationAttr.toCommitmentType(attrs[0]);
Console.WriteLine("\n" + ct);
```

4.10 Sertifika Doğrulama

İmza atma işleminden önce sertifika doğrulama işlemi yapılmaktadır. Bunun amacı, iptal edilmiş veya hatalı bir sertifika ile imza atılmasını önlemektir. İmzacı sertifikasının doğrulanması parametreler yardımıyla devre dışı bırakılabilir. Bu işlem için [Parametreler](#) bölümünde yer alan P_VALIDATE_CERTIFICATE_BEFORE_SIGNING parametresi incelenebilir.

Sertifika doğrulamanın başarısız olduğu durumda CertificateValidationException hatası alınır. Sertifika doğrulama ile ilgili ayrıntılı bilgi için [Sertifika Doğrulama](#) bölümüne bakılabilir.

4.11 İmza Doğrulama İşlemleri

İmza doğrulama işlemi, birçok kontrolcünün bir araya gelmesiyle yapılmaktadır. Bu kontrolcüler imzanın yapısal kontrollerinden ve sertifika kontrollerinden oluşmaktadır. Sertifika kontrolleri; sertifikanın yapısal kontrollerinden, sertifikanın güvenilir bir kökten verilip verilmediği kontrolünden ve sertifika iptal kontrolünden oluşmaktadır. Sertifika iptal kontrolü dışında kalan kontroller, matematiksel işlemlerden oluşmaktadır ve ekstra bir bilgiye ihtiyaç duymamaktadır. Sertifika iptal kontrolü için ise sertifikayı verenin yayınlandığı uygun zamanlı SİL dosyasına veya ÇİSDUP bilgisine ihtiyaç duyulmaktadır. Sertifika doğrulama işlemi için sertifika doğrulama işleminin konfigürasyonunu barındıran xml formatındaki politika dosyasına ihtiyaç vardır. Bu xml

dosyası dışında birçok parametre kullanılabilir. Bu parametrelere [Parametreler](#) bölümünden veya *EParameters* sınıfından bakılabilir.

Bir imzalanmış verinin doğrulama işleminden *SignedDataValidation* sınıfı sorumludur. *SignedDataValidation* sınıfının *verify(...)* metodu kullanılarak imza doğrulama işlemi gerçekleştirir. *verify(...)* fonksiyonu, imzalanmış verinin `byte[]` halini ve imzalanmış verinin hangi parametrelere göre doğrulanacağı bilgisini alır. *SignedDataValidation* sınıfındaki *verify(...)* fonksiyonu ile imzalanmış doküman bir bütün halinde doğrulanır. Doküman içindeki imzalardan biri dahi doğrulanamamışsa, bu fonksiyon başarısız değer döner. İmza doğrulama yapan örnek kod bloğu:

Java

```
byte[] signedData = AsnIO.dosyadanOKU(SIGNATURE_FILE);
ValidationPolicy policy = PolicyReader.readValidationPolicy(new
    FileInputStream(POLICY_FILE));

Hashtable<String, Object> params = new Hashtable<String, Object>();
params.put(EParameters.P_CERT_VALIDATION_POLICY, policy);

SignedDataValidation sdv = new SignedDataValidation();

SignedDataValidationResult sdvr = sdv.verify(signedData, params);

if(sdvr.getSDStatus() != SignedData_Status.ALL_VALID)
    System.out.println("İmzaların hepsi doğrulanmadı.");
System.out.println(sdvr.toString());
```

C#

```
byte[] signedData = AsnIO.dosyadanOKU(SIGNATURE_FILE);
ValidationPolicy policy = PolicyReader.readValidationPolicy(new
    FileStream(POLICY_FILE, FileMode.Open, FileAccess.Read));

Dictionary<String, Object> params_ = new Dictionary<String, Object>();
params_[EParameters.P_CERT_VALIDATION_POLICY] = policy;

SignedDataValidation sdv = new SignedDataValidation();

SignedDataValidationResult sdvr = sdv.verify(signedData, params_);

if(sdvr.getSDStatus() != SignedData_Status.ALL_VALID)
    Console.WriteLine("İmzaların hepsi doğrulanmadı.");
Console.WriteLine(sdvr.ToString());
```

İmzalanan içerik, eğer imza içerisinde ise *BaseSignedData* sınıfının *getContent()* fonksiyonu ile alınabilir.

4.12 İmza Doğrulama Sonucu

SignedDataValidation sınıfının *verify(...)* fonksiyonu, doğrulama sonucu olarak *SignatureValidationResult* tipinde bir nesne döner. *getSDStatus()* fonksiyonu, eğer bütün imzalar doğrulanmış ise *ALL_VALID*, eğer imzalardan en az bir tanesi doğrulanamamışsa *NOT_ALL_VALID* değerini döner. *SignedDataValidation* nesnesinin *toString()* metodu, bütün imzaların doğrulama sonuçlarına ait açıklamaları döner. Eğer özellikle bir imzanın doğrulama sonucu elde edilmek isteniyorsa imza ağacında gezilerek elde edilebilir. Bu ağaç yapısı, *BaseSignedData* yapısındaki imzaların veri yapısı ile aynıdır. Örneğin birinci paralel imzanın, birinci seri imzasına ve bu imzanın doğrulama sonucuna aşağıdaki örnek kod ile ulaşılabilir.

Java

```
BaseSignedData bs = getBaseSignedData();
SignedDataValidationResult sdvr = getValidationResult();
SignerfirstCounterSigner = bs.getSignerList().get(0).getCounterSigners().get(0);
SignatureValidationResult firstCounterSignerVR = sdvr.
    getSDValidationResults().get(0).
    getCounterSigValidationResults().
    get(0);
```

C#

```
BaseSignedData bs = getBaseSignedData();
SignedDataValidationResult sdvr = getValidationResult();
Signer firstCounterSigner = bs.getSignerList()[0].getCounterSigners()[0];
SignatureValidationResult firstCounterSignerVR = sdvr.getSDValidationResults()[0].

    getCounterSigValidationResults()[0];
```

Her bir imzanın doğrulama sonuçları, *SignatureValidationResult* nesnelerinde tutulur. Bir imzacının doğrulama sonucundan, seri imzacılarının doğrulama sonuçlarına da erişilebilir. *SignatureValidationResult* nesnelerinin *toString()* fonksiyonları, imzacının ve seri imzacılarının doğrulama kontrolcülerinin sonuçlarını döner. Eğer sadece o imzaya ait kontrolcülerin açıklamaları elde edilmek isteniyorsa *getValidationDetails()* fonksiyonu kullanılır. İmza tipi geliştikçe kontrol edilmesi gereken yapı ve sertifika artmaktadır. Zaman damgaları, imza yapısına bir imza olarak eklendiklerinden dolayı ayrıca bir imza olarak kontrol edilmektedirler.

SignatureValidationResult nesnesinden bir imzanın doğrulama sonucu, *getSignatureStatus()* fonksiyonu ile *SignatureStatus* yapısında alınabilir. Eğer imza doğrulama sonucu *INCOMPLETE* ise sertifika doğrulama verisine ulaşılammıştır.

4.13 Ön Doğrulama

Bir imza atıldıktan sonra imza doğrulamada kullanılacak olan sertifika iptal bilgilerinin güncellenebilmesi için belirli bir süre geçmesi gerekmektedir. Bu süreye **kesinleşme süresi** (grace period) denilmektedir. API'de bu süre P_GRACE_PERIOD parametresi ile ayarlanabilir; varsayılan değeri 86400 saniye yani 24 saattir.

Bir imza doğrulanmaya çalışıldığında, kesinleşme süresi geçmese bile imza ve sertifika doğrulanabilmektedir. Ön doğrulama denilen bu doğrulama, bir kesinlik içermemektedir. Kesin bir doğrulama yapılabilmesi için kesinleşme süresinin geçmesi gerekmektedir. Bir imza doğrulamasının, ön doğrulama olup olmadığını aşağıdaki örnek kod ile öğrenebilirsiniz. Ön doğrulama için olgunlaşmamış anlamına gelen PREMATURE kelimesi, kesin doğrulama (kesinleşmiş imza) için olgunlaşmış anlamına gelen MATURE kelimesi kullanılmaktadır.

SignatureValidationResult nesnesinin *getValidationState()* fonksiyonu ile ön doğrulama yapıp yapılmadığı bilgisi alınabilir.

Java

```
SignedDataValidationResultsdvr = getValidationResult();

if(sdvr.getSDValidationResults().get(0)
        .getValidationState() == ValidationState.PREMATURE)
    System.out.println("Ön doğrulama yapıldı.");
```

C#

```
SignedDataValidationResult sdvr = getValidationResult();

if(sdvr.getSDValidationResults()[0].getValidationState() ==
ValidationState.PREMATURE)
    Console.WriteLine("Ön doğrulama yapıldı.");
```

Ön doğrulama ile kesin doğrulama arasında doğabilecek fark; ön doğrulama sırasında geçerli olan bir sertifikanın, kesin doğrulamada iptal edilmiş olmasıdır. Kullanıcı akıllı kartını çaldırdığında bu senaryo ile karşılaşılabilir.

ÇiSDuP için kesinleşme süresi kısa olabilmektedir. Yalnız iptal bilgileri için SİL kullanılıyorsa bu süre uzayabilmektedir.

E-İmza Profilleri dokümanı, ÇiSDuP kullanıldığında kesinleşme süresi çok kısa olduğundan bu sürenin yok sayılabileceğini öngörüyor. ÇiSDuP kullanılarak sertifika doğrulama işlemi yapıldığında, kesinleşme süresi kadar beklenmediğinden büyük avantaj sağlanmaktadır.

4.14 Ayırık İmzanın Doğrulanması

Ayrık imza doğrulanırken imzalanan dokümanın parametre olarak verilmesi gerekmektedir. P_EXTERNAL_CONTENT parametresine *ISignable* türünden nesne verilmelidir.

Java

```
byte[] signedData = AsnIO.dosyadanOKU(SIGNATURE_FILE);
ISignable content = new SignableFile(new File(CONTENT_FILE));

ValidationPolicy policy =
PolicyReader.readValidationPolicy(new FileInputStream(POLICY_FILE));
Hashtable<String, Object> params = new Hashtable<String, Object>();
params.put(EParameters.P_CERT_VALIDATION_POLICY, policy);
params.put(EParameters.P_EXTERNAL_CONTENT, content);

SignedDataValidation sdv = new SignedDataValidation();
SignedDataValidationResult sdvr = sdv.verify(signedData, params);

if(sdvr.getSDStatus() != SignedData_Status.ALL_VALID)
    System.out.println("İmzaların hepsi doğrulanmadı.");

System.out.println(sdvr.toString());
```

C#

```
byte[] signedData = AsnIO.dosyadanOKU(SIGNATURE_FILE);
ISignable content = new SignableFile(new FileInfo(CONTENT_FILE));

ValidationPolicy policy =
PolicyReader.readValidationPolicy(new FileStream(POLICY_FILE,
FileMode.Open, FileAccess.Read));
Dictionary<String, Object> params_ = new Dictionary<String, Object>();
params_[EParameters.P_CERT_VALIDATION_POLICY] = policy;
params_[EParameters.P_EXTERNAL_CONTENT] = content;

SignedDataValidation sdv = new SignedDataValidation();
SignedDataValidationResult sdvr = sdv.verify(signedData, params_);

if(sdvr.getSDStatus() != SignedData_Status.ALL_VALID)
    Console.WriteLine("İmzaların hepsi doğrulanmadı.");

Console.WriteLine(sdvr.ToString());
```

4.14.1 Ayırık İmzanın Bütünleşik İmzaya Çevrilmesi

İmzalanan içerik, *attachExternalContent()* fonksiyonuna verilerek ayırık imza bütünleşik imzaya dönüştürülebilir.

Java

```
byte[] input = AsnIO.dosyadanOKU(AYRIK_IMZA);
BaseSignedData bs = new BaseSignedData(input);

File file = new File(IMZALANAN_ICERIK);
ISignable signable = new SignableFile(file, 2048);
bs.attachExternalContent(signable);
```

C#

```
byte[] input = AsnIO.dosyadanOKU(AYRIK_IMZA);
BaseSignedData bs = new BaseSignedData(input);

FileInfo file = new FileInfo(IMZALANAN_ICERIK);
ISignable signable = new SignableFile(file, 2048);
bs.attachExternalContent(signable);
```

4.15 Sertifika Doğrulama

İmza atarken ve imza doğrulama sırasında, CMS Signature kütüphanesi imzacıların sertifikalarını doğrular. Sertifika doğrulama işleminin detayları için [Sertifika Doğrulama](#) bölümüne bakılabilir. Bu dokümanda ayrıntılı bir şekilde açıklanan ve sertifikaların doğrulama işlemlerinin nasıl yapılacağını belirten sertifika doğrulama politika dosyası, EParameters.P CERT_VALIDATION_POLICY parametresi ile *ValidationPolicy* nesnesi tipinde verilir.

4.15.1 İmzacıların Alınması

İmza yapısı, *BaseSignedData* sınıfı tarafından işlenmektedir. Bu yapıda seri ve paralel imzacılar, bir ağaç yapısında bulunmaktadır. *BaseSignedData* sınıfının *getSignerList()* fonksiyonu ile birinci seviye imzacılar alınmaktadır. Sadece paralel imza atılmışsa *getSignerList()* fonksiyonu ile bütün imzacılar alınmış olur. Seri imzacıları almak için ise seri imzacıları alınmak istenen imzacının *getCounterSigners()* fonksiyonu çağırılmalıdır. Eğer imza seviyeleri önemli değilse *BaseSignedData* sınıfının *getAllSignerList()* fonksiyonu kullanılarak bütün imzacılar alınabilir. Daha ayrıntılı bilgi için örnek kodlar içinde yer alan *SignersInJTree* sınıfını inceleyebilirsiniz.

İmza işlemlerinde, imza atan kişiyi tanımlama işlemi kişinin sertifikası üzerinden yapılmaktadır. Yalnız genel davranış olarak sertifika, imza yapısına konur. MA3 API kütüphanesinde de sertifika imza yapısına eklenmektedir. Sertifikadan, kişinin ismi ve Türkiye için T.C. kimlik numarası alınabilir.

Java

```
BaseSignedData bsd = new BaseSignedData(signedData);
ECertificate cert = bsd.getSignerList().get(0).getSignerCertificate();

if(cert == null){
    System.out.println("İmzacı bilgisi yok.");
} else {

    System.out.println("İsim & Soyisim: " +
        cert.getSubject().getCommonNameAttribute());

    System.out.println("TC Kimlik No: " +
        cert.getSubject().getSerialNumberAttribute());
}
```

C#

```
BaseSignedData bsd = new BaseSignedData(signedData);
ECertificate cert = bsd.getSignerList()[0].getSignerCertificate();

if(cert == null)
{
    Console.WriteLine("İmzacı bilgisi yok.");
}
else
{
    Console.WriteLine("İsim & Soyisim: " +
        cert.getSubject().getCommonNameAttribute());

    Console.WriteLine("TC Kimlik No: " +
        cert.getSubject().getSerialNumberAttribute());
}
```

4.16 İmza Tipleri Arasında Dönüşüm

İmza tipleri arasında dönüşüm işlemi, *BaseSignedData* sınıfı aracılığı ile yapılabilir. Öncelikle imzalanmış verinin imzacıları, *BaseSignedData.getSignerList()* fonksiyonuyla imzalanmış veri içinden alınır. İmza tipi değiştirilmek istenen imzacı, liste içinden bulunur ve imzacının *convert()* fonksiyonu çağırılır.

BES imza tipinden EST imza tipine dönüşüm yapan örnek kod bloğu:

Java

```
byte[] inputBES = AsnIO.dosyadanOKU(BES_SIGNATURE_FILE);

BaseSignedData sd = new BaseSignedData(inputBES);

HashMap<String, Object> params = new HashMap<String, Object>();

//necessary for getting signaturetimestamp
params.put(EParameters.P_TSS_INFO, getTSSettings());

ValidationPolicy policy = PolicyReader.readValidationPolicy(
new FileInputStream(POLICY_FILE));

//necessary for validating signer certificate according to time of
//signaturetimestamp
params.put(EParameters.P_CERT_VALIDATION_POLICY, policy);

sd.getSignerList().get(0).convert(ESignatureType.TYPE_EST, params);

AsnIO.dosyayaz(sd.getEncoded(), CONVERTED_TO_EST_FILE);
```

C#

```
byte[] inputBES = AsnIO.dosyadanOKU(BES_SIGNATURE_FILE);
BaseSignedData sd = new BaseSignedData(inputBES);
Dictionary<String, Object> params_ = new Dictionary<String, Object>();

//necessary for getting signaturetimestamp
params_[EParameters.P_TSS_INFO] = getTSSettings();

ValidationPolicy policy = PolicyReader.readValidationPolicy(new
FileStream(POLICY_FILE, FileMode.Open, FileAccess.Read));

//necessary for validating signer certificate according to time of
//signaturetimestamp
params_[EParameters.P_CERT_VALIDATION_POLICY] = policy;
sd.getSignerList()[0].convert(ESignatureType.TYPE_EST, params_);
AsnIO.dosyayaz(sd.getEncoded(), CONVERTED_TO_EST_FILE);
```

4.17 İmza Zamanının Belirlenmesi

API'de imza zamanının belirlenmesi farklı yöntemlerle gerçekleştirilebilir.

- Zaman damgası
- *Signingtime* imza özelliği kullanılarak imza zamanının belirlenmesi
- İmza zamanının dışarıdan verilen zaman parametresine göre belirlenmesi

Bu yöntemlerden en güvenli ve geçerli olanı zaman damgası yöntemidir.

4.18 Zaman Damgası

Zaman damgası, üzerinde bulunduğu verinin belirli bir tarihteki varlığını garantiler. Bizim uygulamamızda zaman damgası, oluşturduğunuz imzaya bağlı olarak oluşturulduğundan, imzanızın gerçekten o tarihte var olduğunu ispatlamak için kullanılır. Zaman damgası, güvenilir bir servis sağlayıcısı olan zaman damgası otoritelerinden alınır. Tüm ESHS'ler bu hizmeti vermektedirler.

İmza tarihinin önemli olduğu uygulamalarda zaman damgasının kullanımı büyük önem arz etmektedir. Çünkü zaman damgası olmayan bir imza üzerindeki zaman bilgisi, kullanıcının belirleyebildiği ve genelde kullanıcının sistem saatinden alınmış olan bir zaman bilgisidir. Dolayısıyla imzayı oluşturan kişi, zaman bilgisini de istediği gibi belirleyebilir. Sertifika iptal durumlarında da imzanın, sertifika iptal edilmeden önce atıldığından emin olunamaz.

Zaman damgası ise imzanın o tarihten (zaman damgası otoritesinin verdiği tarihten) önce var olduğunu garanti eder.

EST ve üzeri imza türleri zaman damgası içermektedir. API'nin zaman damgası alması için zaman damgası sunucu ayarlarının parametreler yardımıyla API'ye verilmesi gerekmektedir.

Java

```
TSSettings tsSettings = new TSSettings("http://tzd.kamusm.gov.tr", 1, "12345678",
DigestAlg.SHA256);
params.put(EParameters.P_TSS_INFO, tsSettings);
```

C#

```
TSSettings tsSettings = new TSSettings("http://tzd.kamusm.gov.tr", 1, "12345678",
DigestAlg.SHA256);
params_[EParameters.P_TSS_INFO] = tsSettings;
```

Zaman damgası ayarlarından ilki zaman damgası adresi, ikincisi kullanıcı numarası, üçüncüsü kullanıcı şifresi ve dördüncüsü de zaman damgasının özet algoritmasıdır.

4.18.1 İmzadaki Zaman Damgasından İmza Zamanının Alınması

İmza zamanının alınabilmesi için imzanın zaman damgası içermesi ve dolayısıyla da imza türünün en az EST olması gerekmektedir.

EST üzeri imza türleri, *EST* sınıfından türediğinden *EST* sınıfının fonksiyonu kullanılabilir. Bu fonksiyondan dönen zaman, *id_aa_signatureTimeStampToken* özelliğinden alınan zaman bilgisidir.

Java

```
byte[] input = AsnIO.dosyadanOKU(ESA);
BaseSignedData bs = new BaseSignedData(input);
EST estSign = (EST)bs.getSignerList().get(0);
Calendartime = estSign.getTime();
```

C#

```
byte[] input = AsnIO.dosyadanOKU(ESA);
BaseSignedData bs = new BaseSignedData(input);
EST estSign = (EST)bs.getSignerList()[0];
DateTime? time = estSign.getTime();
```

Eğer kullanıcının beyan ettiği imza saatine güveniliyorsa *AttributeOIDs.id_signingTime* özelliği kullanılabilir. Yalnız imzadaki *AttributeOIDs.id_signingTime* özelliği zorunlu bir alan değildir ve bu yüzden imza içinde bulunmayabilir.

Java

```
byte[] input = AsnIO.dosyadanOKU(BESwithSIGNING_TIME);
BaseSignedData bs = new BaseSignedData(input);
List<EAttribute> attrs = bs.getSignerList().
    get(0).getSignedAttribute(AttributeOIDs.id_signingTime);
Calendar time = SigningTimeAttr.toTime(attrs.get(0));
System.out.println(time.getTime().toString());
```

C#

```
byte[] input = AsnIO.dosyadanOKU(BESwithSIGNING_TIME);
BaseSignedData bs = new BaseSignedData(input);
List<EAttribute> attrs = bs.getSignerList()[0].
    getSignedAttribute(AttributeOIDs.id_signingTime);
DateTime? time = SigningTimeAttr.toTime(attrs[0]);
Console.WriteLine(time.Value.ToString());
```


Profesyonel kullanıcılar, *AttributeOIDs* sınıfında bulunan özelliklerle diğer zaman damgası bilgilerini de alabilirler. Örnek olarak aşağıda arşiv tipi imza için kullanılan zaman damgası özelliği alınmıştır.

Java

```
byte[] input = AsnIO.dosyadanOKU(ESA);
BaseSignedData bs = new BaseSignedData(input);
List<EAttribute> attrs = bs.getSignerList().get(0).getUnsignedAttribute(
    AttributeOIDs.id_aa_ets_archiveTimestamp);
List<EAttribute> attrsV2 = bs.getSignerList().get(0).getUnsignedAttribute(
    AttributeOIDs.id_aa_ets_archiveTimestampV2);
attrs.addAll(attrsV2);
for(EAttribute attribute : attrs)
{
    Calendar time = ArchiveTimeStampAttr.toTime(attribute);
    System.out.println(time.getTime().toString());
}
```

C#

```
byte[] input = AsnIO.dosyadanOKU(ESA);
BaseSignedData bs = new BaseSignedData(input);
List<EAttribute> attrs = bs.getSignerList()[0].getUnsignedAttribute(
    AttributeOIDs.id_aa_ets_archiveTimestamp);
List<EAttribute> attrsV2 = bs.getSignerList()[0].getUnsignedAttribute(
    AttributeOIDs.id_aa_ets_archiveTimestampV2);
attrs.AddRange(attrsV2);
foreach(EAttribute attribute in attrs)
{
    DateTime? time = ArchiveTimeStampAttr.toTime(attribute);
    Console.WriteLine(time.Value.ToString());
}
```

4.18.2 Zaman Damgası Sunucusunun Test Edilmesi

Zaman damgası ayarları verildikten sonra API zaman damgası alma işlemini kendisi yapmaktadır. Geliştiriciler zaman damgasını test etmek için aşağıdaki örnek kodu kullanabilirler. Zaman damgası işlemlerinden *TSClient* sınıfı sorumludur. Bu sınıf ile zaman damgası alınabilir ve kalan kontör miktarı sorgulanabilir.

Java

```
byte[] sha256Digest = new byte[20];
Random rand = new Random();
rand.nextBytes(sha256Digest);
TSClient tsClient = new TSClient();
TSSettings settings = new TSSettings("http://tzd.kamusm.gov.tr", 1,
"12345678".toCharArray(), DigestAlg.SHA256);
tsClient.setDefaultSettings(settings);
System.out.println("Remaining Credit: " +
                    tsClient.requestRemainingCredit(settings));
ETimestampResponse response = tsClient.timestamp(sha256Digest, settings);
ESignedData sd = new ESignedData(response.getContentInfo().getContent());
ETSTInfo tstInfo = new ETSTInfo(sd.getEncapsulatedContentInfo().getContent());
System.out.println("Time Stamp Time" + tstInfo.getTime().getTime());
System.out.println("Remaining Credit:" + tsClient.requestRemainingCredit(settings));
```

C#

```
byte[] sha256Digest = new byte[20];
Random rand = new Random();
rand.NextBytes(sha256Digest);
TSClient tsClient = new TSClient();
TSSettings settings = new TSSettings("http://tzd.kamusm.gov.tr", 1, "12345678",
DigestAlg.SHA256);
tsClient.setDefaultSettings(settings);
Console.WriteLine("Remaining Credit: " + tsClient.requestRemainingCredit(settings));
ETimestampResponse response = tsClient.timestamp(sha256Digest, settings);
ESignedData sd = new ESignedData(response.getContentInfo().getContent());
ETSTInfo tstInfo = new ETSTInfo(sd.getEncapsulatedContentInfo().getContent());
Console.WriteLine("Time Stamp Time" + tstInfo.getTime());
Console.WriteLine("Remaining Credit: " + tsClient.requestRemainingCredit(settings));
```

4.18.3 Zaman Damgası Alma

MA3 API kütüphanesini kullanarak sadece zaman damgası da alabilirsiniz. Bunun için *asn1rt.jar*, *slf4j.jar*, *ma3api-asn.jar*, *ma3api-common.jar*, *ma3api-crypto.jar*, *ma3api-crypto-gnuprovider.jar*, *ma3api-infra.jar* dosyalarına ihtiyacınız vardır.

Java

```
byte[] data = new byte [ ] {0,1,2,3,4,5,6,7,8,9};
byte[] dataTbs = DigestUtil.digest(DigestAlg.SHA256, data);
TSSettings settings = new TSSettings("http://tzd.kamusm.gov.tr", 1, "12345678",
DigestAlg.SHA256);
TSClient tsClient = new TSClient();
EContentInfotoken = tsClient.timestamp(dataTbs, settings).getContentInfo();
```

C#

```
byte[] data = new byte[] { 0, 1, 2, 3, 4, 5, 6, 7, 8, 9 };
byte[] dataTbs = DigestUtil.digest(DigestAlg.SHA256, data);
TSSettings settings = new TSSettings("http://tzd.kamusm.gov.tr", 1, "12345678",
DigestAlg.SHA256);
TSClient tsClient = new TSClient();
EContentInfo token = tsClient.timestamp(dataTbs, settings).getContentInfo();
```

4.19 Parametreler

İmza atarken veya imza doğrulama yaparken uygulama geliştirme arayüzü çeşitli parametrelere ihtiyaç duymaktadır. Bu parametrelerin kullanımı imza türüne veya imza içerisinde yer alan özelliklere göre değişiklik göstermektedir. Kullanılabilecek ana parametreler aşağıda açıklanmaktadır. Kullanılabilecek diğer parametrelerin açıklamalarına *Eparameters* sınıfından ulaşabilirsiniz.

<i>P_EXTERNAL_CONTENT</i>	Ayrık imza doğrulama işleminde doğrulaması yapılacak veriye işaret eder. Atanacak nesne <i>Isignable</i> tipinde olmalıdır.
<i>P_CONTENT_TYPE</i>	İmzalanan içeriğin tip bilgisine işaret eder. Varsayılan değeri ise veri anlamına gelen "1, 2, 840, 113549, 7" nesne belirteçidir (object identifier).
<i>P_CERT_VALIDATION_POLICY</i>	Sertifika doğrulamada kullanılacak olan politika bilgisine işaret eder. Eğer P_VALIDATE_CERTIFICATE_BEFORE_SIGNING parametresine false atanmışsa politika parametresinin atanmasına gerek yoktur. Diğer her durumda politika bilgisine ihtiyaç duyulur.
<i>P_SIGNING_TIME</i>	Doğrulama sırasında imzanın ne zaman atıldığını belirtmek için kullanılır, hukuki bir geçerliliği yoktur. Beyan edilen zaman sadece zaman damgası bulunmayan imzalarda kullanılır. Örneğin BES tipi bir imza 12.12.2010 tarihinde alındı ve alınma tarihi bir yere kaydedildi. Bu tarih daha sonra imzayı doğrularken kullanılabilir.
<i>P_TRUST_SIGNINGTIMEATTR</i>	İmza doğrulama sırasında kullanılır. İmzaya, imzayı atan tarafından eklenen zaman bilgisine olan güveni belirler. <i>Boolean</i> tipinde nesne atanır; varsayılan değeri ise true'dur. Eğer kullanıcının eklediği zaman bilgisine güvenilmeyecekse false atanmalıdır.

P_TSS_INFO	İmza atarken zaman damgası (Time Stamp) kullanılacaksa, <i>TSSettings</i> tipinde bir nesne atanmalıdır. EST ve üzeri imza tiplerinde zaman damgasının kullanılması mecburidir.
P_POLICY_ID	Birbirinden bağımsız iki taraf, imzaların nasıl doğrulanacağı konusunda kendi aralarında çeşitli politikalar belirleyebilirler. Tarafların, imzanın hangi politika ile doğrulanacağı bilgisini imzalanmış dosyaya eklemeleri gerekmektedir. Yalnız bu özelliğin kullanılabilmesi için imza türünün EPES veya daha üzeri bir imza türü olması gerekmektedir. EPES türü imzalarda varsayılan davranış olarak SignaturePolicyIdentifierAttr imzaya eklenmektedir. Diğer tür imzalarda da kullanılabilmesi için SignaturePolicyIdentifierAttr özelliğinin imzaya eklenmesi gerekmektedir. P_POLICY_ID parametresi ise kullanılacak politikayı işaret eder.
P_POLICY_VALUE	P_POLICY_ID parametresinde anlatıldığı gibi bir politika belirleneceği zaman kullanılır. Kullanılacak politikanın kendisine işaret eder.
P_POLICY_DIGEST_ALGORITHM	P_POLICY_ID parametresinde anlatıldığı gibi bir politika belirleneceği zaman kullanılır. SignaturePolicyIdentifierAttr imzalanan bir özellik olduğundan imzalanması sırasında kullanılacak özet bilgisini işaret eder.
P_INITIAL_CERTIFICATES	İmzaların doğrulanması için gerekli olabilecek sertifikaları işaret eder.
P_INITIAL_CRLS	İmzaların doğrulanması için gerekli olabilecek SİL'leri işaret eder.
P_INITIAL_OCSP_RESPONSES	İmzaların doğrulanması için gerekli olabilecek ÇisDuP cevaplarına işaret eder.
P_VALIDATE_CERTIFICATE_BEFORE_SIGNING	Varsayılan durumda, imza atarken ve imza doğrulaması yapılırken sertifika doğrulaması yapılmaktadır. BES ve EST türü imza atarken bu parametreye false verilerek sertifika doğrulaması es geçilebilir. Yalnız bu kullanım önerilmemektedir.

P_REFERENCE_DIGEST_ALG	SigningCertificateV1/V2, CompleteCertificateReferences, CompleteRevocationReferences özellikleri için kullanılacak özet algoritmasını belirler.
P_GRACE_PERIOD	Sertifika doğrulama verisinin kesinleşmesi için geçecek kesinleşme süresini saniye olarak belirtir. Varsayılan değeri 86400 saniyedir (24 saat).
P_REVOCINFO_PERIOD	Sertifika doğrulama verisinin hangi süre aralığında toplanacağını saniye olarak belirtir. Örneğin bu süre 2 günü işaret ediyorsa; imzanın atıldığı tarihten 2 gün sonrasına kadarki sürede geçerli olan doğrulama verisi, sertifika doğrulamada kullanılır. Bu süre en az P_GRACE_PERIOD süresi kadar olmalıdır. Eğer P_REVOCINFO_PERIOD parametresine herhangi bir süre verilmezse, bu süre mümkün olduğunca geniş tutulmaya çalışılacaktır. Bu sürenin geniş tutulması, sertifika iptal durumlarının yakalanmasında bir dezavantaj oluşturmaz. Bu aralığın kısa tutulması ise askı durumlarının yakalanma ihtimalini arttırır.
P_FORCE_STRICT_REFERENCE_USE	İmzada sertifika doğrulama verisi veya referansı varsa, sadece bu referansları ve bu referanslara ait verilerin doğrulama için kullanılacağını belirtir. Ayrıca bu kontrol, doğrulama verilerinin ve sertifika referanslarının sırasını da kontrol eder.
P_VALIDATE_TIMESTAMP_WHILE_SIGNING	Zaman damgası içeren imzalarda, imzalama esnasında alınan zaman damgasının kontrol edilmesini sağlar.

5. XML İMZA

5.1 Giriş

XML İmza, referans ile gösterilen veri ve bir anahtar arasında ilişkiyi kuran xml formatındaki veri yapısıdır. Anahtar nitelikli sertifika aracılığı ile bir kişiye bağlandığında imzanın bir kişiye ait olması sağlanır.

Nitelikli sertifikalar bir akıllı kart aracılığı ile kullanılmaktadır.

İmzanın geçerli olması için sertifikanın da geçerli olması gerekmektedir.

Bu sebepler ile XML imza kavramlarının yanında;

- sertifika doğrulama,
- akıllı kart gibi kütüphanelerin konfigürasyon ve kullanım bilgisi gerekir/gerekebilir.

Yukarıda bahsedilen kütüphanelerle ilgili detaylı bilgi için dokümandaki [Sertifika Doğrulama](#) ve [Akıllı Kart](#) bölümleri incelenebilir.

Kod ve konfigürasyon örnekleri dağıtım dizinindeki alt klasörlerde bulunmektedir.

5.2 XML İmza Çeşitleri

5.2.1 Yapısına Göre

İmzanın ve imzalanan verinin yapısal özelliklerine göre imzalar 3 kategoriye ayrılır.

Enveloping (Zarflayan): Veri, BASE64 ya da XML formatında imza içinde yer alır.

Enveloped (Zarflanmış): XML verisi, içinde imzayı barındırır.

Detached (Ayrık): İmza ve imzalanan veri ayrı ayrıdır. Özellikle büyük boyutlu veri imzalanırken kullanılması tavsiye edilir.

5.2.2 İmza Tipleri

BES (Basit Elektronik İmza): BES, içinde imza zamanına ait bilgi bulundurabilir ancak imza zamanını ispatlama özelliğinden yoksundur.

EPES (Belirlenmiş Politika Temelli Elektronik İmza): EPES tipindeki bir elektronik imza, oluşturma ve doğrulama kurallarını belirleyen bir politikaya sahip olarak yaratılmış BES tipinde bir imzadır.

ES-T (Zaman Damgalı Elektronik İmza): BES veya EPES tipindeki elektronik imzaya zaman damgası eklenerek elde edilen e-imza formatıdır.

ES-C (Doğrulama Verisi Taşıyan Elektronik İmza): Doğrulama verilerinin referanslarını içeren ES-T formatından türetilen bir e-imzadır.

ES-X (Genişletilmiş Elektronik İmza): ES-C formatından türetilen ve sadece doğrulama verilerinin referanslarına veya ES-C'nin tamamına zaman damgası eklenerek elde edilen e-imza formatıdır.

ES-XL (Genişletilmiş Uzun Elektronik İmza): Doğrulama verisini kendi içinde barındıran e-imza tipidir.

ES-A (Arşiv Elektronik İmza): Kriptografik metodların zaman içinde koruyucu özelliğini yitirmesine karşı periyodik olarak alınan zaman damgası ile korunan e-imzadır.

5.3 İmza Atma

5.3.1 Detached

ornekler/src/.../xades/example/structures/Detached sınıfına göz atınız.

```
// create context with working dir
Context context = new Context(BASE_DIR);

// create signature according to context,
// with default type (XADES_BES)
XMLSignature signature = new XMLSignature(context);

// add document as reference, but do not embed it
// into the signature (embed=false)
signature.addDocument("./sample.txt", "text/plain", false);

// add certificate to show who signed the document
signature.addKeyInfo(CERTIFICATE);

// now sign it by using private key
signature.sign(PRIVATE_KEY);
```

5.3.2 Enveloping

ornekler/src/.../xades/example/structures/Enveloping sınıfına göz atınız.

```
// create context with working dir
Context context = new Context(BASE_DIR);

// create signature according to context,
// with default type (XADES_BES)
XMLSignature signature = new XMLSignature(context);

// add document as reference, and keep BASE64 version of data
// in an <Object tag, in a way that reference points to
// that <Object
// (embed=true)
signature.addDocument("./sample.txt", "text/plain", true);

// add certificate to show who signed the document
signature.addKeyInfo(CERTIFICATE);

// now sign it by using private key
signature.sign(PRIVATE_KEY);

signature.write(new FileOutputStream(SIGNATURE_FILENAME));
```

5.3.3 Enveloped

ornekler/src/.../xades/example/structures/Enveloped sınıfına göz atınız.

```
// here is our custom envelope xml
org.w3c.dom.Document envelopeDoc = new Envelope();

// create context with working dir
Context context = new Context(BASE_DIR);

// define where signature belongs to
context.setDocument(envelopeDoc);

// create signature according to context,
// with default type (XADES_BES)
XMLSignature signature = new XMLSignature(context, SignatureType.XAdES_BES, false);

// attach signature to envelope
envelopeDoc.getDocumentElement().appendChild(signature.getElement());

// add document as reference
signature.addDocument("#data1", "text/xml", false);

// add certificate to show who signed the document
signature.addKeyInfo(CERTIFICATE);
```



```
// now sign it by using private key
signature.sign(PRIVATE_KEY);

// output xml,
// this time we dont use signature. Write because we need to write
// whole document instead of signature
Source source = new DOMSource(envelopeDoc);
Transformer transformer = TransformerFactory.newInstance().newTransformer();

// write to file
transformer.transform(source,
new StreamResult(new FileOutputStream(SIGNATURE_FILENAME)));
```

5.4 İmza Doğrulama

Basit bir imza doğrulama kod örneği aşağıda yer almaktadır. Bu örnekte imza, dosyadan yüklenmekte ve imza içerisinde yer alan imzalama sertifikası kullanılarak doğrulanmaktadır.

```
XMLSignature signature = XMLSignature.parse(new FileDocument(new File(FILE_NAME)),
new Context(BASE_DIR)) ;

// no params, use the certificate in key info
ValidationResult result = signature.verify();
```

Bu örnekte sadece ana imzanın doğrulandığına dikkat edilmelidir. Seri imza doğrulaması yapan örnek kod için `ornekler/src/.../xades/example/validation/XadesSignatureValidation` sınıfına göz atabilirsiniz.

5.5 Akıllı Kart İşlemleri

Akıllı kart kullanarak imzalama yapmak için uygun *BaseSigner* sınıfı oluşturulmalıdır. Gizli anahtar, kart içinde bulunduğundan dolayı imza kütüphanesine parametre olarak verilemez.

`tr.gov.tubitak.uekae.esya.api.smartcard.util.SCSignerWithCertSerialNo` ve `tr.gov.tubitak.uekae.esya.api.smartcard.util.SCSignerWithKeyLabel` sınıfları sizin için *BaseSigner* arayüzünü implement etmektedir.

5.6 XML İmza Konfigürasyonu

XML imza kütüphanesinin, imza yaratma ve doğrulama ayarlarına erişim için kullandığı konfigürasyon dosyasıdır. Bu konfigürasyon dosyası içinde kaynaklara erişim için proxy, zaman damgası, kaynak çözücüleri (*Resolver* arayüzü), varsayılan doğrulama ayarları ve algoritmalar yer alır.

```
<?xml version="1.0" encoding="UTF-8"?>
<xml-signature-config>
```

XML imza konfigürasyonu, *xml-signature-config* kök elemanı ile başlar.

5.6.1 Yerelleştirme

Yerelleştirme seçenekleri, *locale* elementi ile ayarlanır.

```
<locale language="EN" country="EN"/>
```

5.6.2 Proxy Ayarları

```
<http>
<proxy-host></proxy-host>
<proxy-port></proxy-port>
<proxy-username></proxy-username>
<proxy-password></proxy-password>
<basic-authentication-username></basic-authentication-username>
<basic-authentication-password></basic-authentication-password>
<connection-timeout-in-milliseconds>2000</connection-timeout-in-milliseconds>
</http>
```

Eğer proxy üzerinden bağlantı kurulacaksa ilgili ayarlar *http* elementi ile yapılır.

5.6.3 Kaynak Çözücüler

```
<resolvers>
<resolver class="tr.gov.tubitak.uekae.esya.api.xmlsignature.resolver.IdResolver"/>
<resolver class="tr.gov.tubitak.uekae.esya.api.xmlsignature.resolver.DOMResolver"/>
<resolver class="tr.gov.tubitak.uekae.esya.api.xmlsignature.resolver.HttpResolver"/>
<resolver
  class="tr.gov.tubitak.uekae.esya.api.xmlsignature.resolver.XPointerResolver"/>
<resolver class="tr.gov.tubitak.uekae.esya.api.xmlsignature.resolver.FileResolver"/>
</resolvers>
```

Bu alanda imza içindeki URL'lerin çözümünde kullanılacak sınıflar yer almaktadır. Standart bir kullanım için buradaki çözücü tanımlamaları yeterlidir. Farklı protokoller eklenmek istenirse *IResolver* arayüzü gerçekleştirip, konfigürasyonda *resolvers* elementine eklenebilir. Örneğin bir URI'ye karşılık gelen veriyi bir veritabanında tutmak isterseniz, kendi *DatabaseResolver* sınıfınızı yazabilirsiniz.

```
public interface IResolver
{
    boolean isResolvable(String aURI, Context aContext);
    Document resolve(String aURI, Context aContext);
    throws IOException;
}
```

5.6.4 Zaman Damgası

```
<timestamp-server>
<host>http://timestamp_server_address</host>
<userid>fill_id_here</userid>
<password>pass</password>
<digest-alg>SHA-1</digest-alg>
</timestamp-server>
```

userid ve password alanları ESYA tabanlı zaman damgası içindir, açık bir zaman damgası servisi için bu alanları boş bırakınız.

5.6.5 Varsayılan Algoritmalar

```
<algorithms>
<digest-method>http://www.w3.org/2001/04/xmlenc#sha256</digest-method>
</algorithms>
```

5.6.6 Doğrulama Parametreleri

```
<certificate-validation>
<!-- default policy for certificate validation -->
<certificate-validation-policy-file>//path/to/certval-policy.xml</certificate-validation-policy-file>
<!-- grace period is the time that needs to pass to get exact revocation info-->
<grace-period-in-seconds>0</grace-period-in-seconds>
<!-- how old revocation data should be accepted? -->
<last-revocation-period-in-seconds>17280000</last-revocation-period-in-seconds>
```

```

<!-- compare resolved policy with the one at policy uri, if indicated -->
<check-policy-uri>false</check-policy-uri>
<!-- Loosening below 2 settings will cause warnings instead of validation failure -->
<!-- referenced validation data must be used for cert validation is set true -->
<force-strict-reference-use>true</force-strict-reference-use>
<!-- validation data must be published after creation ifs set true, requires grace
period for signers -->
<use-validation-data-published-after-creation>false</use-validation-data-published-
after-creation>
<!-- used to provide validation profile information, P1_1 refers to turkish
profile#1 and so on--->
<validation-profile>P1_1</validation-profile>

<tolerate-signing-time-in-seconds>300</tolerate-signing-time-in-seconds>
</certificate-validation>

```

certificate-validation-policy-file: Sertifika doğrulamada kullanılacak konfigürasyon dosyasıdır.

grace-period-in-seconds: Grace period, saniye cinsindendir ve doğrulama verisinin olgunlaşması için beklenmesi gereken süredir.

last-revocation-period-in-seconds: Doğrulama verisinin üretim zamanı ile doğrulama zamanı arasında ne kadar süre olabileceğini gösterir.

force-strict-reference-use: Referanslarla işaret edilen iptal bilgisi doğrulamada kullanılmalıdır. Güvenlik açısından normal şartlarda bu değer true olmalıdır.

use-validation-data-published-after-creation: Doğrulama verisinin imza tarihinden sonra üretilmesini şart koş anlamına gelmektedir. Varsayılan değeri true'dur. Sağlıklı ve güvenilir bir imzada, doğrulama verisi imza tarihinden sonra yayınlanmış olmalıdır.

validation-profile: Opsiyonel bir parametredir. Kullanıldığında parametre değerinde belirtilen profil bilgisine göre doğrulama yapılır.

- İmza oluşturulurken SignaturePolicyIdentifier özelliği ile eklenmiş profil bilgisi varsa, bu profil bilgisine göre doğrulama yapılmayıp parametrede belirtilen profil bilgisine göre doğrulama yapılır.
- Parametrenin alabileceği değerler P1_1 (P1-Anlık-İmza Profili), P2_1 (P2-Kısa Süreli-İmza Profili), P3_1 (P3-Uzun Süreli-İmza Profili) ve P4_1 (P4-Uzun Süreli-İmza Profili)'dir.

tolerate-signing-time-in-seconds: Beyan edilen zaman alınan zaman damgasından sonra olamaz. Bazı durumlarda kullanıcı makinası zamanı hatalı olduğu için bu durum oluşmaktadır. Bu durumu tolere etmek için bu parametreyi kullanabilirsiniz. Varsayılan değeri 300 saniyedir.

5.6.7 Doğrulayıcılar

XML imza doğrulayıcı sınıflar, validation/validators tagları arasında yer almaktadır. Normalde bu kısımları değiştirmeniz gerekmez. Ekstra(custom) doğrulayıcılar geliştirmek isterseniz bu kısma ekleme yapmak için örnek konfigürasyonları inceleyebilirsiniz. Dikkat etmeniz gereken tek nokta, profil tanımlarında ***inherit-validators-from*** özelliği aracılığıyla profiller arası ortak doğrulayıcı kullanımı yapılabilmesidir.

5.7 Sertifika Doğrulama

İmza atma ve imza doğrulama sırasında CMS Signature kütüphanesi imzacıların sertifikalarını doğrular. Sertifikaların doğrulama işlemleri sertifika doğrulama politikasına göre yapılmaktadır.

Sertifika doğrulama ile ilgili detaylı bilgi için dokümandaki [Sertifika Doğrulama](#) bölümü incelenebilir.

Dağıtım klasöründeki config dizini altında, örnek sertifika doğrulama konfigürasyon dosyaları yer almaktadır.

5.8 XAdES Çoklu İmza Atma

Gerçek hayatta olduğu gibi, elektronik bir dokümanda da birden fazla imzanın yer alması mümkündür.

Konu ile ilgili örnekler, `ornekler/src/.../xades/example/multiple` dizininde bulunmaktadır.

`esya.api.xmlsignature.SignedDocument.java` sınıfı çoklu imzaları kolayca yönetmek için yazılmıştır. Bu sınıf yardımıyla aynı doküman üzerinde birden fazla verinin ve imzanın bulunduğu xml formatlı bir imza oluşturulabilir.

```
<?xml version="1.0" encoding="UTF-8"?>
<ma3:envelope xmlns:ma3="http://uekae.tubitak.gov.tr/xml/signature#"
xmlns:ds="http://www.w3.org/2000/09/xmldsig#">
  <ma3:data>
    <ma3:data-item>...</ma3:data-item>
    <ma3:data-item>...</ma3:data-item>
    ...
  </ma3:data>
  <ma3:signatures>
    <ds:signature>...</ds:signature>
    <ds:signature>...</ds:signature>
    ...
  </ma3:signatures>
</ma3:envelope>
```

5.8.1 Paralel İmza

Aynı doküman içindeki birbirinden bağımsız imzalardır.

Aşağıdaki örnekte *SignedDocument* sınıfı kullanılarak önce imzalanacak veri dokümana eklenmekte, daha sonra bu veriyi imzalayan birbirinden bağımsız iki imza oluşturulmaktadır. Örnek kodlar, *ParallelEnveloped* sınıfı içerisinde bulunmaktadır.

```
Context context = new Context(BASE_DIR);

SignedDocument signatures = new SignedDocument(context);

Document doc = Resolver.resolve("./sample.txt", context);
String fragment = signatures.addDocument(doc);

XMLSignature signature1 = signatures.createSignature();

// add document as inner reference
signature1.addDocument("#"+fragment, "text/plain", false);

// add certificate to show who signed the document
signature1.addKeyInfo(CERTIFICATE);

// now sign it by using private key
signature1.sign(PRIVATE_KEY);

XMLSignature signature2 = signatures.createSignature();

// add document as inner reference
signature2.addDocument("#"+fragment, "text/plain", false);

// add certificate to show who signed the document
signature2.addKeyInfo(CERTIFICATE);

// now sign it by using private key
signature2.sign(PRIVATE_KEY);

// write combined document
signatures.write(new FileOutputStream(FILENAME_PARALLEL_ENVELOPED));
```

Aşağıdaki örnekte dışarıdaki veriyi imzalayan birbirinden bağımsız iki imza oluşturulmaktadır. Örnek kodlar *ParallelDetached* sınıfı içerisinde bulunabilir.

```
Context context = new Context(BASE_DIR);

SignedDocument signatures = new SignedDocument(context);

XMLSignature signature1 = signatures.createSignature();

// add document as reference, but do not embed it
// into the signature (embed=false)
signature1.addDocument("./sample.txt", "text/plain", false);

// add certificate to show who signed the document
signature1.addKeyInfo(CERTIFICATE);

// now sign it by using private key
signature1.sign(PRIVATE_KEY);

XMLSignature signature2 = signatures.createSignature();

// add document as reference, but do not embed it
// into the signature (embed=false)
signature2.addDocument("./sample.txt", "text/plain", false);

// add certificate to show who signed the document
signature2.addKeyInfo(CERTIFICATE);

// now sign it by using private key
signature2.sign(PRIVATE_KEY);

// write combined document
signatures.write(new FileOutputStream(FILENAME_PARALLEL_DETACHED));
```

Aradaki tek fark imzalanacak verinin, *SignedDocument* yerine referans olarak *XML Signature*'a eklenmesidir.

5.8.2 Seri İmza

Veriyi imzalayan imzanın imzalanmasıdır. Buna **counter signature** denir. Örnek kodlar *CounterDetached* sınıfı içerisinde bulunabilir.

Aşağıdaki örnekte, önceden oluşturulmuş imzanın okunması ve sonrasında seri imza ile imzalanması gösterilmektedir.

```
// read previously created signature, you need to run Detached.java first
Document doc = Resolver.resolve(Detached.SIGNATURE_FILENAME, context);
XMLSignature signature = XMLSignature.parse(doc, context);

// create counter signature
XMLSignature counterSignature = signature.createCounterSignature();

// sign
counterSignature.addKeyInfo(CERTIFICATE);
counterSignature.sign(PRIVATE_KEY);

// signature contains itself and counter signature
signature.write(new FileOutputStream(FILENAME_COUNTER_EXISTING));
```

İki paralel imzanın olduğu bir dokümana, bir de seri imza eklenmek istenirse:

```
Context context = new Context(BASE_DIR);

// read previously created signature
Document signatureFile = Resolver.resolve(FILENAME_PARALLEL_DETACHED, context);
SignedDocument signedDocument = new SignedDocument(signatureFile, context);

// get First signature
XMLSignature signature = signedDocument.getSignature(0);

// create counter signature
XMLSignature counterSignature = signature.createCounterSignature();

// sign
counterSignature.addKeyInfo(CERTIFICATE);
counterSignature.sign(PRIVATE_KEY);

// signed doc contains both previous signature and now a counter signature
// in first signature
signedDocument.write(new FileOutputStream(FILENAME_PARALLEL_COUNTER));
```

Yukarıda yer verilen iki örnek arasındaki temel fark, paralel imzaların *SignedDocument* içerisinde tutuluyor olmasıdır.

5.8.3 P1: Anlık - İmza Profili

Anlık doğrulama gerektiren güvenlik ihtiyacı düşük seviyede olan uygulamalarda kullanılır. İmza, doğrulayıcının eline geçtiği an, imza zamanı olarak sayılır. Gelecekte imzayı tekrar doğrulama ihtiyacı olmayacak senaryolarda tercih edilmelidir.

```
// create signature
XMLSignature signature = new XMLSignature(createContext());
signature.addDocument("./sample.txt", "text/plain", false);
signature.addKeyInfo(CERTIFICATE);

// set time now It is better to use TimeProvider!
signature.setSigningTime(Calendar.getInstance());

signature.sign(PRIVATE_KEY);

signature.write(new FileOutputStream(SIGNATURE_FILENAME_P1));
```

Bu imza profili, tanımlanmış bir imza politikasına referans vermemektedir.

5.8.4 P2: Kısa Süreli - İmza Profili

Kısa süreli kullanım ömrü olan imzalarda, ÇiSDuP erişimi bulunmayan ortamlarda tercih edilmelidir. ÇiSDuP erişimi olan ortamlarda P4 profili kullanılmalıdır.

```
XMLSignature signature = createSignature();

// set policy info defined and required by profile
signature.setPolicyIdentifier(OID_POLICY_P2,
    "Kısa Dönemli ve SİL Kontrollü Güvenli Elektronik İmza Politikası",
    "http://www.eimza.gov.tr/EimzaPolitikaları/216792161015070321.pdf");

signature.sign(PRIVATE_KEY);

// add timestamp
signature.upgradeToXAdES_T();

signature.write(new FileOutputStream(SIGNATURE_FILENAME_P2));
```

5.8.5 P3: Uzun Süreli - İmza Profili

ÇiSDuP erişimi olmayan ortamlarda kullanılabilir. Aksi durumda P3 profilinin, P4 profiline tercih edilebilecek herhangi bir avantajı yoktur, SİL boyutları ve imzadan sonra SİL yayınlanmasını bekleme gerekliliği gibi sebeplerle mecbur kalmadıkça tercih edilmemelidir.

```
XMLSignature signature = createSignature();

// set policy info defined and required by profile
signature.setPolicyIdentifier(OID_POLICY_P3,
    "Uzun Dönemli ve SİL Kontrollü Güvenli Elektronik İmza Politikası",
    "http://www.eimza.gov.tr/EimzaPolitikolari/XML_216792121157521.xml");

signature.sign(PRIVATE_KEY);

signature.upgradeToXAdES_T();

// since P3 requires CRLs as revocation info
// below code should be executed after grace period elapses
signature.upgradeToXAdES_C();
signature.upgradeToXAdES_X1();
signature.upgradeToXAdES_XL();

signature.write(new FileOutputStream(SIGNATURE_FILENAME_P3));
```

5.8.6 P4: Uzun Süreli - İmza Profili

En güvenilir, uzun ömürlü ve sorunsuz imza profilidir. Validasyon verisi imza içinde yer alır.

```
XMLSignature signature = createSignature();

// set policy info defined and required by profile
signature.setPolicyIdentifier(OID_POLICY_P4,
    "Uzun Dönemli ve ÇiSDuP Kontrollü Güvenli Elektronik İmza Politikası",
    "http://www.eimza.gov.tr/EimzaPolitikolari/XML_216792121157531.xml");

signature.sign(SIGNER);

// upgrade
signature.upgradeToXAdES_T();
signature.upgradeToXAdES_C();
signature.upgradeToXAdES_X1();
signature.upgradeToXAdES_XL();

signature.write(new FileOutputStream(SIGNATURE_FILENAME_P4));
```

5.8.7 Arşiv İmza

Uzun süreli imza içerisindeki son zaman damgasını imzalayan sertifikanın ömrü dolmadan önce veya imza içinde kullanılan algoritmalar güvenilirlik derecelerini yitirdikçe, imza üzerine arşiv zaman damgası eklenmelidir.

Arşiv formatına erişmek için X-Long (Profil 4) imza üzerinde aşağıdaki metod kullanılır.

```
signature.upgradeToXAdES_A();
```

Halihazırda arşiv tipindeki imzaya, her arşiv zaman damgası eklenmek istendiğinde ise aşağıdaki metod kullanılır.

```
signature.addArchiveTimeStamp();
```

6. PAdES İMZA

6.1 Giriş

PAdES (Pdf Advanced Electronic Signatures), PDS ISO 32000-1 standardında nasıl imza atılması gerektiğini açıklayan ve ETSI tarafından TS 102 778 teknik spesifikasyon doküman seti ile açıklanmış imza standardıdır.

Bu teknik spesifikasyon dokümanları, ilk kısmı genel bakış olmak üzere 6 parçadan oluşmaktadır. MA3 PAdES kütüphanesi, teknik spesifikasyon dokümanının 5. (Profiller) bölümündeki 3. (BES ve EPES imza) ve 4. (LTV-uzun dönemde doğrulanabilir imza) kısımları desteklemektedir.

Bu kütüphane, imza kütüphanesi arayüzleri ile kullanılmakta ve elektronik imza ortak kütüphanesi tanımları ile;

- ES_BES,
- ES_EPES,
- ES_T,
- ES_XL ve
- ES_A

tiplerinde imza oluşturulabilmektedir.

PAdES kütüphanesi altyapısında iText pdf işleme kütüphanesi kullanılmaktadır. Proje bağımlılıkların için iText kütüphanesi ayrıca temin edilmelidir. PAdES kütüphanesi sadece Java platformu için mevcuttur.

6.2 ES_BES İmza Atma

PDF dosyası, imza konteyneri olarak okunur. Daha sonra *createSignature* metodu ile imza yapısı PDF içerisinde yaratılır.

```
//pdf dokümanı oku
SignatureContainer pc = SignatureFactory.readContainer(SignatureFormat.PAdES,
    new FileInputStream("hello.pdf"), new Context());

//imza ekle
Signature signature = pc.createSignature(SIGNER_CERTIFICATE);
signature.setSigningTime(Calendar.getInstance());
signature.sign(SIGNER);

//dosyaya yaz
pc.write();
```

6.3 İmza Doğrulama

```
//imzalı dosyayı oku
SignatureContainer pc = SignatureFactory.readContainer(SignatureFormat.PAdES,
    new FileInputStream("signed-est.pdf"), new Context());

//dogrula
ContainerValidationResult cvr = pc.verifyAll();

System.out.println(cvr);
```

6.4 ES_T İmza Atma

```
PAdESContext context = new PAdESContext();
context.setSignWithTimestamp(true);

//pdf'i oku
SignatureContainer pc = SignatureFactory.readContainer(SignatureFormat.PAdES,
    new FileInputStream("hello.pdf"), context);

//add signature
Signature signature = pc.createSignature(SIGNER.getSignersCertificate());
signature.setSigningTime(Calendar.getInstance());
signature.sign(SIGNER);

//dosyaya yaz
pc.write(new FileOutputStream("signed-est.pdf"));
```

6.5 LTV (ES_A) İmza Atma

```
PAdESContext context = new PAdESContext();

//ilk imzayı ES_T olarak at
context.setSignWithTimestamp(true);

SignatureContainer pc = SignatureFactory.readContainer(SignatureFormat.PAdES,
    new FileInputStream("hello.pdf"), context);

//imza ekle
Signature signature = pc.createSignature(SIGNER.getSignersCertificate());
signature.setSigningTime(Calendar.getInstance());
signature.sign(SIGNER);

//imzayı LTV tipine upgrade et
signature.upgrade(SignatureType.ES_A);

//dosyaya yaz
pc.write(new FileOutputStream("signed-lta.pdf"));
```

7. ASİC E-İMZA

7.1 Giriş

MA3 ASiC kütüphanesi ile ETSI TS 102 918 standardına uygun şekilde imza paketleri oluşturulabilmektedir. Bu sayede bir ya da daha fazla imza, imzalanan veri(ler) ve doğrulama verileri bir zip dosyası içinde taşınabilmektedir.

7.2 Gerekler

MA3 API ASiC Signature kütüphanesinin kullanılabilmesi için lisans dosyasına, sertifika doğrulama politikası ve sertifika deposu dosyalarına ihtiyacınız vardır. Bunun yanında CAdES ya da XAdES imza atma kütüphaneleri atılacak imza türüne göre ihtiyaçlar arasındadır.

İmza doğrulama işlemi için ise yukarıdaki dosyalarla birlikte MA3 kütüphanesi yeterli olacaktır. Kanuni geçerliliği olan nitelikli imzaların atılabilmesi için güvenli bir donanım kullanılması zorunludur. Genel kullanım olarak akıllı kart kullanılmaktadır. Akıllı karta erişilebilmesi için akıllı kart okuyucu sürücüsünün ve akıllı kart sürücüsünün kurulması gerekmektedir. Akıllı kart üreticilerinin sağladığı bir kart izleme programı ile bilgisayarın karta erişimi sağlanabilir ve kart içeriği kontrol edilebilir.

7.3 Kavramlar

Paket tipi:

```
public enum PackageType {
    ASiC_S,
    ASiC_E
}
```

Basit (ASiC_S)	Paket içinde bir imza ve veri bulunur.
Extended (ASiC_E)	Paket içinde bir ya da birden fazla imza ve yine bir ya da birden fazla veri bulunur. Bir imza birden fazla veriyi imzalamış olabilir.

7.4 Anahtar API Arayüzleri ve Tasarım

SignaturePackageFactory imza paketlerini oluşturmakta kullanılan static metodları barındırmaktadır. *SignaturePackage* sınıfı, içinde *SignatureContainer* ve imzalı veriyi veya verileri barındıran ZIP yapısını temsil eden sınıftır.

7.5 Kütüphane Kullanımı

7.5.1 Basit Paket Oluşturma

```
Context c = new Context();
SignatureFormat format = SignatureFormat.CAdES;
//SignatureFormat.XAdES de olabilir.

SignaturePackage signaturePackage = SignaturePackageFactory
    .createPackage(c, PackageType.ASiC_S, format);

//imzalanacak dosyayı pakete ekle
Signable inPackage = signaturePackage
    .addData(new SignableFile(dataFile, "text/plain"),
        "sample.txt");

SignatureContainer container = signaturePackage.createContainer();
Signature signature = container.createSignature(CERTIFICATE);

//paketteki imzalanacak veriyi imzaya ver(false=veriyi imzanın içine ayrıca ekleme)
signature.addContent(inPackage, false);
signature.sign(SIGNER);

//paketi dosyaya yaz
signaturePackage.write(new FileOutputStream(fileName));
```

7.5.2 Çoklu Paket Oluşturma

Paketi okuyup imza (SignatureContainer) ekleme kod bloğu:

```
//read package from file
SignaturePackage sp = SignaturePackageFactory.readPackage(new Context(), inputFile);

//create new container in package
SignatureContainer sc = sp.createContainer();

//create new signature in container
Signature s = sc.createSignature(CERTIFICATE);
```



```
//get signable from package
s.addContent(sp.getDatas().get(0), false);

s.sign(SIGNER);

//write
sp.write(new FileOutputStream(outFileName));
```

7.5.3 Paket Doğrulama

```
//read package from file
SignaturePackage sp = SignaturePackageFactory.readPackage(new Context(), inputFile);

//doğrula
PackageValidationResult pvr = sp.verifyAll();
System.out.println(pvr);
```

7.5.4 İmza Geliştirme

```
//read package from file
Context c = new Context();
SignaturePackage sp = SignaturePackageFactory.readPackage(c, new File(fileName));

//get first signature container
SignatureContainer sc = signaturePackage.getContainers().get(0);

//get first signature in container
Signature signature = sc.getSignatures().get(0);

//upgrade
signature.upgrade(SignatureType.ES_T);

signaturePackage.write(new FileOutputStream(outFileName));
```

Asic imza kütüphanesinin kullanımı ile ilgili kod örneklerine, indirilen MA3 API Elektronik İmza Kütüphane klasörünün içinde bulunan `ornekler/src/.../asic/example/sign` ve `ornekler/src/.../asic/example/upgrades` dizinlerinden ulaşılabilir.

8. AKILLI KART

8.1 Giriş

Kriptografik işlemlerin güvenli bir ortamda yapılması amacıyla akıllı kartlara ihtiyaç duyulmaktadır. Akıllı kartlar, özel anahtara (private key) dışarıdan erişilmesine izin vermeyerek açık anahtar altyapısı için gerekli güvenliği sağlarlar. Akıllı kart içinde kullanıcının sertifikaları, özel anahtarları ve açık anahtarları bulunmaktadır. Her sertifikanın bir açık anahtarı ve bir özel anahtarı yine kart içinde yer almaktadır. Sertifikalar ve açık anahtarlar kart içinden okunabilmektedir. Özel anahtarlara dışarıdan erişilemediği için anahtar ile yalnızca kart içinde kriptografik işlemler yapılabilir.

MA3 API SmartCard modülü, akıllı kart işlemlerinden sorumlu modüldür. Bu modül yardımıyla PKCS7 yapısında basit imza atılabilir.

8.2 Gereksinimler

SmartCard API'si, *ma3api-smartcard-....jar* ve *ma3api-common-....jar* kütüphanelerine ihtiyaç duymaktadır. Ayrıca kullanılacak akıllı kartın ve akıllı kart okuyucu sürücüsünün sisteme kurulmuş olması gerekmektedir.

.NET SmartCard API 'si ise *ma3api-smartcard.dll* kütüphanesinin yanısıra bağımlı olduğu *ma3api-asn.dll*, *ma3api-common.dll*, *ma3api-crypto.dll*, *asn1rt.dll*, *ma3api-cryptobouncyprowider.dll*, *ma3api-iaik_wrapper.dll*, *log4net.dll* ve *ma3api-pkcs11net.dll* kütüphanelerine ihtiyaç duymaktadır.

8.3 Akıllı Karta Erişim

SmartCard sınıfı, akıllı kart ile ilgili işlemlerden sorumlu sınıftır. *SmartCard* sınıfının çalıştırılabilmesi için hangi akıllı kartın kullanıldığının bilinmesi gerekmektedir. Çünkü akıllı kart işlemleri, akıllı kartın sürücüsü üzerinden yapılmaktadır. Java 6 ile java kütüphaneleri kullanarak kart bilgilerine erişilip hangi kart olduğu belirlenebilmektedir. Java 5 ve .NET'te ise hangi akıllı kartın kullanıldığı bilinmelidir.

Java 6 ve C#

```
Pair<Long, CardType> slotAndCardType = SmartOp.findCardTypeAndSlot();
Long slot = slotAndCardType.getObject1();
SmartCard smartCard = new SmartCard(slotAndCardType.getObject2());
Long session = smartCard.openSession(slot);
```

Eğer görsel bir arayüzün API tarafından gösterilmesi istenmiyorsa; *SmartOp* sınıfının *getCardTerminals()* fonksiyonu ile akıllı kart okuyucularının isimleri alınabilir. Bu isimler üzerinden kart, kullanıcıya seçtirildikten sonra *getSlotAndCardType(String terminal)* fonksiyonuyla kullanıcının seçtiği kartın slot numarası ve kart tipi alınabilir.

Eğer kullanıcıya kart tipine göre akıllı kart seçtirilmek isteniyorsa, *SmartOp* sınıfının *findCardTypesAndSlots()* fonksiyonu ile, bağlı olan bütün kartların slot numaraları ve kart tipleri alınabilir.

Akıllı kart ile işlem yapmaya başlamak için *openSession()* fonksiyonu ile oturum açılmalıdır. Karttan sertifika okumak için login olmaya gerek yoktur. İmzalama veya şifreleme işlemi yapılacaksa karta login olunmalıdır.

8.4 Akis Kartlara Erişim

Akis kartlara, Java 6 kullanıldığında Akis'in java kütüphanesi kullanılarak komut (APDU komutları) gönderilebilmektedir. Sistemde akis sürücüsü yüklü olmasa bile AkisCIF.x.x.x.jar olduğunda karta erişilebilmektedir. AkisCIF üzerinden akıllı karta erişmek, akıllı kart işlemlerinin süresini dolayısıyla imza süresini kısaltmaktadır. Yalnız AkisCIF üzerinden karta erişildiğinde diğer programlar karta erişememektedir. Yeni bir sürüm akis kart kullanmaya başladığınızda AkisCIF'i de yenilemeniz gerekecektir.

Akıllı karta, APDU komutları ile AkisCIF.x.x.x.jar üzerinden erişilmesinden *APDUSmartCard* sınıfı sorumludur. Örnek bir kullanım aşağıdaki gibidir.

Java

```
APDUSmartCard sc = new APDUSmartCard();
long[] slots = sc.getSlotList();
sc.openSession(slots[0]);
List<byte[]> certs = sc.getSignatureCertificates();
CertificateFactory cf = CertificateFactory.getInstance("X.509");
X509Certificate cert = (X509Certificate)cf.generateCertificate(new
ByteArrayInputStream(certs.get(0)));

BaseSigner signer = sc.getSigner(cert, Algorithms.SIGNATURE_RSA_SHA256);
```

Yukarıda da belirtildiği gibi AkisCIF arayüzü bütün kartları desteklemeyebilir. AkisCIF desteklendiğinde AkisCIF ile, desteklenmediğinde ise dll ile aşağıdaki şekilde işlemlerinizi yapabilirsiniz.

Java

```

BaseSmartCard bsc = null;
int index = 0;
String[] terminals = SmartOp.getCardTerminals();
String selectedTerminal = terminals[index];
long slot;

if(APDUSmartCard.isSupported(selectedTerminal))
{
    bsc = new APDUSmartCard();
    slot = index + 1;
}
else
{
    Pair<Long, CardType> slotAndCardType =
        SmartOp.getSlotAndCardType(selectedTerminal);
    slot = slotAndCardType.getObject1();
    bsc = new P11SmartCard(slotAndCardType.getObject2());
}

bsc.openSession(slot);

```

8.5 Akıllı Karttan Sertifikanın Okunması

Akıllı karttan sertifika, *SmartCard* sınıfının *getSignatureCertificates()* veya *getEncryptionCertificates()* fonksiyonları ile okunabilir. Eğer imzalama işlemi yapılacaksa *getSignatureCertificates()* fonksiyonu, şifreleme işlemi yapılacaksa *getEncryptionCertificates()* fonksiyonu kullanılmalıdır. Bu fonksiyonlar sertifikaların kodlanmış hallerini byte olarak dönerler. Eğer MA3 API asn modülü (ma3api-asn-....jar/ma3api-asn.dll) kullanılabiliriyorsa, karttan alınan byte değerlerini anlamlı hale getirmek için *ECertificate* sınıfı kullanılabilir.

Atılacak imzanın kanuni hükümlülüklerinin olması için imzalamada kullanılan sertifikanın nitelikli olması gerekmektedir. Bu kontrol, *ECertificate* sınıfının *isQualifiedCertificate()* fonksiyonu ile yapılabilir.

ECertificate sınıfının *getSubject().stringValue()* fonksiyonu ile sertifikalar birbirinden ayırt edilebilir. Kullanıcı bu bilgi ile seçim yapabilir.

Ayrıca *ECertificate* sınıfının *getSubject().getCommonNameAttribute()* fonksiyonu, sertifika sahibinin ismini dönmektedir. Karttaki sertifikaların isim bilgilerinin hepsi aynı olduğu için bu fonksiyon karttaki sertifikaları ayırt etmek amacıyla kullanılmamalıdır. Kimin imzayı attığını görmek için kullanılabilir.

Aşağıdaki kod bloğu, akıllı kart içinden imzalama sertifikalarını alıp nitelikli olanların Subject alanını ekrana yazmaktadır.

Java

```
List<byte[]> certs = smartCard.getSignatureCertificates(session);
for(byte[] bs : certs)
{
    ECertificate cert = new ECertificate(bs);
    if(cert.isQualifiedCertificate())
        System.out.println(cert.getSubject().stringValue());
}
```

C#

```
List<byte[]> certBytes = sc.getSignatureCertificates(session);
foreach(byte[] bs in certBytes)
{
    ECertificate cert = new ECertificate(bs);
    //cert.isQualifiedCertificate()
    Console.WriteLine(cert.getSubject().getCommonNameAttribute());
}
```

Eğer MA3 API *asn* sınıflarına erişim yoksa, *ECertificate* yerine java'nın *x509Certificate* sınıfı kullanılabilir. *ECertificate* sınıfının *isQualifiedCertificate()* fonksiyonu yerine aşağıdaki örnek kodda gösterildiği gibi bir kontrol yapılabilir. Sertifikaları birbirinden ayırt etmek amacıyla *x509Certificate* sınıfının *getSubjectDN().toString()* metodu kullanılabilir.

Aşağıdaki kod bloğu, akıllı kart içinden imzalama sertifikalarını alıp nitelikli olanların Subject alanını ekrana yazmaktadır.

Java

```
List<byte[]> certs = smartCard.getSignatureCertificates(session);
CertificateFactory cf = CertificateFactory.getInstance("X.509");
String qcStatement = "1.3.6.1.5.5.7.1.3";
for(byte[] bs : certs)
{
    X509Certificate cert = (X509Certificate)cf.generateCertificate(new
    ByteArrayInputStream(bs));
    if(cert.getExtensionValue(qcStatement) != null)
        System.out.println(cert.getSubjectDN().toString());
}
```

8.6 Akıllı Karttaki Nesne Adlarının Okunması

Akıllı kartta bulunan sertifikanın, açık anahtarın ve özel anahtarın her biri nesne olarak adlandırılır. Akıllı karttaki nesnelerin adı ile de işlem yapılabilir. Nesne adları değişken olabileceğinden nesne adları ile işlem yapmak önerilmez. Yalnız bazı durumlarda nesne adları kullanıcıya daha anlamlı gelebilir.

SmartCard sınıfının *getSignatureKeyLabels(...)* ve *getEncryptionKeyLabels(...)* fonksiyonları ile anahtarların adları okunabilir. Eğer anahtar sertifikasının adı, anahtar adı ile aynı ise bu ad ile de sertifika okunabilir. Sertifikanın okunması için *readCertificate(long aSessionID, String aLabel)* fonksiyonu kullanılabilir.

8.7 Akıllı Kartta İmzalama - Şifreleme İşlemlerinin Yapılması

Akıllı kartta şifreleme ve imzalama işlemlerinin yapılması için login olunması gerekmektedir.

SmartCard sınıfının *decryptDataWithCertSerialNo(...)*, *decryptData(...)*, *signDataWithCertSerialNo(...)*, *signData(...)* fonksiyonları kriptografik işlemleri yerine getirmek için kullanılabilir. Akıllı kart ile yapılacak işlemler, özel anahtar (private key) ile yapılacak işlemler olmalıdır. Açık anahtar ile yapılan işlemlerin herhangi bir güvenlik kısıtı olmadığından akıllı kartta yapılmasının bir anlamı yoktur. Özel anahtar kullanıldığı işlemler ise imza atma ve şifrelenmiş verinin şifresinin çözülmesi işlemleridir.

İmzalama ve şifreleme işlemlerini kullanan modüller, *BaseSigner* veya *BaseCipher* arayüzünde imzacılar ve şifreleyiciler istemektedir. Bu yüzden *SCSignerWithCertSerialNo*, *SCSignerWithKeyLabel*, *SCCipherWithCertSerialNo*, *SCCipherWithKeyLabel* sınıfları daha çok kullanılacaktır.

Aşağıdaki örnek kodda sertifika seri numarası ile işlem yapan sınıflar vardır.

Java

```
SCSignerWithCertSerialNo signer = new SCSignerWithCertSerialNo(sc, session, slot,
    signatureCert.getSerialNumber().toByteArray(),
    Algorithms.SIGNATURE_RSA_SHA256);
```

C#

```
BaseSigner signer = new SCSignerWithCertSerialNo(sc, session, slots[0],
    cert.getSerialNumber().GetData(), SignatureAlg.RSA_SHA256.GetName());
```

Aşağıdaki örnek kodda anahtar adı ile işlem yapan sınıflar vardır.

Java

```
SCSignerWithKeyLabel signer = new SCSignerWithKeyLabel(sc, session, slot,
"ahmet.uzun#SIGN0", Algorithms.SIGNATURE_RSA_SHA256);
```

C#

```
BaseSigner signer = new SCSignerWithKeyLabel(sc, session, slot,
"ahmet.uzun#ug.netSIGN0", SignatureAlg.RSA_SHA256.getName());
```

8.8 Akıllı Kart Kütüphanesi Konfigürasyonu

Akıllı kart kütüphanesinde konfigürasyon aracılığı ile:

- Yeni kart tipi tanımlanabilir.
- Kart sürücüsü değiştirilebilir.
- Tanımlı kart tiplerine, kartı tanımaya yarayan yeni ATR değerleri eklenebilir.

Böylece sistemde yeni bir kart (marka/versiyon) tanımlanıp kullanılabilir.

Bu işlemler, API ile birlikte dağıtılan smartcard-config.xml adındaki konfigürasyon dosyası ile yapılmaktadır. Bu dosyada ilk değerler vardır. Bu değerler API içinde gömülü olduğu için yeni bir ayar yapılmayacaksa bu dosyayı kullanmaya gerek yoktur.

Eğer konfigürasyonda değişiklik yapılırsa bu konfigürasyon dosyası *SmartCardConfigParser* aracılığı ile okunmalı ve *CardType* sınıfı bu konfigürasyondan haberdar edilmelidir.

Örnek Kod: Çalışma dizini içindeki smartcard-config.xml ile akıllı kart konfigürasyonu yapma:

```
List<CardTypeConfig> cards = new SmartCardConfigParser().readConfig();
CardType.applyCardTypeConfig(cards);
```

Eğer konfigürasyon dosyası farklı bir isimde ya da başka bir lokasyonda ise *SmartCardConfigParser* sınıfının *InputStream* nesnesi alan metodu da kullanılabilir:

```
List<CardTypeConfig> cards = new SmartCardConfigParser().readConfig(inputStream);
CardType.applyCardTypeConfig(cards);
```

Konfigürasyon ile Kart Tipi Tanımlama

Aşağıda örnekte bir akıllı kart tanımlaması görülmektedir.

```
<card-type name="AKIS">
  <lib name="akisp11"/>
  <atr value="3BBA11008131FE4D55454B41452056312E30AE"/>
  <atr value="3B9F968131FE45806755454B41451112318073B3A180E9"/>
  ...
</card-type>
```

<card-type> elemanı ile kart tipi tanımlanır. <lib> elemanı işletim sistemine özel driver belirtmek için kullanılır.

Eğer 32 ve 64 bit mimariler için farklı driver gerekiyorsa bu ayrım, **arch** özelliği ile aşağıdaki şekilde belirtilir.

```
<card-type name="NCIPHER">
  <lib name="cknfast" arch="32"/>
  <lib name="cknfast-64" arch="64"/>
```

ATR Değeri

Kart tipini anlamak için karta özel ATR değerini bilmek gerekmektedir. ATR ifadesi, **Answer To Reset** ifadesinin kısaltmasıdır. Protokol gereği kart resetlendiğinde, tanıtıcı bir byte dizisi gönderir. Eğer bu byte dizisi biliniyorsa ve karta özel ayırt edici bilgi içeriyorsa (historical bytes) kart tipi tespit edilebilir. Bilinen her bir ATR byte dizisi için <card-type> elemanı içine yukarıdaki örnekte olduğu gibi bir <atr> elemanı eklenmelidir.

Java ile ATR değeri hesaplamak için aşağıdaki kod parçasını kullanabilirsiniz. Kod, ilk terminaldeki (kart okuyucu) akıllı kartın ATR'sini ekrana yazdırmaktadır.

```
Card card = TerminalFactory.getDefault().terminals().list().get(0).connect("*");
ATR atr = card.getATR();
String historicalBytesStr = StringUtil.toString(atr.getHistoricalBytes());
System.out.println("historical bytes >"+historicalBytesStr);
String atrHex = StringUtil.toString(card.getATR().getBytes());
System.out.println("ATR >"+atrHex);
```


8.9 SmartCardManager Sınıfı

Dağıtılan paket içinde örnek kodlar içerisinde *SmartCardManager* sınıfını bulabilirsiniz. Kendinize göre uyarlayabilmeniz için açık kaynak olarak dağıtılmaktadır. Bu sınıf ile temel imza işlemlerinizi gerçekleştirebilirsiniz. *SmartCardManager* sınıfı, aşağıdaki işlemleri sağlayabilir.

- Sisteme bir kart takılı ve kartta belirtilen özellikte bir sertifika varsa doğrudan bu kart ve bu sertifika üzerinden işlem yapar.
- Birden fazla kart takılı ise kullanıcıya kart seçtirir. Birden fazla belirtilen özellikte sertifika yüklü ise kullanıcıya sertifika seçtirir.
- Eğer APDU ile karta erişilmek isteniyorsa ve kart APDU erişimini destekliyorsa APDU ile karta erişim sağlar. C# tarafında APDU erişimi olmadığından sadece pkcs11 erişimi sağlanmaktadır.
- Aynı kart ile imzalama işlemlerinde, sertifikayı ve imzacıyı bellekten çekerek hız açısından kazanım sağlar.
- Bir kart ile işlem yaptıktan sonra eğer yeni bir kart takılmışsa veya işlem yapılan kart çıkartılmışsa, kart ve sertifika seçme işlemlerini tekrarlar.

Örnek bir kullanım aşağıdaki gibidir.

Java

```
//Enable APDU usage
SmartCardManager.useAPDU(true);

//Connect a smartcard. If more than one smart card connected, user selects one of
//them.
SmartCardManager scm = SmartCardManager.getInstance();

//Get qualified certificate. If more than one qualified certificate, user selects
//one of them.
ECertificate cert = scm.getSignatureCertificate(true, false);

//Create signer
BaseSigner signer = scm.getSigner("12345", cert);

/**
 * Create signature
 */

//If not sign again with selected card logout.
scm.logout();

//To select new card and new certificate, call reset.
scm.reset();
```

Yeni bir kartın takılıp takılmadığı, seçili kartın çıkartılıp çıkartılmadığı *getInstance()* metodu içinde kontrol edilmektedir. Her imzalama işleminden önce *SmartCardManager* nesnesini, *getInstance()* metodu ile alınız. Yukarıdaki örnek kodda, işlemler kısa zamanda ardışık olarak yapıldığından nesne bir kere alınmış ve o nesne üzerinden işlem yapılmıştır.

8.10 SmartCard Modülü ile BES Tipi İmza Atılması

PKCS7 yapısı en basit imza yapılarından biridir. *PKCS7Signature* sınıfı, PKCS7 formatında imza atılmasından sorumlu sınıftır. Ayırık imza veya bütünleşik imza atılabilir.

signExternalContent() fonksiyonu ile ayırık imza, *signInternalContent()* fonksiyonu ile de bütünleşik imza atılabilir.

Aşağıdaki örnek kodda, PKCS7 yapısında imzanın nasıl atılacağı gösterilmiştir. Örnekte ayırık imza atılmıştır. *signInternalContent()* fonksiyonu kullanılırsa bütünleşik imza atılacaktır. Bütünleşik imzadan içerik, PKCS7 nesnesinin *getContentInfo().getContentBytes()* fonksiyonu ile alınabilir.

Aşağıdaki örnek kod sadece Java için geçerli olup PKCS7 yapısının, .NET MA3 API tarafında desteği bulunmamaktadır.

Java

```
PKCS7Signature pkcsSignature = new PKCS7Signature();
ByteArrayOutputStream signature = new ByteArrayOutputStream();

SmartCard sc = new SmartCard(CardType.AKIS);
long[] slots = sc.getSlotList();
//sc.getSlotInfo(slots[0]).slotDescription;
long session = sc.openSession(slots[0]);
sc.login(session, "12345");

//Gets first certificate, it must be asked to user if it is more than one
//certificate.
byte[] certBytes = sc.getSignatureCertificates(session).get(0);
CertificateFactory cf = CertificateFactory.getInstance("X.509");
X509Certificate cert = (X509Certificate)cf.generateCertificate(new
ByteArrayInputStream(certBytes));

BaseSigner signer = new SCSignerWithCertSerialNo(sc, session, slots[0],
cert.getSerialNumber().toByteArray(), Algorithms.SIGNATURE_RSA_SHA256);

ByteArrayInputStream bais = new ByteArrayInputStream(toBeSigned);
pkcsSignature.signExternalContent(bais, cert, signature, signer);

Assert.assertEquals(true, validate(new
ByteArrayInputStream(signature.toByteArray()), cert));
```

Java

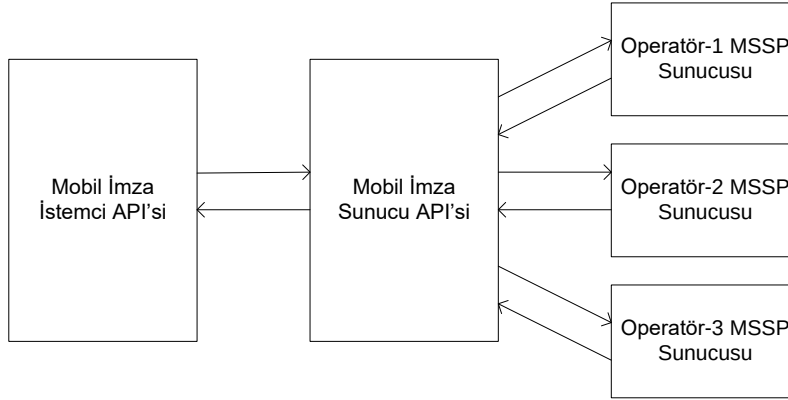
```
PKCS7 p = new PKCS7(signature);

//validates the signature, not the person.
SignerInfo[] signerInfo = p.verify(toBeSigned);

if(signerInfo == null)
    return false;
else
{
    //Checks whether the expected person signed the data.
    return signerInfo[0].getCertificateSerialNumber()
        .equals(cert.getSerialNumber()) == true;
}
```

9. MOBİL İMZA

Mobil imza desteği, API'nin 1.4.2 versiyonu ile birlikte gelmiştir. Mobil imzanın kullanılması için istemci ve sunucu yapısının kurulması gerekmektedir. Mobil servis sağlayıcılar (operatörler), mobil imza isteklerini tanımladıkları bir sunucudan almak istemektedirler. Bunun için mobil imza isteklerinin geleceği sunucunun IP'si, operatör tarafında tanımlanmalı ve istekte bulunacak sunucu, kullanıcı adı ve parolaya sahip olmalıdır.



İmza işlemine girecek veri, istemci tarafında hesaplanır ve istek mobil imza sunucu API'sine gönderilir. Mobil imza sunucu API'si, MSSP sunucusundan istekte bulunur. MSSP sunucusu, isteği imza atmak isteyen kişinin cep telefonuna iletir. Cep telefonundan dönen veri, önce MSSP tarafından mobil imza sunucu API'sine iletilir. Mobil imza sunucu API'si de sonucu mobil imza istemci API'sine iletir ve imza yapısı oluşturulur.

9.1 Mobil İmza İstemci Tarafı

Mobil imza istemci API'si, MA3 İmzalama API Paketi'nde infra modülü içerisinde bulunur. İmza oluştururken mobil imza kullanımında tek fark, akıllı kart erişim işlemlerinde olmaktadır. Akıllı kart yerine cep telefonu kullanılıyor gibi düşünülebilir. Aşağıdaki örnek kod, istemci tarafında mobil imza kullanarak imza atmaktadır. Sadece gri renkte yazılı işlemler normal imzalama işleminden farklıdır.

```

BaseSignedData bs = new BaseSignedData();
bs.addContent(new SignableByteArray("test".getBytes()));

HashMap<String, Object> params = new HashMap<String, Object>();
ValidationPolicy policy = PolicyReader.readValidationPolicy(new
FileInputStream(POLICY_FILE));
params.put(EParameters.P_CERT_VALIDATION_POLICY, policy);

//Create a communication interface for your system.
MSSPClientConnector connector = null;
UserIdentifier user = new PhoneNumberAndOperator("05336564727", Operator.TURKCELL);
BaseSigner mobileSigner = new MobileSigner(connector, user, null,
    "Doc1234 numaralı dokümanı imzayı onaylıyorum.",
    Algorithms.SIGNATURE_RSA_SHA256, null);

bs.addSigner(ESignatureType.TYPE_BES, null, mobileSigner, null, params);

//write the contentinfo to file
AsnIO.dosyayaz(bs.getEncoded(), SIGNATURE_FILE);

```

Sunucu API'si ve istemci API'si arasındaki iletişim için *MSSPClientConnector* arayüzü tanımlanmıştır. Sunucu API'sinin ihtiyacı olan bilgiler, bu arayüz üzerinden gönderilecektir. Farklı teknolojilerle bilgiler iletebileceğinden burada sadece bir arayüz tanımlanmıştır. API kullanıcıları kendi sistemlerine özgü bir iletişim altyapısı kurabilirler.

9.2 Mobil İmza Sunucu Tarafı

Mobil imza API'si kullanılarak yazılacak olan mobil imza sunucusu, MSSP'ye bağlanarak kullanıcı sertifika sorgulamasını yapar ve gönderilmiş olan verinin imzalanmasını sağlar. Bunun için imzalanacak veri, operatör, kullanıcı telefon numarası vb. bilgiler istemci tarafından çağrılıp (örneğin bir web servisi aracılığı ile) bu bilgilerle MSSP'ye imzalama isteği göndermelidir.

İmzalama metodu, kullanıcının göndermiş olduğu imzalanacak yapısal veriyi imzalamalı ve geriye imza değeri dönmelidir.

Örnek olarak imzalanacak verinin base64 değerini, kullanıcının telefonunda görülecek imzalama mesajını, kullanıcı telefon numarasını ve operatörünü alan örnek servis metodu aşağıdaki gibi olabilir. Metodun detaylı kullanımına, MA3 İmzalama API Paketi'ndeki örnek kodlardan bakılabilir.

```

public String SignHash(String hashForSign64, String displayText,
                        String phoneNumber, int iOperator)
{
    Operator mobileOperator = fromInt(iOperator);
    PhoneNumberAndOperator phoneNumberAndOperator =
        new PhoneNumberAndOperator(phoneNumber, mobileOperator);
    MSSParams mobilParams =
        new MSSParams("http://MImzaTubitakBilgem", "*****",
            "www.turkcelltech.com");
    EMSSPRequestHandler msspRequestHandler = new EMSSPRequestHandler(mobilParams);

    byte[] dataForSign = Base64.decode(hashForSign64);
    byte[] signedData;

    try {
        signedData = msspRequestHandler.sign(dataForSign, SigningMode.SIGNHASH,
            phoneNumberAndOperator, displayText, SignatureAlg.RSA_SHA256.getName(),
            null);
    } catch (Exception e) {
        e.printStackTrace();
        return null;
    }

    return Base64.encode(signedData);
}

```

10. ANDROID'DE İMZA ATMA

Android sisteminde temel imza formatında (BES) imza oluşturulabilmektedir. Mevcut sürümde AKİS marka kart ve ACS marka kart okuyucular ile imza oluşturulabilmektedir. İmzalama sırasında kart okuyucunun android kütüphanesi kullanılmaktadır. Mevcut sürümde sadece ACS kart okuyucunun android kütüphanesi kullanılmaktadır. ACS android kütüphanesinin desteklediği kart okuyucular, <http://android.acs.com.hk> adresinden görülebilir.

Android sisteminde imza oluşturulurken kart ile iletişim APDU komutları üzerinden sağlanmaktadır. İmzalama işlemleri için *APDUSmartCard* sınıfı uygun parametrelerle oluşturulmakta ve *getSigner()* metodundan alınan *BaseSigner* nesnesi imzalama kullanılmaktadır. İmzalama işlemlerinde kullanılacak olan *APDUSmartCard* sınıfı oluşturulurken uygun *TerminalHandler* sınıfı oluşturulmalı ve bu kullanılmalıdır. Mevcut sürümde *ACSTerminalHandler* sınıfı oluşturulabilmektedir. Kartla yapılan işlemlerde, android sistemi ilk kullanımda karta erişmek için kullanıcıdan usb erişim onayı istemektedir. Bu onay ekranının görülebilmesi ve düzgün çalışması amacıyla *ACSTerminalHandler* sınıfına usb erişim hakları için oluşturulmuş bir android sınıfı olan *PendingIntent* nesnesi verilmelidir. Android sistemindeki benzer akışlardan dolayı kart işlemleri doğrudan ana gui sınıfında yapılmamalı, *AsyncTask* sınıfından türetilen bir sınıf içerisinde işlemler yapılmalıdır.

Takılı olan kart okuyuculardaki sertifikaları listeleyen ve seçilen dosyayı imzalayan örnek bir android uygulaması, eclipse projesi paketi içerisinde bulunmaktadır. İmzalama için gerekli jar dosyaları, bu örnek eclipse projesine bakılarak görülebilir.

Android imzada test lisansı ile çalışırken sadece test sertifikaları ile işlem yapılabilecek ve işlemlerde 5 sn'lik bir gecikme yaşanacaktır.

Yukarıda bahsedilen akışla ilgili örnek bir fonksiyon aşağıda yer almaktadır. Kod içerisinde gerekli kısımlarda yorumlar bulunmaktadır.

```
public void signWithFirstCertificate()
{
    try {
        //Burada gömülü lisans dosyası yüklenmektedir.
        Resources res = getResources();
        InputStream lisansStream = res.openRawResource(R.raw.lisans);
        LicenseUtil.setLicenseXml(lisansStream);
        lisansStream.close();
        Activity callerActivity = this;

        //ACSTerminalHandler oluşturulurken bunu çağıran Activity parametre olarak
        //verilmelidir.
        ACSTerminalHandler acsTerminalHandler = new
            ACSTerminalHandler((Activity)this);
```

```

//APDUSmartCard sınıfı uygun TerminalHandler sınıfı ile çağrılmalıdır.
APDUSmartCard apduSmartCard = new APDUSmartCard(acsTerminalHandler);

//Kullanıcıdan usb erişim onayı alınabilmesi için oluşturulmuş olan
//PendingIntent nesnesi terminal handler sınıfına verilmelidir.
PendingIntent permissionIntent = PendingIntent.getBroadcast(callerActivity,
0,
new
Intent("tr.gov.tubitak.bilgem.esya.android.signexample.USB_PERMISSION"),
0);
acsTerminalHandler.setPermissionIntent(permissionIntent);

//Akis kart iletişimi için SecureMessaging devre dışı bırakılmalıdır.
apduSmartCard.setDisableSecureMessaging(true);

//Bağlı kart okuyucular okunuyor.
CardTerminal[] terminalList = apduSmartCard.getTerminalList();
if(terminalList == null || terminalList.length == 0)
{
    throw new Exception("Bağlı kart okuyucu sayısı 0");
}
CardTerminal cardTerminal = terminalList[0];
apduSmartCard.openSession(cardTerminal);

//İlk kart okuyucudan sertifika listesi alınıyor.
List<byte[]> signCertValueList = mApduSmartCard.getSignatureCertificates();
if(signCertValueList == null || signCertValueList.size() == 0)
{
    throw new Exception("Kart içerisinde sertifika sayısı 0");
}

//İlk sertifika ile işlem yapılacaktır.
ECertificate signingCert = new ECertificate(signCertValueList.get(0));
String cardPin = "511661";
apduSmartCard.login(cardPin);

//İmzalamada kullanılacak BaseSigner APDUSmartCard sınıfından alınıyor.
BaseSigner signer =
apduSmartCard.getSigner(signingCert.asX509Certificate(),
Algorithms.SIGNATURE_RSA_SHA256);
BaseSignedData bsd = new BaseSignedData();

//İmzalanacak olan dosya yolu
String sourceFilePath = "/tmp/TextForSign.txt";
ISignable content = new SignableFile(new File(sourceFilePath));
bsd.addContent(content);

//Since SigningTime attribute is optional, add it to optional attributes
//list
List<IAAttribute> optionalAttributes = new ArrayList<IAAttribute>();
optionalAttributes.add(new SigningTimeAttr(Calendar.getInstance()));
HashMap<String, Object> params = new HashMap<String, Object>();

```



```
//Android ile imza atılırken sertifika kontrolü devre dışı bırakılmalıdır.  
//Mevcut sürümde sertifika doğrulama desteği bulunmamaktadır.  
params.put(EParameters.P_VALIDATE_CERTIFICATE_BEFORE_SIGNING, false);  
bsd.addSigner(ESignatureType.TYPE_BES, signingCert, signer,  
optionalAttributes, params);  
byte[] signedDocument = bsd.getEncoded();  
String destFilePath = sourceFilePath + ".imz";  
  
//İmzalı hali dosyaya yazılıyor.  
AsnIO.dosyayaz(signedDocument, destFilePath);  
  
apduSmartCard.logout();  
apduSmartCard.closeSession();  
}  
catch (Exception e) {  
    e.printStackTrace();  
}  
}
```

Ek A. Hızlı Başlangıç

Kütüphaneyi kullanabilmeniz için kütüphane ile birlikte lisans dosyasına, sertifika doğrulama politika dosyasına ve sertifika deposu dosyasına, imza atabilmek için de sertifika ve özel anahtara ihtiyacınız vardır.

İndirdiğiniz paket içinde test lisansı bulunmaktadır. Lisans dosyasını, *LicenseUtil.setLicense(...)* fonksiyonu ile verebilirsiniz. [Lisans Ayarları](#) bölümüne bakınız.

Örnek bir politika dosyası paket içersinden çıkmaktadır. Bu politika dosyasını doğrudan kullanmanızda hiçbir sakınca yoktur.

Kullanmanız gereken sertifika deposu dosyası, yine indirdiğiniz paket ile birlikte gelmektedir. Politika dosyasından, sertifika deposunun dosya yolunu ayarlayabilir veya dosya yolu ayarını silerek sertifika deposunu, **user_home** altında oluşturacağınız **.sertifikadeposu** isimli klasörün içerisine **SertifikaDeposu.svt** dosyası olarak kaydedebilirsiniz.

İmza atmanız için sertifika ve özel anahtara ihtiyacınız vardır. Pfx dosyaları sertifikaları ve özel anahtarları şifreli olarak saklayabilmektedir. Test amacıyla kullanılmak üzere bir pfx dosyasını, indirdiğiniz paketin içinde bulabilirsiniz. Dosya ismindeki sayılar pfx dosyasının şifresidir. Bu pfx dosyasını, bir akıllı karta yükleyebilirsiniz veya bu pfx dosyasını doğrudan kullanabilirsiniz. Pfx içinden sertifikayı nasıl alacağınızı ve pfx'ten nasıl imzacı oluşturacağınızı görmek için örnek kodlarda bulunan *PfxSigner* sınıfına bakabilirsiniz.

Ek B. Lisans Ayarları

MA3 Elektronik İmza Kütüphanesi ücretsiz kullanım lisansı ile dağıtılmaktadır. İndirdiğiniz kütüphane paketi içerisinde, lisans klasörü altında **lisans.xml** dosyası bulunmaktadır. Lisansı, windows üzerinde NotePad programı ile görüntüleyebilirsiniz.

Lisans kontrolünün başarıyla sonuçlanması için lisans dosyasının düzgün bir şekilde gösterilmesi gerekmektedir. Kütüphane kullanım durumuna göre lisans.xml dosyası gösterilirken aşağıda sıralanan yöntemler tercih edilebilir.

- Varsayılan olarak *working directory* altında lisans klasörü içindeki **lisans.xml** dosyası aranmaktadır.
- *LicenseUtil.setLicenseXml("...")* fonksiyonu kullanılarak herhangi bir klasörde bulunan **lisans.xml** dosyası verilebilir.
- Lisans dosyasının yolu (Variable value), çevresel değişken (environment variables) olarak kullanıcı değişkenleri (user variables) bölümüne, **ma3LicenseFilePath** ismiyle (Variable name) eklenerek lisans dosyası gösterilebilir.

1. Bakım Sözleşme Bitiş Tarihi

Lisans dosyanızda bakım sözleşme bitiş tarihi ile belirtilen bir alan bulunmaktadır. Bu tarihten daha önce yayınlanan kütüphane sürümlerini kullanmaya devam edebilirsiniz. Bu tarihten sonra yayınlanan kütüphane sürümlerini kullanabilmeniz için bakım sözleşmesi yapmanız gerekecektir.

Ek C. Parola Tabanlı Şifreleme

Bir parola üzerinden verilerinizi şifreleyip aynı parola ile şifrelerinizi çözebilirsiniz. Lisans dosyasını veya sertifika doğrulama politika dosyanızı korumak için kullanabilirsiniz.

Parola tabanlı şifreleme için indirdiğiniz dağıtım paketinde bulunan örnek kodların içindeki *PasswordBaseCipher* sınıfından yararlanabilirsiniz.

Ek D. Log Tutma

MA3 API JAVA Kütüphaneleri, log işlemi için slf4j önyüzünü kullanmaktadır. Bu önyüz kendiliğinden bir loglama kütüphanesi sunmamakta fakat farklı alternatiflerin kullanılmasını desteklemektedir.

Log alabilmek için öncelikle slf4j'nin desteklediği bir log kütüphanesi kullanılmalıdır. Bunlar jul (java.util.logging) veya log4j olabilmekle birlikte slf4j'nin kendi kütüphanesi de kullanılabilir. Eğer jul kullanılacaksa jul için olan bağlayıcı sınıf (slf4j-jdk14-<sürüm>.jar) kullanılmalıdır. Eğer log4j kullanılmak isteniyorsa log4j versiyonuna göre aşağıdaki konfigürasyonlardan yararlanılabilir.

Konfigürasyon ayarlarına gerek kalmadan hızlı bir şekilde log açmak için aşağıdaki kod satırı kullanılabilir. Bu kod satırı logları konsola basar. Java ve C#'da kullanılabilir.

```
BasicConfigurator.configure()
```

Log4j 1.x

Log4j 1.x versiyonları ile log tutmak için referans olarak "slf4j-log4j12-1.x.x.jar" kütüphaneleri eklenmelidir.

Log4j 1.x için *PropertyConfigurator.configure("dosya_ismi")* komutu ile log konfigürasyon dosyası verilebilir. Java ve .Net ortamları için aşağıdaki örnek konfigürasyon dosyalarından yararlanılabilir.

Java

```
#PropertyConfigurator.configure("log4j.properties");
# Set root logger level to DEBUG and its appender to console, rolling, lf5rolling
log4j.rootLogger=debug,rolling
# BEGIN APPENDER: CONSOLE APPENDER (console)
log4j.appender.console=org.apache.log4j.ConsoleAppender
log4j.appender.console.layout=org.apache.log4j.PatternLayout
log4j.appender.console.layout.ConversionPattern=%p %d{DATE} %c{2} (%F:%M:%L) - %m%n
# END APPENDER: CONSOLE APPENDER (console)
# BEGIN APPENDER: ROLLING FILE APPENDER (rolling)
log4j.appender.rolling=org.apache.log4j.RollingFileAppender
log4j.appender.rolling.File=ESYA_API.log
log4j.appender.rolling.MaxFileSize=50MB
log4j.appender.rolling.MaxBackupIndex=20
log4j.appender.rolling.layout=org.apache.log4j.PatternLayout
log4j.appender.rolling.layout.ConversionPattern=%p %d{DATE} %c{2} (%F:%M:%L) - %m%n
# END APPENDER: ROLLING FILE APPENDER (rolling)
```

Jul için, kullanılan jre'nin içindeki lib klasöründe logging.properties dosyası değiştirilerek log konfigürasyonu yapılabilir.

Java

```
handlers= java.util.logging.ConsoleHandler.level= FINE
java.util.logging.FileHandler.pattern = %h/java%u.log
java.util.logging.FileHandler.limit = 50000
java.util.logging.FileHandler.count = 1
java.util.logging.FileHandler.formatter = java.util.logging.XMLFormatter
java.util.logging.ConsoleHandler.level = FINE
java.util.logging.ConsoleHandler.formatter = java.util.logging.SimpleFormatter
com.xyz.foo.level = FINE
```

MA3 API C# Kütüphanesi ise log4net kullanmaktadır. Konfigürasyon dosyası *XmlConfigurator.Configure(new FileInfo("dosya_ismi"))* şeklinde gösterilebilir.

C#

```
<log4net>
<!-- A1 is set to be a ConsoleAppender -->
<appender name="A1" type="log4net.Appender.ConsoleAppender">
<!-- A1 uses PatternLayout -->
<layout type="log4net.Layout.PatternLayout">
<conversionPattern value="%-4timestamp [%thread] %-5level %logger %ndc -
%message%newline" />
</layout>
</appender>
<appender name="FileAppender" type="log4net.Appender.FileAppender">
<file value="ESYA_API.log" />
<appendToFile value="true" />
<lockingModel type="log4net.Appender.FileAppender+MinimalLock" />
<layout type="log4net.Layout.PatternLayout">
<conversionPattern value="%date [%thread] %level %logger %ndc(%F:%L) -
%message%newline" />
</layout>
</appender>
<!-- Set root logger level to DEBUG and its only appender to A1 -->
<root>
<level value="ALL" />
<appender-ref ref="FileAppender" />
</root>
</log4net>
```

Log4j 2.x

Log4j 2.x versiyonları ile log tutmak için referans olarak “log4j-api-2.x.x.jar” ve “log4j-core-2.x.x.jar” ve “log4j-slf4j-impl-2.x.x.jar” kütüphaneleri eklenmelidir.

Örnek olarak aşağıdaki log4j konfigürasyonundan yararlanılabilir. Bu konfigürasyon dosyasının uygulama çalışma dizinine koyulması yeterlidir.

```
<?xml version="1.0" encoding="UTF-8"?>
<Configuration status="debug" strict="true" name="XMLConfigTest"
    packages="tr.gov.tubitak.uekae.esya.api">
    <Properties>
        <Property name="filename">ma3api.log</Property>
    </Properties>

    <Appenders>
        <Appender type="File" name="File" fileName="${filename}">
            <Layout type="PatternLayout">
                <Pattern>%d %p %C{1.} [%t] %m%n</Pattern>
            </Layout>
        </Appender>
    </Appenders>

    <Loggers>
        <Logger name="tr.gov.tubitak.uekae.esya.api" level="debug" additivity="true">
            <AppenderRef ref="File"/>
        </Logger>
    </Loggers>
</Configuration>
```

Ek E. SÖZLÜK

PKI (AAA)	Açık Anahtar Altyapısını (AAA) ifade eder. Kriptolojinin temel çalışma alanlarından birisidir.
MA3	Milli Açık Anahtar Altyapısı'nın kısaltmasıdır.
ESYA	Elektronik Sertifika Yönetim Altyapısı'nın kısaltmasıdır.
X.509	Bir PKI sertifika ve SİL standardıdır (Bkz. RFC 5280).
ÇİSDUP	Çevrimiçi Sertifika Durum Protokolü (OCSP). Sertifika iptal durumunun çevrimiçi olarak sorgulanabilmesini sağlayan RFC 2560'ta tanımlı sorgulama protokolünü ifade eder.
SİL	Sertifika İptal Listesi'nin (CRL) kısaltmasıdır.
ESHS	Elektronik Sertifika Hizmet Sağlayıcı' nın kısaltmasıdır.
Yürürlükteki dizin	Programınızın koştığı dizindir (working directory).
Bağıl adres	Belirli bir adres referans alınarak tanımlanan adrestir (relative path).